Current Line-Following Robot: Construction, Wiring and Control

Jan Balewski, NERSC, LBNL

Updated October 11, 2026

This document describes the current three-sensor, two-wheel robot and its working local line follower. The ASUS network is configured and SSH is verified. The remote Desktop controller is implemented and deployed; motor-disabled checks and short moving line-following runs are recorded. Clock-delay measurements, the telemetry video and the earlier timing results are in the appendices, with long-term neural workloads, motion simulation and historical prototypes.

CONTENTS

1 Physical Construction of the Current Robot 3
- 1.1 Mechanical Stack and Component Placement . . . 3
- 1.2 CAD Sources and Limits of the Drawings . . . 3

2 Network Configuration and Access 4
- 2.1 Three SSH Paths . . . 4
- 2.2 Network Profiles and Routing . . . 4

3 Electrical Connections and Onboard I/O 5
- 3.1 Shared Battery and Two Power Rails . . . 5
- 3.2 Pi, Teensy and Motor-Driver Wiring . . . 6
- 3.3 Three Reflectance Sensors . . . 8

4 Local Line Following: One Program on Pi 9
- 4.1 Execution and Data Flow . . . 9
- 4.2 Normalization and Tape Detection . . . 9
- 4.3 Significance of a Change in One Channel . . . 9
- 4.4 Significance of the Steering Gradients . . . 10
- 4.5 Wheel Commands and Resulting Motion . . . 10
- 4.6 Representative Responses and Operating Parameters . . . 10

5 Remote Robot Control: Desktop Implementation 12
- 5.1 Programs and Responsibilities . . . 12
- 5.2 Initialization Datagram . . . 12
- 5.3 Independent Sampling and Immediate Decisions . . . 14
- 5.4 Transport and Version 2 Message Contract . . . 15
- 5.5 Freshness, Stop Behavior and Recovery . . . 15
- 5.6 Timing Measurements and Demonstration Sequence . . . 16
- 5.7 C++ Packages and Validation . . . 16
- 5.8 Why the Remote Implementation Moved from Python to C++ . . . 17

Appendix A: Clock Alignment and End-to-End Message Timing 19
- A.1 Current measurement . . . 19
- A.2 Path-B summary on October 11, 2026 . . . 19
- A.3 Stale path-B result from October 11, 2026 . . . 20
- A.4 Pi–Teensy Alignment Using a Shared GPIO Edge . . . 20
- A.5 Desktop–Teensy Alignment through Pi . . . 22
- A.6 Standalone Desktop–Pi Clock-Offset Measurement . . . 28
- A.7 Available Cycle Time and Smaller-Message Options . . . 30

Appendix B: Telemetry Video 33
B.1 Capture . . . 33
B.2 Banner . . . 33
B.3 Times on the banner . . . 33
B.4 Motors-disabled check . . . 34

Appendix C: Long-Term Goal: Remote Neural Control 35
C.1 Future Functional Communication Schematic . . . 35
C.2 Future Workload and Connectivity Requirements . . . 35
C.3 System Architecture & Workload Segregation . . . 36
C.4 Desktop Container and Shared-Network Operation . . . 37
C.5 Neural Model: Scope and Baseline . . . 37
C.6 Delay-Margin Experiment and Comparisons . . . 41
C.7 Real-Time Control Protocol & Closed-Loop Data Pipeline . . . 42

Appendix D: Robot-Motion Simulation and Delay Experiments 44
D.1 Geometry baseline from the September 15 CAD assembly . . . 44
D.2 Differential-drive kinematics and turning authority . . . 45
D.3 Line-relative state and sensor geometry . . . 45
D.4 Actuator dynamics, traction, and chassis effects . . . 46
D.5 Mechanical interpretation of feedback delay . . . 47

Appendix E: Three-Sensor Reflectance Calibration 48
E.1 Motion and Sampling . . . 48
E.2 Separating White and Black Plateaus . . . 48
E.3 Reported Statistics and Acceptance . . . 49
E.4 Parameters . . . 49

Appendix F: Bench-Test History 50
F.1 Motor-B Bench Prototype (September 13) . . . 50
F.2 Three-Sensor Bench Readout and Wiring History . . . 52

Appendix G: Historical CAD Baseline and Mass Estimates 54
G.1 Chassis 3D Printing & PETG . . . 54
G.2 Physical Stack & Component Placements . . . 54
G.3 Mass Budgeting and Gravity Metrics . . . 54
G.4 September 15 CAD Visualization . . . 55

1 Physical Construction of the Current Robot

The robot is a differential-drive platform with two independently powered wheels, a passive rear caster and three downward-facing TCRT5000 reflectance modules. The installed computer is a **Raspberry Pi Model B Plus Rev 1.2**, with a Realtek RTL8188CUS USB Wi-Fi adapter. One Teensy 4.1 reads the sensors and generates the motor PWM. Desktop and the ASUS router are off the robot.

1.1 Mechanical Stack and Component Placement

The current mechanical source package is `cad_parts/assembly_sep24/`. Its native part files, rather than an overview snapshot, define the latest geometry. Printed parts use PETG. Figure 1 summarizes placement without implying measured clearances or a completed assembly test.

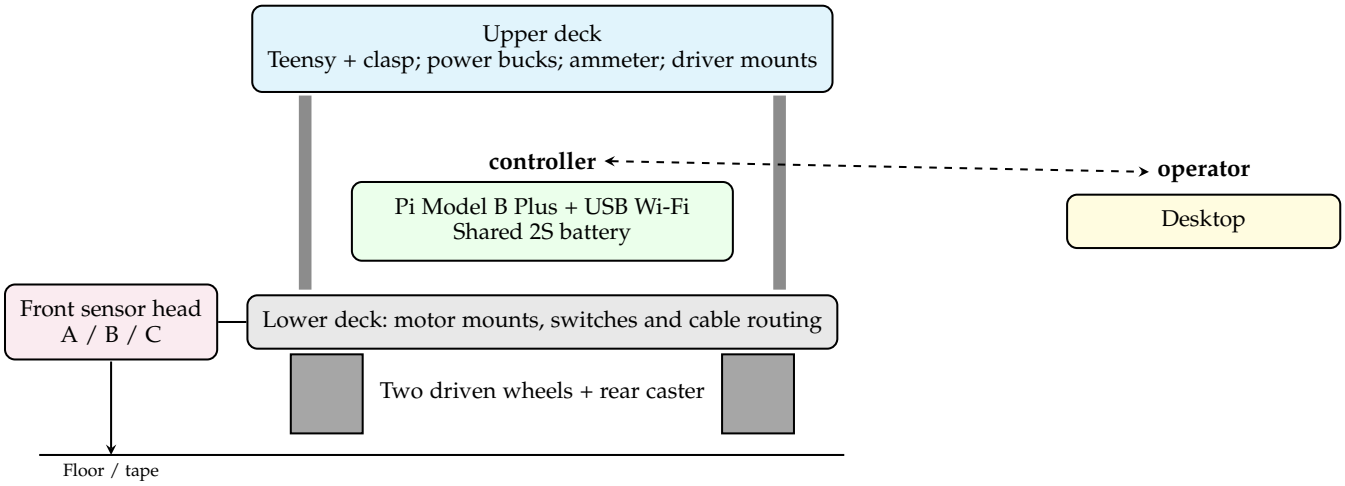


Fig. 1. Current mechanical arrangement, schematic and not to scale. The front sensor head reads the floor; the rear caster is passive. The Pi runs the controller. The Desktop is off the robot and runs the operator. Dimensions below are CAD values, not measurements of the assembled robot.

TABLE 1
Current mechanical parts and their roles.

Part	Current design description
Lower deck	Nominal $110 \times 115 \times 4$ mm, with a tapered rear. Supports the Pi, battery, motor clamps, rear caster and switches; utility openings and cable clamps provide routing access.
Pillar frames	Front and aft braced pairs support the upper electronics deck. The active design attaches directly to the lower deck; the old T-head joints are historical parts.
Upper deck	Nominal $110 \times 115 \times 3$ mm, with Teensy rails and retaining clasp, Power A/B mounting areas, ammeter opening and inclined driver-board supports. Service openings provide access beneath components.
Sensor holder	Three module positions A/B/C, interpreted as left/center/right when looking forward. Mounting arms attach below the front deck. Physical channel order and working height must agree with calibration.
Roof	A 2 mm plate with a 15 mm perimeter rim and four 50 mm columns. All four column-to-brim transitions are analytic cones. The Power A brim is 25 mm long; the other brim arms remain 15 mm. Nominal bracket clearance to the upper deck is 0.4 mm.

1.2 CAD Sources and Limits of the Drawings

The active native models are `main_lower_deck.FCStd`, `main_pillars.FCStd`, `main_upper_deck.FCStd`, `main_photo_sensor.FCStd` and `main_roof.FCStd`, all in the current package. Overview renders can lag part revisions and are not manufacturing masters. The roof's detailed dimensions are in `cad_parts/assembly_sep24/roof/README.md`.

Historical September 15 views and PETG estimates are retained in Appendix G; the motion model is in Appendix D. CAD checks do not establish physical fit, printed completion, assembled mass or loaded center of gravity.

2 Network Configuration and Access

2.1 Three SSH Paths

Figure 2 shows the three ways Desktop reaches Pi. All addresses are fixed and all prefixes are /24. Path A is the Pi with only Wi-Fi connected. Paths B and C require the Pi Ethernet cable: plugged into an ASUS LAN port for B, and into the home router for C.

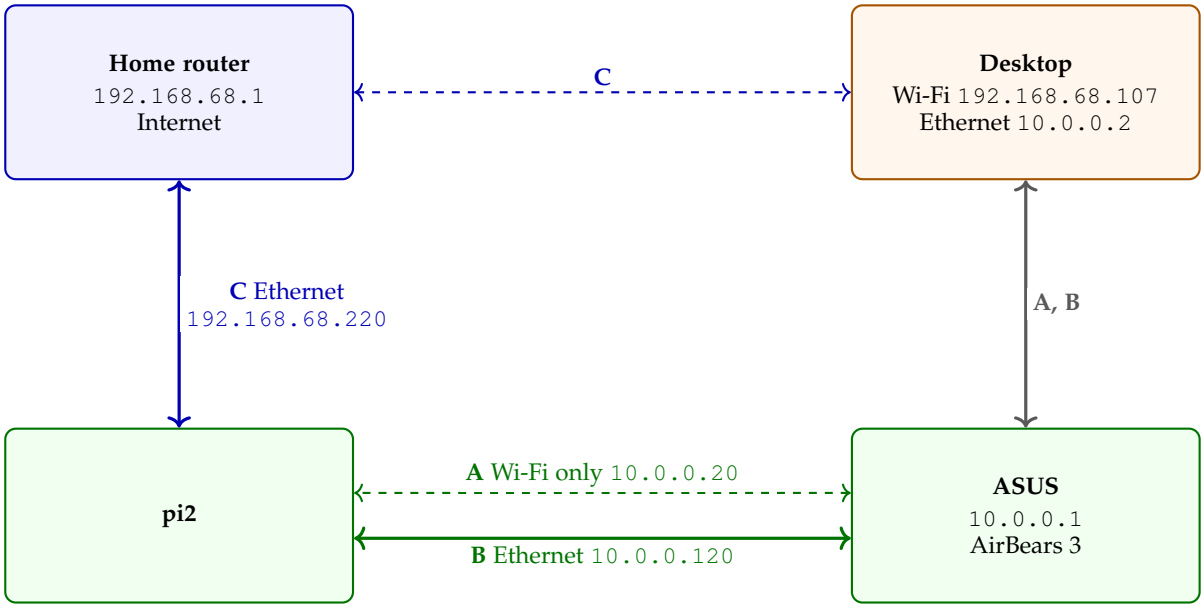


Fig. 2. SSH paths. The host name is pi2. A, Wi-Fi only: `ssh pi2` at `10.0.0.20`. B and C require the Ethernet cable. B: that cable to ASUS, `ssh pi2-asus-eth` at `10.0.0.120`. C: that cable to the home router, `ssh pi2-home-eth` at `192.168.68.220`. Desktop is `10.0.0.2` on A and B, and `192.168.68.107` on C. The login account is `pi`.

TABLE 2
Fixed SSH addresses.

Path	pi2 Address	Connection Details
A	`10.0.0.20`	Wi-Fi only (to ASUS `AirBears 3`). Desktop on ASUS Ethernet `10.0.0.2`.
B	`10.0.0.120`	Ethernet cable to ASUS LAN port. Desktop on ASUS Ethernet `10.0.0.2`.
C	`192.168.68.220`	Ethernet cable to Home router. Desktop on Home Wi-Fi `192.168.68.107`.

2.2 Network Profiles and Routing

The host name is `pi2` and the login user is `pi`. All subnet masks are `/24`.

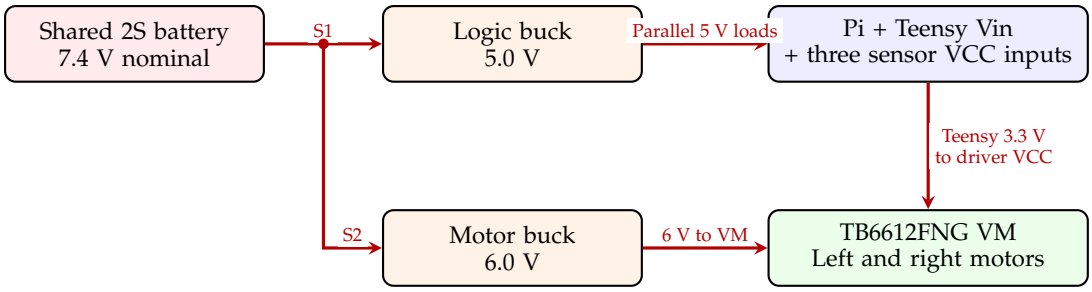
- **Path A (`asus-robot`)**: Wi-Fi connection to the ASUS router at `10.0.0.20/24` (gateway `10.0.0.1`).
- **Path B (`asus-ethernet`)**: Ethernet connection to the ASUS router at `10.0.0.120/24` (gateway `10.0.0.1`).
- **Path C (`Wired connection 1`)**: Ethernet connection to the home router at `192.168.68.220/24` (gateway and DNS `192.168.68.1`).

The script `select_pi_ethernet.sh` detects whether the Ethernet cable is connected to the ASUS router or the home router and activates `asus-ethernet` or `Wired connection 1` accordingly. The ASUS network operates isolated without requiring upstream Internet, while the home router provides direct Internet access.

3 Electrical Connections and Onboard I/O

3.1 Shared Battery and Two Power Rails

The recorded wiring uses one shared two-cell 18650 pack (2S, 7.4 V nominal), with separate 5 V logic and 6 V motor buck converters. S1 enables the logic branch and feeds the downstream S2 motor switch. Opening S2 removes motor power while leaving computing and sensing powered. Desktop and ASUS use their own external supplies.



Common return: loads / bucks / signal grounds → ammeter → battery negative

Fig. 3. Current power topology. One battery feeds two converters; this replaces the earlier independent-power-bank concept. Grounds join on the load side of the return ammeter. Pi–Teensy control data uses 3.3 V UART, not USB CDC.

The 5 V output feeds Pi, Teensy Vin and all three sensor VCC inputs in parallel. Teensy's USB power bridge is cut to isolate VUSB from Vin. Teensy 3.3 V feeds TB6612FNG VCC; the 6 V buck feeds VM. Motor leads connect across each bridge's two outputs, not to ground. Converter capacity, motor stall current and pack discharge performance remain uncharacterized in this record.

All signal grounds share a reference. Route motor current back to the motor buck rather than through Teensy or sensor ground leads. The detailed diagram omits charging/protection and bypass components. Retain the documented local bypassing and motor suppression; the earlier recommendation was a 470–1000 μF motor-rail bulk capacitor and 0.1 μF across each motor, not a measured inventory of fitted parts.

3.2 Pi, Teensy and Motor-Driver Wiring

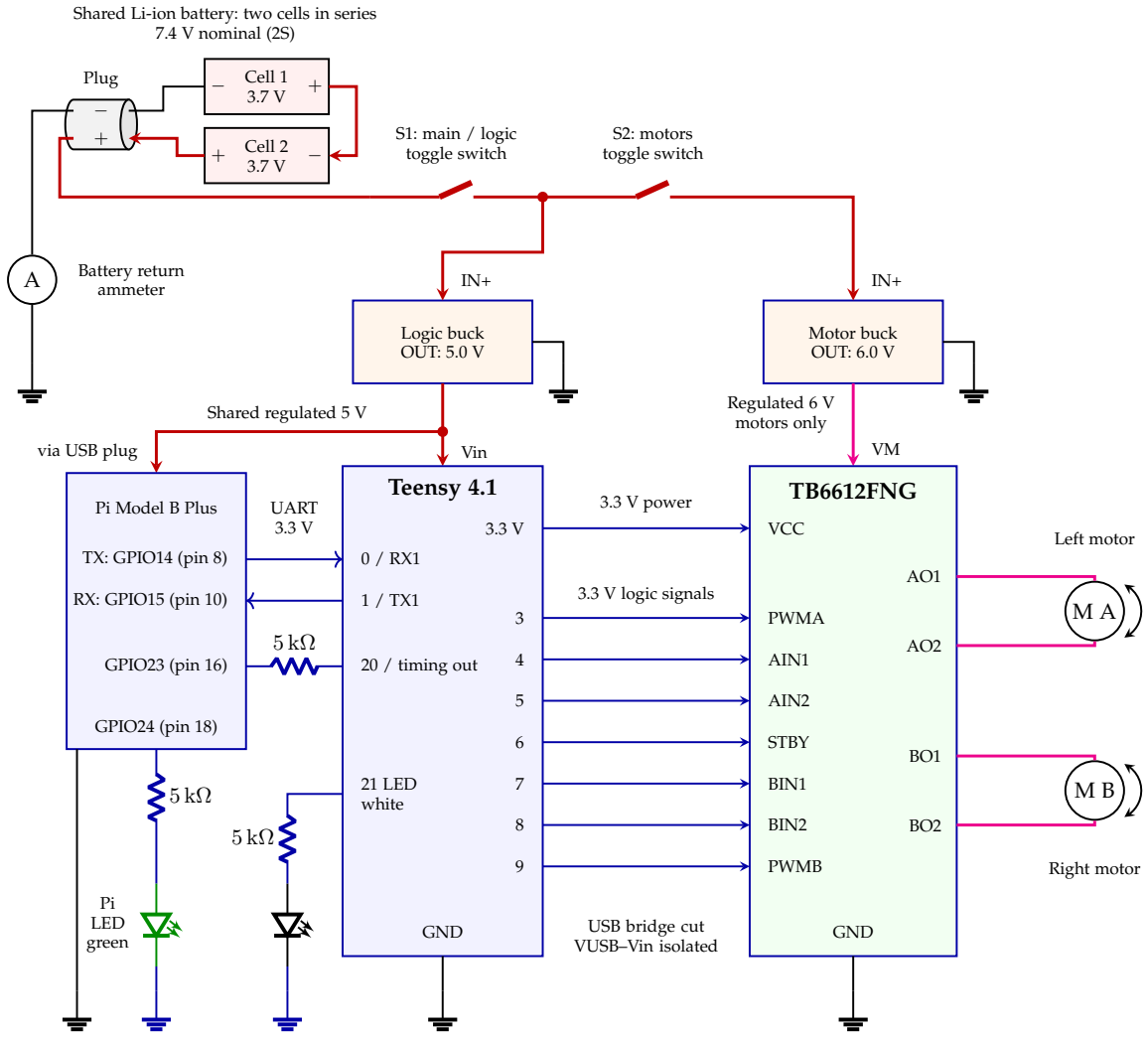


Fig. 4. Pin-level motor wiring, timing connection and indicator LEDs. Channel A drives the left wheel; channel B drives the right wheel. Teensy pin 20 connects through 5 kΩ to Pi GPIO23 (header pin 16) for shared-edge timing. Pi GPIO24 (header pin 18) and Teensy pin 21 each drive a separate grounded LED through 5 kΩ. S1 and S2 are cascaded; all grounds join the ammeter's load side.

Pi GPIO14/TX (header pin 8) connects to Teensy pin 0/RX1. Pi GPIO15/RX (header pin 10) connects to Teensy pin 1/TX1. Both are 3.3 V signals, with common ground. The current client opens `/dev/serial0` at 115200 baud, 8N1. USB is retained as a programming option, not the normal control-data path.

The timing wire is separate from UART: Teensy pin 20 is the pulse output and Pi GPIO23 is the input. Each indicator LED has its anode connected through its own 5 kΩ resistor to its GPIO output and its cathode connected to common ground. Firmware revision 16 retains the standalone `measure_pi_teensy_clock` test: both LEDs flash twice (0.5 s on, 0.5 s off), then numbered timing pulses compare Teensy timestamps with Pi kernel GPIO-event timestamps. Motors remain disabled. The reported offset applies no latency correction; the assumed 0.1 ms allowance is not an independently verified accuracy bound.

TABLE 3
Current dual-motor GPIO allocation.

Function	Teensy pins	Behavior
Left motor A	PWM 3; direction 4, 5	Signed command $-255 \ldots 255$; hardware PWM 20 kHz.
Right motor B	PWM 9; direction 7, 8	Signed command $-255 \ldots 255$; hardware PWM 20 kHz.
Common standby	6	LOW disables both bridges; `STOPALL` enters standby.
Logic / motor power	3.3 V to VCC / 6 V to VM	Separate supply pins; common ground.

Physical wheel polarity is a configuration value (currently `rr`), applied after computing signed physical-direction targets. PWM is open-loop power control; wheel speed and rotation are not measured by this program. The firmware starts stopped and has a 750 ms drive-command watchdog. ADC queries do not refresh it. A stop disables drive; mechanical coasting time is additional.

3.3 Three Reflectance Sensors

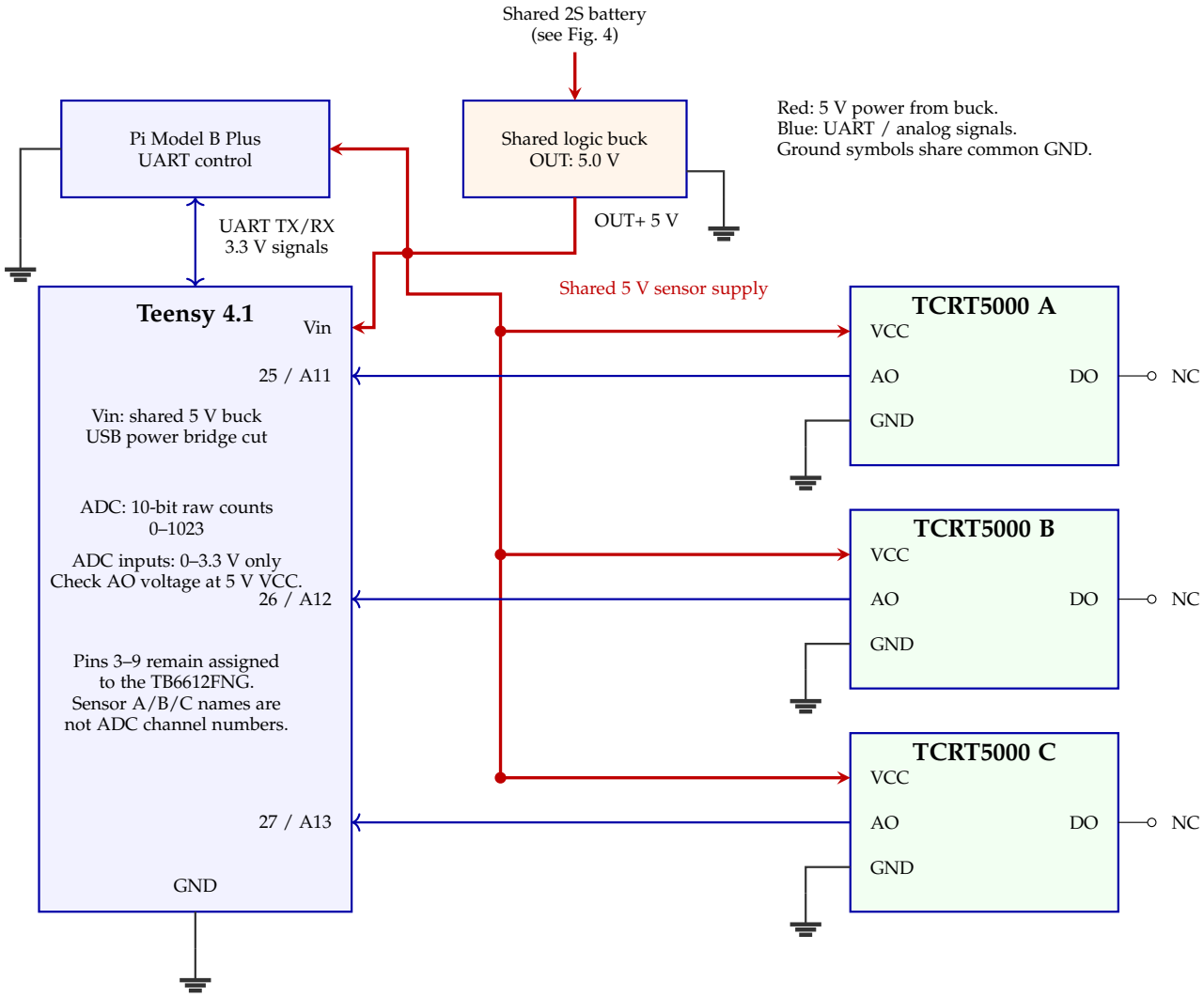


Fig. 5. Current A/B/C sensor wiring: analog outputs to Teensy GPIO 25/26/27 (A11/A12/A13); shared 5 V sensor power and common ground. DO is unused. The 5 V supply does not establish AO compatibility: Teensy ADC inputs must remain within 0–3.3 V. No attenuation circuit is shown.

`ADC3` reads A, B and C sequentially with 10-bit resolution and returns `ADC3 A B C`, with each count in 0–1023. The firmware performs neither reflectance normalization nor an acquisition timestamp. The installed calibration assumes black gives higher counts than white; validate this on the actual floor and tape. Sensor calibration is documented in Appendix E.

No independent emitter switching or emitter-off subtraction is implemented. Lighting, working height and saturation can affect calibration. The Teensy specification gives a 0–3.3 V analog-input range without 5 V tolerance; the earlier bench acquisition test did not establish the full AO voltage range of the powered modules. Historical readout and motor tests are in Appendix F.

4 LOCAL LINE FOLLOWING: ONE PROGRAM ON PI

This section describes the local controller implemented by `lineFollower_ver1.py`, with defaults updated on September 24, 2026. It uses the three-channel calibration from Appendix E and independently drives the left and right wheels. The controller permits an inside wheel to reverse on a sharp curve. Its three decisions are: whether any sensor detects tape, whether a channel has changed enough to update its accepted reading, and whether a left or right gradient is distinguishable from zero. One parameter, $k = \texttt{nSigma}$, controls all three decisions; the default is $k = 2.5$.

4.1 Execution and Data Flow

The source file is `lineFollower_ver1.py` in `onPi/localControl/`. The program imports serial helpers from `photo3_calib.py`; the existing local program is not a standalone one-file distribution.

Before opening UART, it validates the calibration and arguments. It then opens `/dev/serial0` exclusively at 115200 baud, verifies firmware features and pin mapping, requests `STOPALL`, and checks stopped status and an ADC triplet. The normal loop is sequential: request `ADC3`, evaluate the following equations, send `MOTORS left right` or `STOPALL`, require an acknowledgement, and sleep for the remainder of the target period. Desktop and ASUS are used for SSH and management; neither computes steering in this mode.

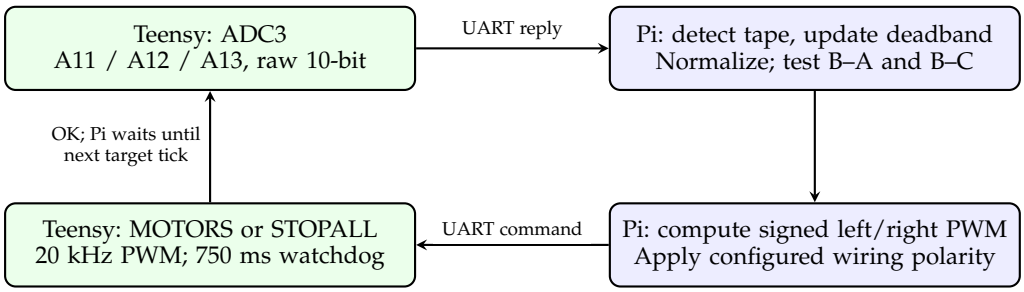


Fig. 6. Implemented local controller. All line-following decisions run on Pi; Teensy supplies ADC acquisition and hardware PWM. Three ADC channels are read sequentially.

The default period is 50 ms (20 Hz), configurable from 10 to 100 Hz. This is a target, not a hard real-time guarantee: each serial read/write has a 150 ms timeout and the loop waits for replies. Console status is limited to about 5 Hz. The current firmware source configures both wheel PWM channels at 20 kHz, independently of the Python update rate. A PWM command is not a measured wheel speed.

4.2 Normalization and Tape Detection

Let $a_i(t)$ be the current raw ADC value of sensor i, and let its white and black calibration means be $\mu_{i,W}$ and $\mu_{i,B}$. Define $\Delta_i = \mu_{i,B} - \mu_{i,W} > 0$ and

$$x_i(a) = \mathrm{clip}_{[0,1]}\left(\frac{a - \mu_{i,W}}{\Delta_i}\right). \tag{1}$$

Thus white maps to zero, black to one, and each channel's pedestal and gain are accounted for separately. Clipping limits the controller input when a reading is outside its calibrated endpoint means.

Tape is considered present if at least one *current raw* reading exceeds its white mean by more than k white standard deviations:

$$\mathrm{tape}(t) = \bigvee_{i\in\{A,B,C\}} \left[a_i(t) > \mu_{i,W} + k\sigma_{i,W}\right]. \tag{2}$$

If no channel passes this test, both motors stop and the robot waits for manual repositioning. Sampling continues, and following resumes when a channel passes again, provided the run time has not expired. Detection uses the spread of readings on white.

4.3 Significance of a Change in One Channel

Let $\widehat{a}_i$ denote the last accepted ADC reading. To estimate noise between the measured white and black endpoints, use the linear interpolation

$$s_i(a) = \big[1 - x_i(a)\big]\sigma_{i,W} + x_i(a)\sigma_{i,B}. \tag{3}$$

This interpolation provides an empirical noise model across the reflectance range. Evaluate it at both the new and accepted readings, and combine their uncertainties in quadrature:

$$s_{i,\mathrm{change}} = \sqrt{s_i\big(a_i(t)\big)^2 + s_i(\widehat{a}_i)^2}. \tag{4}$$

The accepted reading is updated according to

$$\widehat{a}_i \leftarrow \begin{cases} a_i(t), & |a_i(t) - \widehat{a}_i| > k\, s_{i,\mathrm{change}}, \\ \widehat{a}_i, & \text{otherwise.} \end{cases} \tag{5}$$

The reference stays at the last accepted value, so gradual drift can eventually exceed the band. Equal single-reading uncertainties s give a difference uncertainty $\sqrt{2}\,s$.

The first triplet is accepted immediately. The accepted values are also reset during tape loss and on resumption, so stale steering cannot prevent stopping or restarting. Equation (2) is evaluated every cycle independently of held steering readings, but still uses the same k. Setting $k = 0$ accepts every triplet and detects tape whenever a current ADC is strictly above its white mean.

4.4 Significance of the Steering Gradients

Normalize the accepted readings as $\widehat{x}_i = x_i(\widehat{a}_i)$. The two steering differences are

$$g_L = \widehat{x}_B - \widehat{x}_A, \qquad g_R = \widehat{x}_B - \widehat{x}_C. \tag{6}$$

Their noise must be expressed in the same normalized units. For each channel,

$$\nu_i = \frac{(1 - \widehat{x}_i)\sigma_{i,W} + \widehat{x}_i \sigma_{i,B}}{\Delta_i}, \qquad \nu_L = \sqrt{\nu_B^2 + \nu_A^2}, \quad \nu_R = \sqrt{\nu_B^2 + \nu_C^2}. \tag{7}$$

Applying the shared nSigma setting gives

$$\widetilde{g}_j = \begin{cases} g_j, & |g_j| > k\nu_j, \\ 0, & |g_j| \le k\nu_j, \end{cases} \qquad j \in \{L, R\}. \tag{8}$$

Both positive and negative gradients are tested. A statistically insignificant gradient contributes zero to steering, leaving the wheel at its base forward command. For example, normalized channel deviations $\nu_A = 0.03$ and $\nu_B = 0.04$ give $\nu_L = 0.05$. At $k = 2.5$, $|g_L|$ must exceed 0.125: a difference of 0.10 is suppressed, whereas 0.20 is retained.

The quadrature formulas treat the reading errors as independent and use the calibrated endpoint means as fixed references. Under this empirical noise model, k is a practical significance multiplier. The same k has three roles: raising it requires more contrast above white, larger temporal changes, and stronger spatial gradients.

4.5 Wheel Commands and Resulting Motion

When tape is present, the significant gradients determine the physical-direction PWM commands:

$$D = \mathrm{round}(255s), \tag{9}$$

$$p_L = \mathrm{round}\Big[D\,\mathrm{clip}_{[-1,1]}(0.5 + \widetilde{g}_L)\Big], \tag{10}$$

$$p_R = \mathrm{round}\Big[D\,\mathrm{clip}_{[-1,1]}(0.5 + \widetilde{g}_R)\Big], \tag{11}$$

where $s = \texttt{speed}$, default 0.5. Positive commands mean forward and negative commands mean reverse. Fixed wiring signs are applied only when transmitting these commands. With the current `rr` polarity, both transmitted signs are reversed relative to physical forward.

The offset 0.5 maintains forward drive when a gradient is zero or suppressed. For a significant gradient, the associated wheel stops at $g = -0.5$ and reverses below that value. Because $g \in [-1, 1]$, the command factor spans $[-0.5, 1]$ after clipping. At the default $s = 0.5$, the nominal command range is approximately -0.25 to $+0.5$, with 8-bit rounding.

When A rises relative to B, the *left* wheel slows and can reverse, turning toward that side. When C rises relative to B, the *right* wheel does so. If all sensors lose tape according to Eq. (2), both commands are replaced by a motor stop regardless of gradient significance. The displayed values $p_L/255$ and $p_R/255$ are commanded PWM fractions; small nonzero values may be insufficient to start a loaded motor.

4.6 Representative Responses and Operating Parameters

Table 4 illustrates the wheel law before integer rounding. Nonzero gradients in its first five rows are assumed to pass Eq. (8), and tape must pass Eq. (2).

TABLE 4
Idealized line-following responses. Wheel factors multiply base PWM D.

Accepted A, B, C	Left factor	Right factor	Robot action
0, 1, 0	1	1	Centered tape: equal forward commands.
1, 1, 0	0.5	1	Left wheel slows; curve left.
0, 1, 1	1	0.5	Right wheel slows; curve right.
1, 0, 0	-0.5	0.5	Left wheel reverses; turn left in place ideally.
0, 0, 1	0.5	-0.5	Right wheel reverses; turn right in place ideally.
Both gradients insignificant	0.5	0.5	Equal forward commands while tape is detected.
No current channel detects tape	0	0	Stop and wait for repositioning.

The controller targets 20 updates per second and prints current ADCs, accepted normalized readings and commanded wheel fractions at about 5 Hz. Commands continue to refresh while steering readings are held. A run ends after 60 s by default; stopped waiting is included in this limit, which starts after initial verification. Communication latency limits exact stopping time. Time expiry, interruption or communication failure requests a stop; the motor firmware also has a 750 ms drive-command watchdog. These stops put the driver in standby, allowing the wheels to coast to rest.

TABLE 5
Line-follower parameters. All effective values are printed at startup.

Parameter	Default	Meaning
`-t, --time`	60 s	Maximum control-loop duration, including stopped waiting; positive.
`--speed`	0.5	Base PWM scale s; $0 \leq s \leq 1$.
`--nSigma`	2.5	Nonnegative multiplier k in tape detection, reading updates and gradient tests.
`--rate`	20 Hz	Target control-loop rate; 10–100 Hz.
`--polarity`	`rr`	Left/right wiring signs; options `ff`, `fr`, `rf`, `rr`.
`--calibration`	`config/photo3\_calib.json`	Reads each sensor's white and black mean and standard deviation.
`--port`	`/dev/serial0`	Pi–Teensy UART device.
`--check`	False	Verify calibration and readout while keeping motors stopped.

5 Remote Robot Control: Desktop Implementation

Status: C++17 implementation, started by `onDesktop/runOperator`. Teensy firmware revision 16 owns the acquisition timer; no automatic startup service is installed. The operator receives three ADC readings and returns motor commands through the ASUS Wi-Fi path. It reuses the local line-following decision rules at 100 sensor triplets/s. The timing numbers later in this section were recorded with motor output disabled.

5.1 Programs and Responsibilities

The source is one tree, `lineFollowerDistributed/`, compiled on the Pi and on the Desktop. The Pi entry point is `controller`; the Desktop entry point is `run_operator`, started by `onDesktop/runOperator`. The two binaries default to motor-disabled operation. `runOperator` passes `--enable-motors` to both unless `--disable-motors` is given, in which case the Pi writes `STOPALL` and the wheels stay stopped. The same duration is given to both programs and starts after the job handshake. The default speed is 0.5.

The Pi controller owns the UART, motor polarity (default `rr`), limits, session handling and the stop policy. The Desktop operator owns calibration use and the line-following decision. It does not keep a calibration file of its own. At job start the controller reads three JSON files on the Pi and sends their bytes in one UDP datagram. A C++17 compiler and Make build the programs natively on Pi and macOS. The control programs have no Python dependency. JSON and SHA-256 headers are vendored with their licenses; Teensy requires firmware revision 16.

TABLE 6
Implemented division of the current program.

Responsibility	Pi: Controller	Desktop: Operator
Sensor acquisition	Own UART exclusively; receive timer-driven Teensy samples; validate counts and forward timestamps.	Receive readings; use the newest valid sample for the next decision.
Line following	No tape detection or steering calculation.	Calibration, `SigmaDeadband`, `tape_seen`, `normalize`, `gradient_noise`, `motor_targets`.
Motor execution	Validate targets; apply wiring polarity once; send MOTORS/STOPALL; track acknowledgement.	Return signed physical left/right PWM values together, including unchanged targets while active.
Lifetime and status	Enforce session, sequence, sample-age limit, local run limit and stop policy.	Accept the initialization datagram, then start, arm and stop the session. Identify the sensor file by the hash of the received bytes. Report transmit latency and its uncertainty.

The Pi remote process owns UART exclusively; no other serial client may run concurrently. The three JSON files stay on the Pi. The operator receives their bytes over UDP and does not read them from a shared disk. Desktop sends physical signed targets in $[-255, 255]$, not raw direction-pin settings. These represent direction and PWM magnitude, not measured wheel speed.

`runOperator` refuses any Pi address other than Wi-Fi `10.0.0.20`. It prints the effective arguments, checks the plant, and starts the controller over SSH. The controller binds the addresses of that SSH connection and waits. The operator then uses Pi port 5000 and Desktop port 5001; the Desktop address is the one reported by the controller, `10.0.0.2` when the Desktop Ethernet is on the ASUS. SSH starts and stops the controller. It does not copy `boot.json`, `clock_calib.json` or `photo3_calib.json`.

5.2 Initialization Datagram

When the controller accepts a job request it reads `/proc/uptime` and three files in `/home/pi/plant_lineFollower/config/`. `photo3_calib.json` is required. `boot.json` and `clock_calib.json` are included when present and are empty sections when absent. One UDP datagram carries all three. It is not one of the 128-byte control frames. Its 27-byte header is the six characters `LFINIT`, version 1, Pi uptime in milliseconds as a big-endian 64-bit integer, and the three file lengths as big-endian 32-bit integers. The raw file bytes follow, in the order boot, clock, then photo. Each file is at most 20 000 bytes. The controller sends the same datagram again with every job-ready retry. The operator accepts that datagram and the job-ready frame in either order, writes the three files into the run log, and only then builds the decision and the clock report.

From `photo3_calib.json` the operator reads only the white and black mean and standard deviation of sensors A, B and C. It treats those means as usable and does not consult the file's validity flag. It does not take motor polarity from the file; polarity remains the controller setting, default `rr`. The SHA-256 of the received photo bytes is the calibration identity in the following HELLO, so both sides agree on the same bytes before arming.

`boot.json` carries `hash8`, the Pi local time and `uptime_s` written at boot. `clock_calib.json` is the summary from Appendix A. The operator uses that summary only when its `hash8` equals the boot file, the Pi–Teensy half completed, and at least one Wi-Fi probe was valid. Elapsed time Δt, in seconds, is the uptime in the datagram minus `uptime_s` in the clock file. With w the median Wi-Fi half-width in milliseconds, r the detrended residual range in microseconds and b the fitted drift in ppm,

$$u = w + \frac{r}{1000} + |b|\,\Delta t \times 10^{-3} \qquad [\mathrm{ms}]. \tag{12}$$

The Ethernet half-width is not used on a robot run. The Wi-Fi midpoint and the nominal Pi−Teensy offset convert the Teensy sample-complete time into Desktop time. Transmit latency is Desktop receipt of the SENSOR datagram minus that instant. The midpoint is a clock origin, not a delay. If the hashes differ, either file is missing, or the summary is incomplete, each sample instead contributes the raw delivery upper bound already in the SENSOR frame: Pi completion time minus the conservative earliest bound. The median of those bounds is then reported as the uncertainty. A Teensy-only reset does not change `hash8`.

A 10 s motor-enabled run on October 11 used the summary in Section A.2. The initialization datagram carried `hash8 f03276bd` for both files, so the clocks matched. Pi uptime in that datagram was 2592.47 s, 390.5 s after the clock file. Equation (12) was then $1.036+0.038+1.070 = 2.143$ ms: most of the uncertainty is drift since the calibration, not the Wi-Fi half-width. The operator received 998 sensor packets with no sequence gaps and sent 989 commands. Transmit latency had median 3.947 ms, 99th percentile 12.096 ms and maximum 20.195 ms, each with that 2.143 ms uncertainty. Desktop computation had median 0.001 ms. On the Pi, 999 triplets were acquired and all 998 successive start intervals were 10 ms. It accepted 989 commands, acknowledged 974, replaced 15 that were still waiting, and made 974 nonzero motor writes. Teensy standby was confirmed on the Pi. The Desktop log is `lineFollowerDistributedBin/out/20261011_115032_cpp_operator_43586/`; the Pi log is `lineFollowerDistributedBin/out/20261011_111348_cpp_controller_1879/`. The Pi wall clock lags the Desktop, which is why those directory times differ.

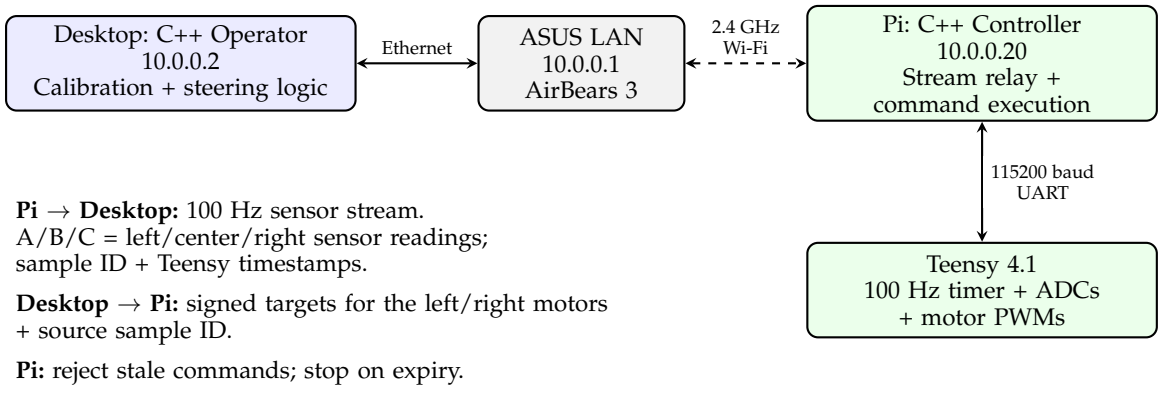


Fig. 7. Implemented remote split on the configured network. Teensy owns acquisition timing and motor PWM; Pi relays samples and validates commands; every line-following decision is made on the Desktop. ASUS only transports packets, including the initialization datagram. A run with `--disable-motors` replaces proposed targets with STOPALL at the Pi UART boundary; otherwise the Pi applies the signed targets.

5.3 Independent Sampling and Immediate Decisions

Teensy is the sampling-clock master. Its PIT hardware timer (`IntervalTimer`) invokes acquisition every 10 ms at the default 100 Hz. The interrupt handler reads A/B/C sequentially, records start and completion in Teensy microseconds, and puts the triplet in a fixed ring. Each conversion has a 100 μs timeout. There is no UART formatting or transmission inside the interrupt. A conversion fault drives STBY low to disable the motors and ends the stream. The timer uses a peripheral separate from the motor PWM timers.

The firmware main loop sends complete sample lines when UART transmit space is available, while handling motor commands and watchdogs. Its 16-slot ring holds at most 15 pending triplets; overflow drops a new sample and increments a counter. Sequence gaps expose missing data, and the original timestamps expose buffering. The three readings are sequential conversions, not simultaneous ADC captures. Hardware-timer interrupts still have finite latency; timing stability is measured, not assumed.

Pi sends `STREAM 100` once, then consumes unsolicited samples. It does not issue periodic `ADC3` requests or pace sampling with Linux sleeps. One C++ event loop handles nonblocking UART and UDP, with a parser that separates sample lines from command acknowledgements. Only one command reply is outstanding, but any number of sample lines may arrive between replies. The Pi periodically queries `CLOCK` to relate Teensy time to Pi time; this query does not trigger acquisition. Stop takes priority over ordinary commands. A missing stream, invalid timestamp/sequence, UART timeout or parser failure stops the controller. The stream timeout is the larger of 80 ms and three sample periods.

Packet buffers, a 512-entry sample history, pending targets and telemetry rings have fixed capacities. JSON parsing, calibration hashing and log-file writing remain outside the control loop. The UART writer checks target expiry again before transmitting pending motor bytes. Normal exit sends `STOPALL`, ends streaming and verifies standby. No boot service is installed.

Desktop is event driven. It receives a valid sensor packet, performs the existing calibrated calculation and sends both wheel targets immediately, without a separate 100 Hz sleep. If readings accumulated while it was busy, it drains the bounded input buffer and computes from the newest reading, recording skipped samples. The deadband then evolves on the processed samples; after drops this is not identical to processing every sample in the original loop. Replay comparisons therefore use identical processed sample sequences.

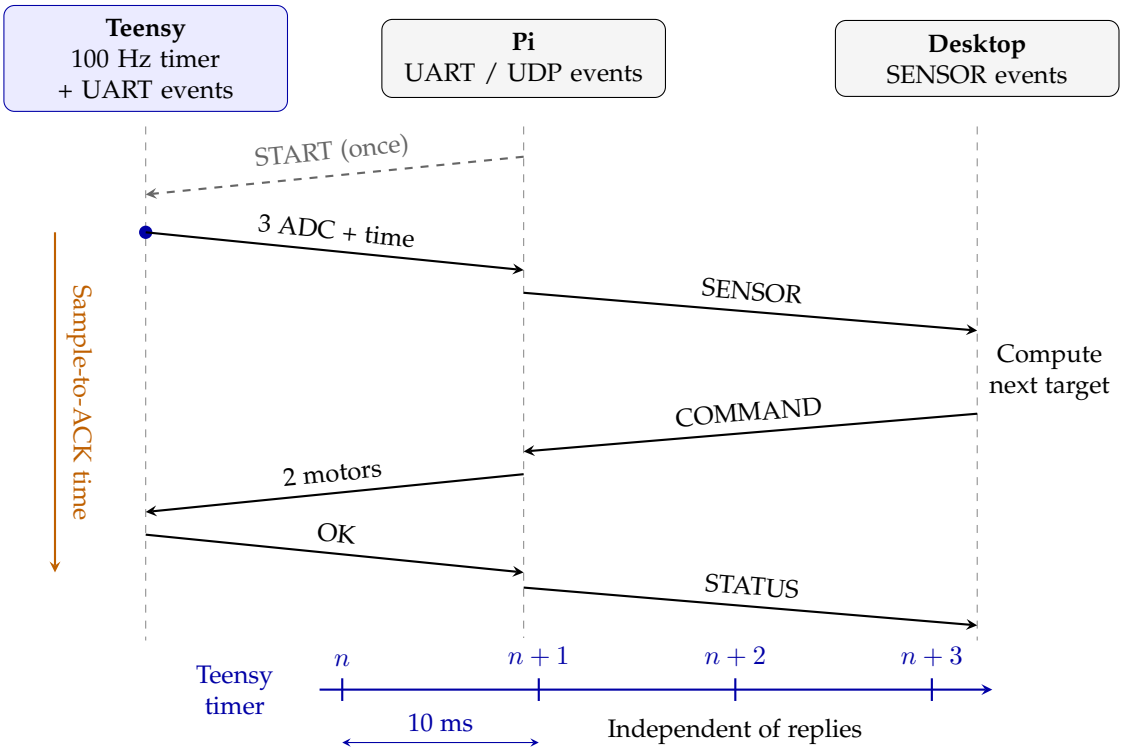


Fig. 8. Teensy samples every 10 ms from its hardware timer (blue). Pi responds to UART/UDP arrivals; Desktop computes on SENSOR arrival. START is sent once per controller launch. The bottom timer runs independently of replies; the sample-to-ack interval is reported as a clock-mapping upper bound. Message-sequence spacing is schematic, not a latency scale.

5.4 Transport and Version 2 Message Contract

Control traffic is direct binary UDP on the ASUS LAN. `runOperator` uses Pi `10.0.0.20:5000` and the Desktop address reported by the controller, port 5001. The operator runs natively on macOS, without a container or VM. Only one active operator session controls the robot. The initialization datagram in Section 5.2 uses this same socket and is not an SSH or `scp` transfer.

TABLE 7
Implemented version 2 message contents. Exact big-endian struct layouts and golden-frame tests are included in each package.

Packet	Required information
Common header	Magic, protocol version, message type, payload length, session ID and sequence number; explicit network byte order and fixed-width fields.
SENSOR: Pi to Desktop	Sample ID; Teensy start/end timestamps (microseconds); A/B/C uint16 counts; missing-sample count; conservative Pi completion bound and Pi receipt time (nanoseconds); original Teensy sequence.
COMMAND: Desktop to Pi	Command ID; source sample ID; signed left/right int16 PWM targets ($-255 \ldots 255$). Session and sample identity link the decision to its original observation.
STATUS: Pi to Desktop	Last command acknowledged by Teensy; source sample ID; Pi send/ack timestamps; armed/stopped/fault state and rejection/loss counters. Acknowledgement is not proof of motor rotation.
Initialization	One datagram, magic `LFINIT`: Pi uptime and the raw bytes of `boot.json`, `clock_calib.json` and `photo3_calib.json`. Sent with each job-ready reply, before HELLO.
Session control	HELLO/configuration agreement, ARM, STOP and acknowledgements. Agree rate, freshness limit, maximum PWM, run duration, motor enable, polarity and the SHA-256 of the received sensor file before arming.

At 100 Hz the three raw uint16 values are 600 bytes/s before headers. Version 2 uses a 24-byte application header, an 82-byte SENSOR frame and a 36-byte COMMAND frame: 11.8 kB/s combined at 100 messages/s in each direction. These frame lengths are UDP payload sizes, excluding UDP/IP headers. STATUS is 75 bytes, sent after each UART command acknowledgement and periodically. Session traffic and UDP/IP/Wi-Fi overhead are additional. SENSOR, COMMAND, STATUS and the session-control frames do not exceed 128 bytes. The initialization datagram is separate and larger: the three current summaries are a few kilobytes, and each included file is limited to 20 000 bytes. Section A.5.5 separates binary LAN payloads from ASCII UART record lengths and compares the stale 25, 50 and 100 Hz runs. Larger steady-state payloads must use an explicit size/rate budget, avoid IP fragmentation, and include pacing and overload handling; they cannot simply expand without limit.

UDP preserves message boundaries but does not guarantee delivery or order. Drop duplicate, reordered and expired control data; do not retransmit obsolete samples or drive commands. Lost data is exposed by sequence gaps. Session-control exchanges use bounded retries and idempotent acknowledgements; repeated STOP is harmless. A fresh session ID prevents packets from a previous run from taking control. Pin the active peer address/port; the session ID distinguishes runs and is not cryptographic authentication. This protocol is intended for the isolated robot LAN, not direct Internet exposure. These choices follow the UDP usage guidelines.

5.5 Freshness, Stop Behavior and Recovery

The installed default freshness limit is **100 ms**. Each run records its actual limit; acquisition remains 100 Hz. Pi retains a fixed-size ring from recent sample IDs to conservative completion-time bounds on its own clock. A `CLOCK` query establishes that the Teensy-reported instant occurred after Pi sent the query. Mapping allows 1,000 ppm relative oscillator error and 2 μs timestamp quantization; the mapping is refreshed every second and expires after two seconds. This is a documented drift assumption, not synchronized wall clocks. Buffering before Pi receives a sample consumes its lifetime instead of resetting it. At reception and immediately before serial transmission, it rejects commands referencing unknown or expired samples, invalid ranges, older command IDs or the wrong session. Thus no synchronized Desktop/Pi clocks are required for command-age checks.

A valid command expires at its source sample's conservative completion bound plus the freshness limit, not a new full lifetime after a delayed network arrival. Repeated or duplicate packets cannot extend this deadline. Both motors stop when the active command expires, on explicit STOP, local run-time expiry, sensor/serial failure, or process shutdown when communication is still available. An invalid packet never refreshes drive lifetime. Desktop loss therefore stops the robot even if Pi and Teensy remain running. Stop dispatch and UART acknowledgement add measurable delay; the freshness limit is a software policy, not a guaranteed physical stopping time.

Start stopped. Require explicit arming with a fresh session and valid recent sensor exchange. After a communication timeout/fault, remain stopped until explicitly rearmed; delayed packets must not restart motion. Normal tape loss is

different: Desktop sends fresh zero targets while the session remains active, allowing the original tape-reacquisition behavior within the run limit. Pi retains a local maximum PWM and duration limit even though Desktop chooses the steering.

The existing Teensy firmware watchdog stops drive after approximately 750 ms without a valid drive refresh. It is backup for a hung Pi process, not the configured network freshness policy. Pi must never continually refresh a command after its source sample expires. Revision 10 preserves this independent motor watchdog; sampling and clock queries do not refresh it. Tests verify serial replies and stopped status; the firmware watchdog under real motion and physical stopping distance remain untested.

5.6 Timing Measurements and Demonstration Sequence

Current telemetry video is Appendix B. The September recordings use the stale per-job GPIO-plus-network alignment and sample-to-target-receipt timing defined in Section A.5. A current `runOperator` session does not repeat that calibration; it reports the transmit latency of Section 5.2. The sample-to-ACK upper bound below is an earlier diagnostic with different endpoints; it must not be compared directly with that transmit latency.

Measure actual acquisition intervals, missed ticks, command age, UDP sequence gaps, queue drops, Pi CPU load and Desktop compute duration. Report median, 99th/99.9th percentile, maximum observed delay and the longest interruption, with run duration and sample count. Keep per-packet logging out of the critical path; use bounded buffering and summarized console output.

Acquisition duration and sample intervals use Teensy's own microsecond timestamps. Delivery age and sample-to-ack latency use a conservative mapping to Pi monotonic time:

$$T^{\text{upper}}_{\text{sample to ack}} = t_{\text{Teensy ack,Pi}} - t^{\text{lower}}_{\text{sample completion,Pi}}. \tag{13}$$

This bound includes sample forwarding, Desktop computation, return transmission and UART acknowledgement. It includes clock-mapping uncertainty and is not directly equivalent to the historical Pi-receipt-to-ack metric below. Desktop reports its local compute duration without subtracting timestamps from different hosts. The firmware sends both ADC start and end; reported sample intervals on Pi use successive starts.

1) Replay recorded ADC sequences with the same calibration through the local reference and Desktop decision logic; compare signed motor targets, tape-loss behavior and deadband resets.
2) Run live sensors at 100 Hz with motors disabled; exercise the full sensor/decision/return path, rejecting or replacing nonzero targets before UART drive. Report synthetic-response and actual UART-ack timing separately.
3) Inject loss, duplicates, reordering and delay; stop Desktop and interrupt Wi-Fi. Verify that stale data cannot extend motion or restart an expired session. Exercise bounded UART timeout paths.
4) With motor testing explicitly enabled, use raised wheels for a short low-PWM trial, verify directions and measured stop behavior, then conduct a short floor trial on tape.
5) Increase sample rate and payload size separately. Sampling already uses Teensy timestamps and a hardware timer. For higher rates, reassess ADC interrupt cost, UART bandwidth, framing and buffering; consider hardware ADC triggering/DMA.

The first acceptance goal is a demonstrated loop with measured timing and stop behavior, not a hard real-time claim. Linux/macOS scheduling and network delivery still introduce jitter; a C++ port does not establish hard real-time guarantees. Neither working SSH nor sufficient average bandwidth demonstrates a sustainable 100 Hz control loop. Appendix C holds the larger future workload and Appendix D preserves the simulation model for later comparisons.

5.7 C++ Packages and Validation

The measurements in this subsection used firmware revision 10. The firmware installed now is revision 16, and both C++ packages are built. The updated unit suite passed 3,100 checks on Pi and Desktop, including all 3,012 steering-reference decisions; virtual-UART tests also passed on both hosts (with AddressSanitizer/UndefinedBehaviorSanitizer on Mac).

A 15.01 s Desktop session on the home-network comparison path received 1,500 triplets and sent 1,484 decisions. Pi collected 2,499 triplets over its 25 s sampling run, with no missing sequence numbers. All reported Teensy start intervals were 10,000 μs; ADC duration was about 53 μs. The sample-to-ACK upper bound had median 10.37 ms, 99th percentile 22.92 ms and maximum 30.47 ms. These are clock-mapping bounds, not directly comparable to the historical round-trip numbers below. STOP was acknowledged and standby confirmed.

In a separate five-second test, deliberately suspending Pi produced a 203.40 ms loop gap. Teensy still reported 10,000 μs intervals for all 499 received triplets, with no sequence gaps; the delivery-age bound reached 198.18 ms. This demonstrates that Pi scheduling no longer sets acquisition timing. A live motor-disabled protocol probe also verified invalid/duplicate packet rejection, command-loss expiry, late-ARM rejection and confirmed STOP. No physical motor drive was enabled.

The ASUS link recovered after the adapter reset: the original script attempted activation just before NetworkManager declared the device ready, then automatic reconnection succeeded four seconds later. The recovery script now waits up to 30 s for readiness and recognizes automatic reconnection. That test used Wi-Fi `10.0.0.20` on host `pi2`. A robot run still uses that Pi address; the Desktop address is the one the controller reports for the SSH session.

The subsequent ASUS test collected 7,499 triplets over 75.059 s on Pi with zero sequence gaps and all reported start intervals at 10,000 μs. Desktop received 5,016 triplets and sent 4,957 decisions, but the 100 ms freshness policy stopped its session after 50.188 s of the intended 60 s. The longest command-arrival gap was 102.806 ms while Pi's largest loop gap was only 5.689 ms and Teensy sampling continued. This identifies a command-delivery delay; it does not distinguish Desktop scheduling from Wi-Fi delay. The acknowledged sample-to-ACK upper bound had median 9.215 ms, 99th percentile 16.563 ms and maximum 40.085 ms. Motors stayed disabled, STOP was acknowledged, and final standby was confirmed. Default freshness remains 100 ms; uninterrupted wireless operation at that limit is not yet established. Details are in `VALIDATION_TEENSY_CLOCK.md`.

5.7.1 Earlier C++ Pi-Clock Baseline

The measurements below predate Teensy-clock firmware and are preserved for comparison.

Each package includes a README, configuration, protocol documentation, dependency declaration and tests. Pi additionally includes the serial engine and independently tested command-lifetime policy. In this earlier package, Desktop carried the pure steering functions and a calibration snapshot copied from the Pi; source and calibration hashes were recorded in `provenance.json`, and updating calibration was an explicit paired copy. The current operator does not keep that snapshot. It receives the Pi files in the initialization datagram.

The native C++ tests passed on Pi and Desktop: 3,012 reference steering decisions matched, with 3,086 checks in the original unit executable (3,100 after the Teensy-clock changes). Virtual-UART integration tests verify motor-disabled output, polarity mapping with simulated motors, command-loss stopping, late ARM rejection, a missing sensor stream, partial/interleaved sample and acknowledgement lines, and expiry before UART transmission. Clock tests check stale-buffer handling, reset rejection and 64-bit timestamps across a 32-bit microsecond rollover. The same controller tests passed on macOS with AddressSanitizer and UndefinedBehaviorSanitizer. Those simulated tests never open the robot UART.

Current C++ reports use `events.csv` and `summary.json`. A 60,000-event ring and 30,000 observations per timing metric bound memory use. Reports include total counts, overwritten-event counts, retained-window percentiles and whole-run maxima. Logs are written after final stop. The two runtime entry points and all current automated tests are C++.

The first C++ ASUS protocol probe passed with proposed targets $+80/-40$ and zero applied PWM. It rejected invalid/duplicate packets, stopped on command loss, rejected late rearming and acknowledged STOP. A following strict run collected 7,499 triplets over 75.05 s with one missed acquisition tick. Desktop's 100 ms session expired after 30.74 s, when the incoming-command gap reached 128.70 ms. The Pi's largest loop gap was 19.59 ms and largest ADC interval 24.91 ms. The acknowledged round-trip median was 4.08 ms and the 99th percentile 15.42 ms. Pi continued sampling without a UART fault and confirmed standby on exit. A package-backup transfer overlapped the beginning of this run; it was not an isolated-load benchmark.

A separate C++ run with temporary 250 ms freshness and the unchanged 80 ms UART deadline completed the entire 60.007 s Desktop session. Desktop received 5,999 sensor triplets and sent 5,930 decisions. Pi acquired exactly 7,500 triplets during its 75.051 s process lifetime, with zero missed acquisition ticks. It acknowledged 5,909 commands and replaced 21 pending targets. Median acknowledged round trip was 3.88 ms, the 99th percentile 12.69 ms, and the maximum 42.81 ms. The whole-run maximum Pi loop gap was 6.46 ms; acquisition intervals had median 9.98 ms, 99th percentile 11.26 ms and maximum 14.20 ms. Desktop computation had median 0.001 ms and 99th percentile 0.002 ms. Pi consumed 15.066 CPU seconds over its 75.051 s lifetime (about 20% of one core, including 15 s of sampling after Desktop stopped).

Both programs exited normally, STOP was acknowledged, and final Teensy standby was confirmed. In all three live C++ runs, the UART audit contained only ADC3 and STOPALL, with zero nonzero motor writes and no audit overwrites. No motor motion was enabled. Defaults remain 100 ms freshness and an 80 ms serial deadline. Each package's `VALIDATION_CPP.md` contains exact report paths, reproduction commands and limitations. These results do not establish reliable long-term operation through a wireless link at the stricter freshness limit.

5.8 Why the Remote Implementation Moved from Python to C++

The split Python prototype demonstrated the communication loop, but measurements on the current Pi motivated its replacement:

- **Garbage-collection pauses.** A measured 54.7 ms collection coincided with a 57.2 ms controller gap—almost six periods of the 100 Hz sampling schedule then running on Pi.
- **CPU overhead.** The Python diagnostic used about 82% of one Pi core, compared with about 20% in the later C++ run. These separate runs indicate substantially more processing headroom, rather than a controlled language benchmark.

- **Control-loop latency.** In the diagnostic runs, median Pi-receipt-to-ack latency fell from about 15.7 ms to 3.9 ms with C++; fixed buffers and preallocated telemetry also make runtime work easier to bound.

C++ removes Python garbage collection, but it does not eliminate Linux scheduling or wireless delivery delays. Acquisition timing therefore moved to Teensy's hardware timer as a separate improvement. Desktop retains the same calibrated steering rules, verified against 3,012 reference decisions; a network freshness watchdog remains necessary.

APPENDIX A
CLOCK ALIGNMENT AND END-TO-END MESSAGE TIMING

Message latency requires timestamps at clearly defined boundaries and a mapping between the clocks that produced them. Offset is a difference between numerical clock readings, not a communication delay. `runOperator` does not collect these delays. The measurement command is `onDesktop/runClocksCalib`. It requires path B: Pi Ethernet `10.0.0.120`, Pi Wi-Fi `10.0.0.20`, and Desktop Ethernet `10.0.0.2`. A shared GPIO edge aligns Pi with Teensy, and four-timestamp UDP probes align Desktop with Pi.

A.1 Current measurement

`runClocksCalib` leaves the wheels stopped and does not change Teensy firmware. The controller must not be running, because this command owns `/dev/serial0`. The optional `--pairs` count defaults to 100, and probes on the two paths are 25 ms apart. The command refreshes `config/boot.json`, measures both network paths, then measures the timing wire. The Mac builds one summary and copies only that file to `/home/pi/plant_lineFollower/config/clock_calib.json`.

The Mac–Pi half starts a `clock_probe` server on each Pi address: Ethernet `10.0.0.120` on `eth0`, port 53120, and Wi-Fi `10.0.0.20` on `wlan0`, port 53121. Desktop binds `10.0.0.2` and alternates probes between those servers. Each probe records Desktop send D_1, Pi receive P_2, Pi reply P_3 and Desktop receive D_4. Pi turnaround is removed from the corrected round trip,

$$R_{DP} = (D_4 - D_1) - (P_3 - P_2). \tag{14}$$

The saved half-width of a probe is half of that corrected round trip. It is a single-probe bound, not an intersection of many probes. The saved midpoint is the Desktop$-$Pi clock origin, not a network delay. One-way delay is not obtained by splitting the round trip in half. For each path the summary keeps the valid and failed counts, the minimum and median corrected round trip, and the median half-width and midpoint.

The Pi–Teensy half then runs `gpio_clock` for 200 rising edges at up to 20 Hz. Teensy pin 20 drives Pi GPIO23 through 5 kΩ. Three 0.5 s flashes on both LEDs check the wire and are excluded from the fit. The offset convention is Pi time minus Teensy time. The summary keeps whether the run completed, the valid and requested edge counts, standby confirmation, the first-pulse offset, the nominal offset at the Pi reference time, the quartiles of the change from the first pulse, the fitted drift in ppm, the detrended residual range, the maximum Teensy write bracket, and the median delay from the kernel edge into the Pi process. That userspace delivery delay is not part of the offset.

Individual probes and individual edges are discarded after the summary is written. The file copies `hash8` from `config/boot.json` and adds the Pi local time and `uptime_s`. A later reboot writes a new `boot.json`. Section 5.2 sends this file to the operator in the initialization datagram. The operator uses it only when the two `hash8` values match; a robot run then takes the Wi-Fi half-width, not the Ethernet one. Equation (12) is the uncertainty reported from that summary.

A.2 Path-B summary on October 11, 2026

One default `runClocksCalib` run, 100 pairs, was taken with the robot idle and the motors stopped. The Pi boot `hash8` was `f03276bd`. The summary was written at Pi local time 2026-10-11T11:07:22−07:00, with `uptime_s` $= 2201.95$. All 200 Mac–Pi probes were valid, 100 on each path, and all 200 Pi–Teensy edges were valid. The summary does not store a round-trip maximum. The Desktop$-$Pi midpoint was 5.13328376×10^8 ms on Ethernet and 5.13328377×10^8 ms on Wi-Fi; both are clock origins, not delays.

TABLE 8
Mac–Pi corrected round trip and offset half-width, in milliseconds, from `config/clock_calib.json` on October 11, 2026. Each half-width is half of that probe's corrected round trip. These are single-probe bounds. A robot run uses only the Wi-Fi half-width.

Path	Valid	RTT min	RTT median	Half-width median
Ethernet	100	0.740	0.999	0.500
Wi-Fi	100	1.644	2.072	1.036

The Pi–Teensy half completed 200 of 200 edges and confirmed standby. The first-pulse Pi$-$Teensy offset was -5.530234 s, and the nominal offset at Pi reference time 2193.042 s was -5.530222 s. Change from the first pulse had quartiles $+5$, $+12$ and $+19$ μs. The fitted drift was 2.739 ppm and the detrended residual range was 37.7 μs. The Teensy write bracket never exceeded 1 μs. Median delivery of the kernel edge into the Pi process was 138 μs; that delivery delay is not part of the offset. The same file was sent to the operator for the run in Section 5.2.

The numerical results after this subsection are stale. They come from earlier captures, including full probe and edge archives that the current command does not keep.

A.3 Stale path-B result from October 11, 2026

Stale. This shorter run used `runClocksCalib --pairs 40` and kept the full Mac and Pi archives named below. It is not the summary now written to `config/clock_calib.json`.

TABLE 9
Stale. Mac–Pi corrected round trip and raw offset half-width, in milliseconds, October 11, 2026. Each half-width is half of that probe's corrected round trip. These are single-probe bounds, not the expanded intersection used for the September 28 session later in this appendix. The Desktop−Pi midpoint was about 4.968×10^8 ms on both paths; that value is the difference of clock origins, not a network delay.

Path	Probes	RTT min	RTT median	RTT max	Half-width median
Ethernet	40	0.706	0.970	2.799	0.485
Wi-Fi	40	1.621	2.110	9.398	1.055

The Pi–Teensy half of this stale run used 200 rising edges at up to 20 Hz after three 0.5 s LED flashes. The first-pulse Pi−Teensy offset was -5.483318 s, again a clock-origin difference. Change from that first pulse had quartiles $+10$, $+19$ and $+27$ μs. The fitted drift was 3.008 ppm and the detrended residual range was 71.5 μs. The Teensy write bracket never exceeded 1 μs. Median delivery of the kernel edge into the Pi process was 152 μs; that delivery delay is not part of the offset. The Mac records are `lineFollowerDistributedBin/out/clocks/20261011_063747/`. The Pi record is `lineFollowerDistributedBin/out/pi_teensy_clock.latest.json`, archived as `out/raspberry_pi/pi_teensy_clock_20261011_061014_1144.json`.

The subsections below keep the September 27–28 recordings and the earlier per-job procedure. Their numbers are stale as well. The shared-edge and four-timestamp definitions are the ones `runClocksCalib` still uses; the launch, the saved file and the lack of a per-job repeat are the current method in Section A.1.

A.4 Pi–Teensy Alignment Using a Shared GPIO Edge

A.4.1 Purpose and Wiring

Before subtracting a UART send timestamp on one processor from a receive timestamp on the other, their clocks must be aligned. The measurement uses a shared GPIO edge for the *Pi–Teensy clock offset*. It measures clock offset, not UART transit time. The September 27 controller runs below used this map; a robot run started by `runOperator` no longer collects it. Unlike a ping-based estimate, it does not require equal communication delays in the two directions: both processors observe the same electrical event, and UART subsequently transports the timestamp record.

Figure 4 shows the installed timing connection: **Teensy pin 20 output → 5 kΩ → Pi GPIO23 input (physical header pin 16)**, with common ground and 3.3 V signaling. The green indicator uses Pi GPIO24 (header pin 18); the white indicator uses Teensy pin 21, each through a separate 5 kΩ resistor to its grounded LED. UART remains Pi GPIO14/TX (header 8) to Teensy RX1/pin 0, and Teensy TX1/pin 1 to Pi GPIO15/RX (header 10), at 115200 baud, 8N1.

A.4.2 Measurement Procedure and Timestamp Meaning

The test stops ADC streaming and keeps both motors disabled. At startup Teensy raises its LED and the timing wire together; Pi mirrors the detected edges to its LED. Both LEDs flash twice, each time 0.5 s on and 0.5 s off. This provides a visible wiring check; the four startup edges are excluded from timing statistics. The recorded intervals were within 0.014 ms of 500 ms in the two long runs. Pi's LED write followed the kernel edge timestamp by approximately 0.18–0.23 ms, so the flashes are visually synchronized, not simultaneous electrical outputs.

Pi then requests one numbered pulse at a time, at up to 20 Hz. A random session identifier and strictly increasing sequence number associate each GPIO event with exactly one UART record. On Teensy, interrupts are briefly masked while the firmware reads its extended 64-bit microsecond clock immediately before raising pin 20 and again after a register barrier and GPIO readback. These readings, T_i^- and T_i^+, bracket the software output operation. The pin stays high for 2 ms with interrupts enabled, returns low, and Teensy sends the session, sequence and timestamps over UART. Its microsecond counter is extended across 32-bit wrap; neither processor's clock is reset.

Pi uses the Linux GPIO v2 character-device interface on `/dev/gpiochip0`. It reads the kernel-provided edge timestamp P_i, in `CLOCK_MONOTONIC` nanoseconds, rather than timestamping when its process wakes up. The default event clock and buffered event semantics are documented in the Linux GPIO line-request API and edge-event API. No debounce or hardware timestamp engine is requested. Missing or extra edges, session/sequence mismatches, duplicate replies and timeouts invalidate the attempt and stop the run instead of shifting subsequent pairings.

All reported offsets here use the current convention

$$\theta_i = \frac{P_i}{1000} - \frac{T_i^- + T_i^+}{2} \qquad [\mu\text{s}], \qquad \theta = \text{Pi time} - \text{Teensy time}. \tag{15}$$

A positive value means Pi's numerical clock reading is ahead. The approximately 2188 s offset reflects different clock origins; it is not a transmission delay. Console progress reports the smaller change $\theta_i - \theta_1$, which can have either sign.

Archived JSON schema 1 used the opposite sign; the analysis below converts it to this convention. Schema 2 and the current CSV name the convention explicitly. Raw timestamps are unchanged.

A.4.3 Software and Reproduction

The current Pi timing program is `lineFollowerDistributed/gpio_clock.cpp`, started by `runClocksCalib` for 200 edges. That command keeps only the summary fields in Section A.1. The archived captures below were taken with the earlier standalone program `measure_pi_teensy_clock` and are stale. Teensy runs Arduino C++ in `lineFollowerDistributed/firmware/halo_teensy_firmware/halo_teensy_firmware.ino`, firmware revision 16, with the `GPIO_EDGE_CLOCK` diagnostic commands. C++ gives direct access to Linux GPIO/UART calls and Teensy GPIO registers, avoids an interpreter in the acquisition path, and matches the existing controller implementation. It does not make Linux scheduling deterministic: precision here primarily comes from the kernel event timestamp and the Teensy write bracket.

The stale September captures were run on Pi, with other UART clients stopped:

```
cd ~/robot0
./measure_pi_teensy_clock
./measure_pi_teensy_clock --samples 1000 --label idle
./measure_pi_teensy_clock --samples 1000 --cpu-load --label cpu-load
```

That older default was 200 pulses, taking about 12 s including the flashes. It printed progress and summary statistics, saved `out/pi_teensy_clock.latest.json` and `out/pi_teensy_clock.latest.csv`, and retained timestamped captures under `out/raspberry_pi/`. Those JSON files include raw edges, UART records, timestamps, errors, LED transitions, fit statistics and final motor-standby confirmation. The optional load was one busy C++ worker on Pi, not a full reproduction of moving-robot or network activity. The offline report script `docs/tools/plot_gpio_clock.py` uses Python's standard library to read these saved records and bin the data; PGFPlots renders the histogram. Python is not involved in acquiring the measurements.

A.4.4 Measured Results and Distribution

Table 10 compares two completed runs collected on September 27, 2026. Both matched all 1,000 requested pulses and confirmed motor standby afterward. Each fit uses

$$\theta_i = \theta_{\mathrm{ref}} + b(t_i - t_{\mathrm{ref}}) + r_i, \qquad t_i = P_i/10^9 \quad [\mathrm{s}]. \tag{16}$$

Here b is in μs/s, numerically ppm, and r_i is the residual after removing a least-squares line. Figure 9 plots all residuals from these runs; it does not omit outliers. The raw offsets change during acquisition, so a histogram without removing drift would mix timestamp variation with relative clock-rate mismatch.

TABLE 10
Stale. Shared-edge clock measurement, September 27, 2026, normalized to Pi minus Teensy. Each reference time is Pi `CLOCK_MONOTONIC`, not wall time.

Quantity	Idle	CPU load
Matched / requested edges	1000 / 1000	1000 / 1000
Elapsed time, including startup (s)	52.98	53.47
Pi reference time (s)	2282.586009	2491.321708
Nominal Pi–Teensy offset at reference (s)	2188.464122723	2188.464834611
Fitted drift (μs/s = ppm)	+3.339	+3.186
Residual Q1 / median / Q3 (μs)	-1.9 / -1.1 / +0.1	-1.6 / -0.6 / +0.4
Residual minimum / maximum (μs)	-5.5 / +67.9	-7.4 / +34.2
Residual full range (μs)	73.4	41.6
Maximum Teensy bracket (μs)	1.0	1.0
Median kernel-event-to-userspace delay (μs)	147.0	156.0

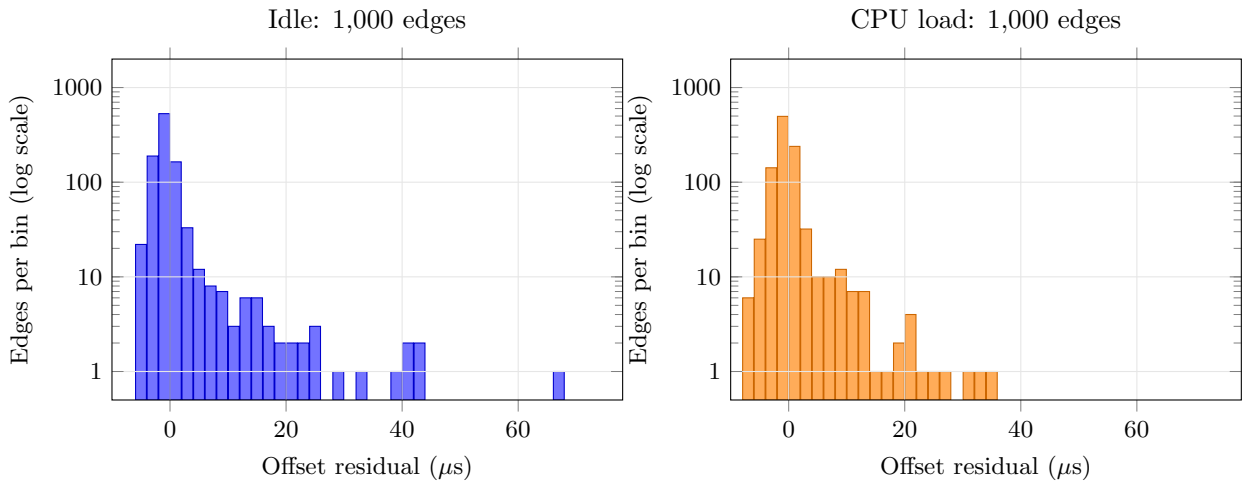


Fig. 9. Stale. Pi-minus-Teensy offset residuals after removing each run's fitted offset and drift, September 27, 2026. Common 2 μs bins and logarithmic counts expose the small positive tails. Each panel contains all 1,000 valid pulses; empty bins are omitted. These are clock-measurement residuals, not UART-delay distributions.

The full residual ranges were 73.4 μs at idle and 41.6 μs under CPU load; the Teensy bracket never exceeded 1 μs. The smaller range in the loaded run does not establish that load improves accuracy. Event delivery to Pi userspace had medians of 147 and 156 μs, and maxima of 14.659 and 5.236 ms, respectively. These delivery delays do not enter the offset calculation. The fitted relative drift, about 3.2–3.3 ppm, corresponds to roughly 0.2 ms per minute if that rate persists, so a fixed offset alone should not be extrapolated over long jobs.

The plotted captures, under `robot0/out/raspberry_pi/`, are:

- `pi_teensy_clock_20260927_164431_1292.json` (idle);
- `pi_teensy_clock_20260927_164800_1323.json` (CPU load).

The analysis stores their SHA-256 hashes, derived samples and bin counts in `docs/figures/gpio_clock/`. Short smoke/display checks are separate; one interrupted capture with 68 valid pulses is also excluded from this comparison. It recorded a 181.8 μs residual range and confirmed standby, illustrating why the completed-run ranges are not universal bounds.

A.4.5 Accuracy and UART Timestamp Boundaries

No correction is applied for unmeasured latency. A 100 μs allowance is retained as an engineering assumption, not a validated absolute error bound. Delay between the physical edge and Pi's kernel timestamp biases Pi-minus-Teensy upward; averaging cannot remove that bias. The bracket does not independently bound GPIO pad, input or interrupt latency. These histograms demonstrate repeatability, not proof of ± 0.1 ms absolute accuracy.

With the fitted conversion $M(T)$ into Pi time, UART-path timing can be computed as $P_{\text{receive}} - M(T_{\text{send}})$ or $M(T_{\text{receive}}) - P_{\text{send}}$, using consistent units. A software dispatch-to-complete-message interval includes serialization, buffering and receiver processing. The standalone GPIO reply is deliberately sent after a 2 ms pulse, so its arrival minus the edge timestamp is not a pure UART delay. The running controller instead records actual sensor and motor-message boundaries, as defined below.

A.5 Desktop–Teensy Alignment through Pi

A.5.1 Four Timestamps and the Combined Clock Map

Desktop reaches Teensy through Ethernet to ASUS, Wi-Fi to Pi, and UART to Teensy. The primary alignment therefore combines two measurements: the shared-edge Pi–Teensy map above and a Desktop–Pi UDP exchange. It does not estimate the entire clock offset by dividing a Desktop–Teensy round trip in half.

Each UDP calibration probe records Desktop send time D_1, Pi receive time P_2, Pi reply time P_3, and Desktop receive time D_4, all converted to nanoseconds. Desktop and Pi use their own `CLOCK_MONOTONIC` clocks. For an approximately constant offset $\phi = \text{Pi} - \text{Desktop}$, nonnegative delivery times imply

$$P_3 - D_4 \;\le\; \phi \;\le\; P_2 - D_1, \qquad R_{DP} = (D_4 - D_1) - (P_3 - P_2). \tag{17}$$

The interval width is the corrected network round trip R_{DP}. Pi's measured turnaround is removed; asymmetric network delay remains unknown. The implementation expands the bounds for the permitted clock-rate difference and intersects

the intervals from 100 exchanges. Their midpoint is an estimate, not an assertion of symmetric paths. Repeated probes can remove variable queueing from the bounds, but cannot identify a fixed path asymmetry.

Let $M(T)$ be the GPIO-derived conversion of Teensy time into Pi time, including fitted relative drift. The combined conversion to Desktop time is

$$\widehat{D}(T) = M(T) - \widehat{\phi}. \tag{18}$$

For a constant-offset approximation this becomes $D - T = \theta - \phi$, with $\theta = \text{Pi} - \text{Teensy}$ as defined above. The software retains uncertainty intervals for both terms. Neither processor's clock is set or restarted, and wall-clock date/time is used only for run labels. A reboot writes a new `hash8` and requires a new `runClocksCalib`. A Teensy-only reset does not change that hash.

A second set of 100 probes traverses Desktop–Pi–Teensy and back. Teensy timestamps complete request receipt and reply construction; Desktop timestamps send and receipt. Request and response on UART are each 51 bytes including LF. These exchanges supply an independent causal consistency check and a round-trip distribution. Equal byte counts balance serial wire time, but do not prove equal buffering or network delay. This end-to-end estimate is not substituted for the tighter GPIO-plus-UDP map.

A.5.2 Calibration Sequence, Drift and Saved Records

Stale procedure. The integrated controller used to give each job a new session identifier and measure the clocks again before any motor arming. That repeat is no longer part of `runOperator`. The controller receives a job, sends the existing summary files, and arms; Section A.1 is the measurement that produces `clock_calib.json`. In the old sequence, Pi ran its package-local `gpio_clock` helper with exclusive UART ownership and released that ownership before reopening UART for the controller. One controller launch handled one job. Calibration failure, inconsistent clock mappings or a detected reset prevented normal continuation. The September numbers below belong to that procedure.

TABLE 11
Stale. Previous calibration and LED sequence for an integrated remote job. Each flash is 0.5 s on and 0.5 s off. `runOperator` does not run this sequence.

Phase	Measurement or action	LED indication
Pi–Teensy startup	200 shared GPIO edges at up to 20 Hz; motors stopped	Both LEDs flash 3 times
Desktop calibration	100 Desktop–Pi UDP exchanges, then 100 Desktop–Teensy end-to-end exchanges	Teensy LED flashes 2 times; Pi LED off
Robot job	Combine GPIO and network clock maps, check end-to-end consistency, then arm	Teensy LED continuously on
Stop and drift check	Stop motors; repeat 200 GPIO edges and save the pre/post pair	Run LED goes off, then both flash 3 times for calibration; both off afterward

The run LED remains on for zero wheel targets and motor-disabled jobs too: it indicates an active control job. STOP or the configured duration turns it off. Teensy firmware also turns it off and stops motors after 750 ms without a valid motor-target heartbeat; unrelated clock queries cannot keep that indication alive. The Pi controller's existing command-freshness safeguard remains independent. Ctrl-C stops the job and skips the post-run calibration, recording an incomplete pair. The GPIO stages each add approximately 13 s; they do not consume the requested drive duration.

In that stale integrated job, the primary mapping combined Pi-minus-Teensy from GPIO with Pi-minus-Desktop from network probes. End-to-end pings provided a consistency check and round-trip measurement, without requiring UART path symmetry for the primary estimate. Its reported uncertainty included the assumed 100 μs GPIO latency allowance, an assumed 50 ppm error allowance for extrapolating the GPIO rate fit, and the network clock bounds. These remain conditional estimates, not a validated absolute accuracy guarantee. During the robot job, periodic Desktop–Pi probes and UART clock-anchor checks continued, and the post-run GPIO measurement supplied an additional drift estimate. `runOperator` does not repeat those probes. Section 5 gives the uncertainty it reports from the saved summary.

Both calibrations are written atomically to `.latest.json` files, with timestamped job archives retained:

- Pi: `pi_teensy_clock.latest.json` in the configured output directory (for these recordings, `out/raspberry_pi/integrated_clock_jobs/` relative to `/home/pi/robot0/`), also copied to `/home/pi/robot0/out/pi_teensy_clock.latest.json`. The job wrapper contains full `pre` and `post` GPIO captures, completion/error status, Pi boot identifier and drift between their reference points.
- Desktop directory: `2026_lineFollowerOperator/out/`. Its file `desktop_teensy_clock.latest.json` contains the GPIO summary, raw network and end-to-end probes, combined mapping and conditional uncertainty. A separate `desktop_pi_clock.latest.json` preserves the network calibration alone.

Match the `job_session` strings before joining Pi and Desktop records from those stale jobs. The complete post-run GPIO capture remained on Pi and was joined to Desktop data during later analysis. A failed attempt was recorded explicitly and was not silently reused. The current command is different: `runClocksCalib` writes one summary, and a later robot run reuses it until `hash8` changes. Implementation notes for the old integrated jobs are in `INTEGRATED_CLOCK.md` in both self-contained packages.

A motor-disabled integrated validation on September 27 completed the five-second control phase with 486 accepted targets. Pi replaced seven waiting targets before UART transmission; all 479 targets actually sent to Teensy were acknowledged, with no nonzero motor writes. Both GPIO stages matched all 200 pulses; both Desktop calibration stages received all 100 replies. The combined startup uncertainty was ± 1.403 ms under the stated assumptions, compared with ± 5.699 ms from end-to-end pings alone. The pre/post GPIO drift estimate was 2.865 ppm. Final standby and LED shutdown were verified; `VALIDATION_INTEGRATED_CLOCK.md` identifies the archived job records.

A.5.3 Calibration-Ping Distributions

Figure 10 uses the complete startup calibration of the unpadded 50 Hz moving run labeled `2026-09-27 20:38`. Each histogram contains all 100 valid probes. The Desktop–Pi corrected network round trip had minimum 1.788 ms and median 2.078 ms; the end-to-end calibration round trip had minimum 11.442 ms and median 12.143 ms. The latter includes the longer, equal-sized calibration messages, so it is not the control-loop round trip. Its end-to-end-only offset uncertainty was ± 5.734 ms, whereas the combined GPIO-plus-UDP startup estimate was ± 1.439 ms under the stated assumptions.

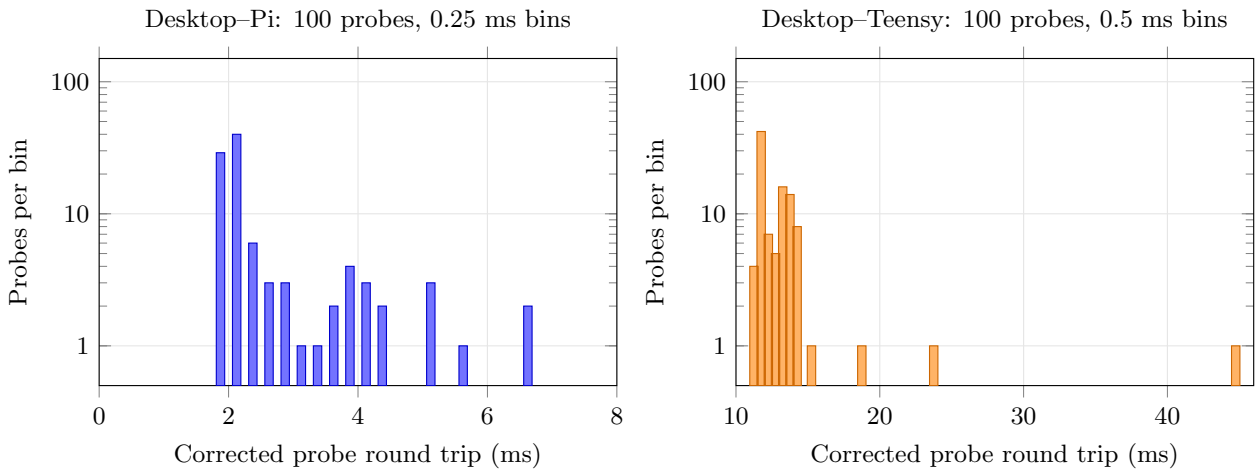


Fig. 10. Stale. Startup clock-calibration probes for the 50 Hz reference run, September 27, 2026. Turnaround between the responder's receive and reply timestamps is subtracted from Desktop's send-to-receive interval. The panels have different horizontal scales and bin widths; logarithmic counts show all tails. These are observed probe round trips, not distributions of the unknown true clock offset.

A.5.4 From Aligned Timestamps to Control-Loop Latency

For each acknowledged motor command, identify its originating sensor sample. Let T_s denote Teensy's ADC-start time, D_r Desktop's sensor-receipt time, D_c Desktop's completed-target computation, and T_r Teensy's complete motor-command receipt. Using one local clock-map estimate across this short cycle,

$$L_{T\to D} = D_r - \widehat{D}(T_s), \qquad C_D = D_c - D_r, \tag{19}$$

$$L_{D\to T} = \widehat{D}(T_r) - D_c, \qquad R = T_r - T_s, \tag{20}$$

$$S = 1000/f - R \qquad \text{(all durations in ms; } f \text{ in Hz).} \tag{21}$$

The offline code uses a common offset at the cycle midpoint, with a drift allowance across the cycle, so $L_{T\to D} + C_D + L_{D\to T} = R$ per matched row. Marginal medians or quartiles need not add. Forward latency starts at ADC acquisition, includes the three conversions (about 0.05 ms), UART and network delivery; it is not wire-only transmission. Reverse latency ends when Teensy has received the complete target, before motor-output application. ACK return delivery to Desktop is excluded from R. Spare S is the time remaining until the next sampling tick at target receipt, not CPU idle time; negative values are retained.

During offline analysis, pre/post GPIO-fit bounds are intersected at each cycle and combined with nearby Desktop–Pi network probes within a 2 s window. The implementation includes the 100 μs GPIO allowance, 50 ppm GPIO-fit extrapolation allowance, a 2000 ppm Desktop–Pi relative-rate guard, a 2 μs network timestamp guard, and a small

within-cycle drift/quantization allowance. Causality also requires both directional delays to be nonnegative. The interval midpoint gives the plotted estimate and its half-width gives conditional uncertainty. Offset errors shift the two directional estimates in opposite directions; a narrow latency histogram cannot rule out a fixed clock-offset bias. The raw Teensy round trip and spare are independent of that offset uncertainty.

A.5.5 Message Sizes and Sampling-Rate Comparison

The 25, 50 and 100 Hz recordings used the same message formats. Pi translates between ASCII records on the Teensy UART and binary packets on the Desktop LAN; a message therefore has a different size on each link. The three ADC values themselves occupy only 6 bytes as three unsigned 16-bit integers. The two signed motor targets occupy 4 bytes. Sequence identifiers, timestamps and diagnostic fields account for the rest.

TABLE 12
Binary control messages, unchanged across the three sampling rates. The application frame is the complete UDP payload: a 24-byte protocol header plus the message body. UDP/IP and Ethernet/Wi-Fi framing are excluded.

Message and direction	Body (bytes)	Header + body (bytes)	Main contents
SENSOR, Pi → Desktop	58	82	3 ADCs, IDs, timestamps, drop count
COMMAND, Desktop → Pi	12	36	Source sample ID, left/right targets
STATUS, Pi → Desktop	51	75	State, acknowledged IDs, times, targets, counters

The 24-byte header contains magic/version, message type, body length, session ID and sender sequence. At one SENSOR and one COMMAND per sample, their combined application traffic is 2.95, 5.90 and 11.80 kB/s at 25, 50 and 100 Hz, respectively (1 kB = 1000 bytes). STATUS messages, clock probes, startup traffic and network framing add further traffic. These are nominal rate budgets; coalescing and stopping can reduce the actual command count. Pi forwards each ADC triplet together, not as three separate packets.

On UART, a sensor record is

```
S sequence start_us end_us ring_drops A B C\n
```

and a motor record is `MOTORST left right\n`. Here `\n` denotes one LF byte; the field names are replaced by decimal values. Sizes include the command text, spaces, signs, all decimal digits and LF. They vary with sensor values, sequence number, timestamp magnitude and motor targets; the sampling rate does not directly set the byte count. Table 13 reconstructs their lengths from each run's complete UART audit log.

TABLE 13
Unpadded UART records at 115200 baud, 8N1. Rate is in Hz; byte counts are median [minimum–maximum], including LF. Wire time is calculated for the median length, not measured end-to-end latency. Counts include all logged sensor records and dispatched motor commands, so they differ from matched timing-cycle counts.

Rate	Samples	Sensor bytes	Sensor wire (ms)	Commands	Motor bytes	Motor wire (ms)
25	199	41 [39–43]	3.56	198	17 [15–18]	1.48
50	399	40 [36–41]	3.47	393	17 [15–18]	1.48
100	799	42 [40–43]	3.65	770	17 [15–18]	1.48

Each UART byte needs 10 serial bits (start, eight data, stop), giving 11,520 bytes/s per direction on this full-duplex link. The median sensor record therefore occupies about 3.5–3.6 ms on the wire, compared with 1.48 ms for the median 17-byte motor command. UART acknowledgements and clock traffic are additional. Serialization alone already contributes about 2 ms more in the sensor direction. These wire times exclude ADC acquisition, software scheduling, buffering, parsing and LAN delivery, which are included in the observed control-loop intervals below.

TABLE 14
Stale. Moving-run timing results from September 27 with the payloads above; the final row is the separate padded-command experiment. Directional columns and round trip are medians in ms; u is the maximum per-cycle conditional one-way uncertainty in ms, rounded upward to 0.1 ms. Cycles are complete matched observations, not all issued commands.

Run	Active (s)	Cycles	$T \to D$	$D \to T$	Round trip	Max. $\pm u$
25 Hz	7.9	198	5.4	2.7	8.2	2.4
50 Hz	8.0	392	5.2	2.7	8.1	2.6
100 Hz	8.0	770	5.6	2.6	8.3	2.4
50 Hz, padded	4.0	186	5.4	4.8	10.3	2.4

The original 25, 50 and 100 Hz jobs all completed their requested 8 s duration normally (the measured active interval excludes arming overhead). Sampling periods were exactly 40, 20 and 10 ms in the recorded Teensy timestamps, with no missing sample slots. Median transport intervals remained similar across rates: 5.2–5.6 ms from Teensy sampling to Desktop receipt, and 2.6–2.7 ms from Desktop computation completion to Teensy target receipt. Desktop computation itself had medians 0.010, 0.013 and 0.013 ms at 25, 50 and 100 Hz. The median sample-to-target round trip was 8.1–8.3 ms; changing the sample period did not scale this latency. Median spare time at target receipt was 31.8, 11.9 and 1.7 ms, respectively. The robot started at different track positions, so these runs do not isolate sampling rate as the cause of small timing differences.

The GPIO-plus-network startup uncertainty grows to approximately ± 2.3–± 2.5 ms at the least-constrained cycles in these recordings because the probe distances and drift allowances differ. The startup ± 1.4 ms figure must not be reused as a fixed uncertainty for every video frame. The later standalone wired Desktop–Pi calibration belongs to a different session and does not retroactively reduce these recordings' uncertainty.

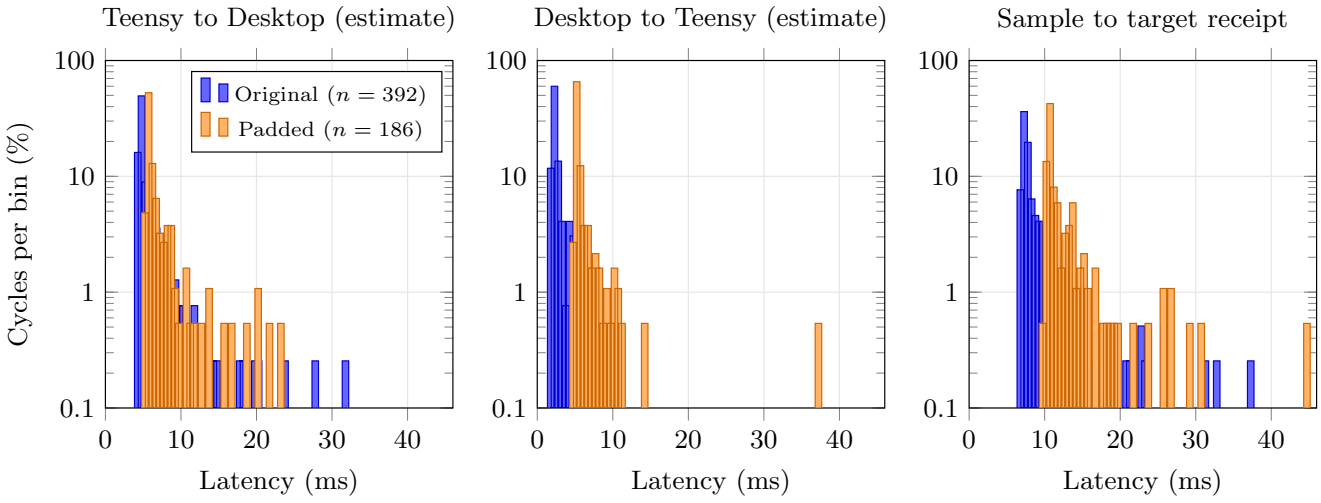


Fig. 11. Stale. Control-loop latency at 50 Hz, September 27, 2026, before and after padding the motor UART message. Histograms use common 0.5 ms bins, normalized by each run's matched-cycle count, with logarithmic vertical scales and all outliers included. Adjacent colored bars represent the same bin. The first two panels depend on clock alignment; the third uses only Teensy's clock. The padded run stopped early and used a different track segment, so this is not a controlled steering-performance comparison.

A.5.6 Testing the Directional Asymmetry by Padding UART Commands

At 115200 baud, 8N1, every byte takes $10/115200$ s on the UART wire. In the original recordings, sensor records typically used 40–42 bytes, versus about 17 bytes for a motor command. The optional Pi switch `--match-motor-wire-size` prefixes a signed motor magnitude with zeros, preserving its value while matching the complete command's length, including LF, to its originating sensor record. Desktop UDP payloads, motor policy and firmware are unchanged; the switch defaults off.

The padded 50 Hz run verified exact corresponding lengths and unchanged targets for all 186 dispatched commands; all 186 were acknowledged. The median predicted added serial time was 2.170 ms. Relative to the earlier 50 Hz reference, median round trip increased from 8.098 to 10.313 ms, a 2.215 ms increase. Pi-dispatch-to-Teensy-receipt timing, computed with the GPIO map and independent of Desktop offset, increased from 1.568 to 3.716 ms. These changes closely support the UART serialization explanation. The remaining difference between directional estimates does not establish a network asymmetry within their uncertainty.

The padded job ended after 4.045 s when the 100 ms freshness safeguard expired. Pi acknowledged STOPALL and confirmed motor standby; Desktop's separate cleanup STOP was not acknowledged. Two pending targets were replaced at Pi and three issued commands were not accepted there. The exact network/host cause of the freshness gap was not identified, and no UART command or ACK loss was observed. The histogram includes the full captured active interval rather than presenting this as a completed 8 s run.

A.5.7 Software and Reproducing These Figures

The current measurement programs are C++17: `lineFollowerDistributed/clock_probe.cpp` on both machines and `gpio_clock.cpp` on the Pi, launched by `onDesktop/runClocksCalib`, which saves only the summary in Section A.1. `controller.cpp` and `operator.cpp` run the robot and do not collect this clock delay. Teensy uses the shared Arduino C++ firmware described above. These implementations use direct GPIO/UART/socket interfaces and

avoid Python interpreter and garbage-collection work in the timing path. C++ does not eliminate OS scheduling or Wi-Fi delays. The four recordings and figures below are stale September 27 results; the current command does not write those archives.

Offline Python scripts from the earlier Desktop package join sample/command identities and timestamps: `prepare_stats_video.py`, `clock_alignment.py` and `integrated_clock_analysis.py`. The resulting per-cycle `timing_cycles.csv` contains the directional estimates, their bounds, compute time, round trip and spare. Those scripts render the September telemetry movies. The current recording is Appendix B; its Swift renderer draws the banner and does not acquire control-loop timestamps.

The plotting script `docs/tools/plot_desktop_clock.py` uses Python's standard library to read the archived JSON, writes raw derived samples and histogram bins, and generates PGFPlots sources. Its default input is the sibling `2026_lineFollowerOperator/` directory; an alternate package root can be passed as an argument. Source paths and SHA-256 hashes are saved in `docs/figures/desktop_clock/sources.json`. The four input recordings under the operator's `out/` are:

- `integrated_25hz_20260927_205834`;
- `integrated_50hz_20260927_203812`;
- `integrated_100hz_20260927_205707`;
- `integrated_padded_50hz_20260927_212141`.

Reproduce the figures from the robot repository root:

```
python3 docs/tools/plot_desktop_clock.py
cd docs/figures/desktop_clock
pdflatex -halt-on-error calibration_histograms.tex
pdflatex -halt-on-error latency_histograms.tex
```

A.6 Standalone Desktop–Pi Clock-Offset Measurement

Stale results. The September 28 experiment isolates the Desktop–Pi part of the clock map. It compares a temporary wired Pi connection with the normal Pi Wi-Fi connection, without operating the motors or accessing Teensy. It measures clock offset and network round trip; it does not directly measure either one-way delay. All results in this subsection belong to this standalone session, not to the September 27 moving runs, and not to the summary `runClocksCalib` now writes.

A.6.1 Connections and Acquisition

Figure 12 shows the actual arrangement. Desktop remains wired to an ASUS LAN port, and an additional cable connects Pi to another ASUS LAN port. Pi Wi-Fi remains on AirBears 3. Desktop keeps its home-router Wi-Fi connection for Internet access. The Pi Ethernet address is `10.0.0.120`. Desktop `10.0.0.2` and Pi Wi-Fi `10.0.0.20` remain in use. This is Ethernet through ASUS, not a direct Desktop–Pi cable.

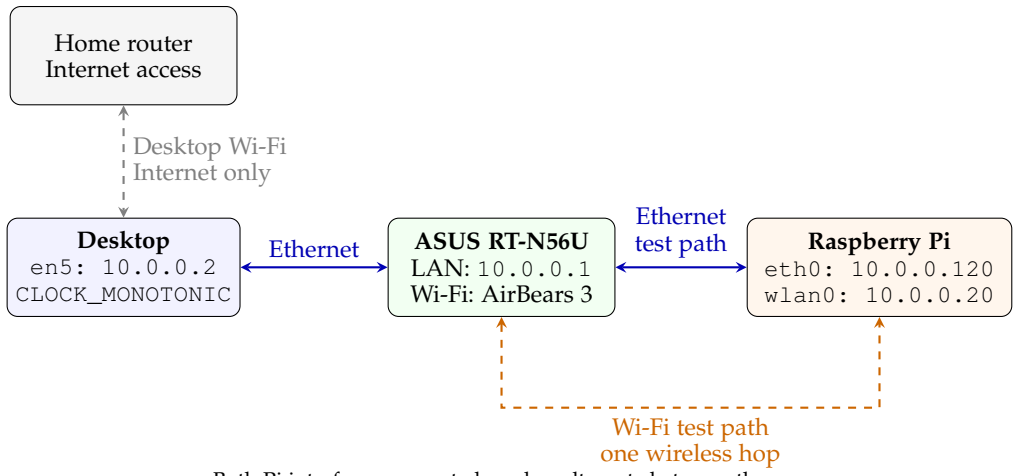


Fig. 12. Standalone clock-offset experiment. The wired and wireless Pi paths are measured separately while both remain connected. The Pi Ethernet link negotiated 100 Mb/s full duplex. Desktop's home Wi-Fi carries Internet traffic, not the measurement probes.

The September 28 acquisition used the standalone package `2026_measure_pi_desktop_clock/` on Desktop. Its C++17 `clock_probe` executable ran on both processors; Pi's copy was under `/home/pi/robot0/measure_pi_desktop_clock/`. The same probe now lives in `lineFollowerDistributed/clock_probe.cpp` and is started by `runClocksCalib`. Python launched the September processes and analyzed saved data, outside the timestamp-critical path. C++ provides direct socket and timestamp-control access without an interpreter in the acquisition loop; it does not make either OS deterministic.

Two Pi UDP servers bind explicitly to `eth0` and `wlan0`, using ports 53120 and 53121. Desktop binds to `en5`. Incoming interface indices are checked, preventing a nominal Wi-Fi reply from silently taking Ethernet. Probes alternate between paths at approximately 20 probes/s/path, reversing pair order. Each request/reply application frame is 192 bytes, with session, probe and server-instance identifiers plus timestamps and diagnostics. This size differs from the robot's 82/36-byte SENSOR/COMMAND pair.

A.6.2 Timestamp Method and Uncertainty

For each exchange, D_1 is Desktop send time, P_2 Pi receive time, P_3 Pi reply time and D_4 Desktop receive time. Both receivers use kernel software receive timestamps. Desktop transmission is timestamped immediately before the application sends. Pi Ethernet uses a kernel software transmit timestamp matched by packet ID and returned in a follow-up datagram; D_4 still refers to the original reply. Pi's Wi-Fi driver does not advertise software transmit timestamping, so that path uses an application send timestamp. Thus the link comparison also includes different Pi transmit boundaries.

The Linux socket timestamping interface supplies receive/transmit metadata and packet IDs for matching transmit timestamps. Neither tested Pi interface exposes a hardware PTP clock; this experiment uses no NIC hardware timestamps. Kernel realtime timestamps are explicitly converted to each host's `CLOCK_MONOTONIC` through nearby bracketed clock reads. Conversion and quantization allowances are retained in the uncertainty budget; clocks are never set or reset.

Here the displayed sign is $\psi = \text{Desktop} - \text{Pi} = -\phi$, giving positive values for this boot pair. Causality gives

$$D_1 - P_2 \le \psi \le D_4 - P_3, \qquad R_{DP} = (D_4 - D_1) - (P_3 - P_2). \tag{22}$$

Pi turnaround is removed from the corrected round trip. The midpoint estimates offset; the half-width bounds the unknown directional asymmetry under the clock assumptions. The analysis expands each interval for timestamp mapping, quantization and an assumed 2000 ppm rate guard across the probe. It does not divide latency equally between the two directions and call the result exact.

One-second window intersections additionally use a stated 50 ppm rate allowance, with 2000 ppm sensitivity results retained. A separate affine fit permits constant relative drift with at most 10 μs model deviation over the run. These are conditional models, not verified oscillator specifications. A narrow histogram or a large sample count cannot remove an unknown fixed directional bias.

A.6.3 Results and Interpretation

The idle capture lasted 120 s with 2400 probes per path; a second 60 s capture used one busy C++ thread on Pi's single core and 1200 probes per path. All 7200 probes were valid, with no missing/rejected replies or Ethernet transmit-timestamp matching failures. Both runs used the same Pi boot ID. The load test represents CPU contention, not a full moving-robot workload.

TABLE 15
Stale. Standalone Desktop–Pi results, September 28. All times are ms. The final column is the narrowest expanded individual-probe offset interval half-width, at that probe's instant; it is not a guaranteed uncertainty throughout a later job.

Condition	Pi link	Probes	RTT min.	RTT median	Best $\pm u$
Idle, 120 s	Ethernet	2400	0.521	0.994	0.266
Idle, 120 s	Wi-Fi	2400	1.403	1.996	0.706
CPU load, 60 s	Ethernet	1200	0.504	0.971	0.257
CPU load, 60 s	Wi-Fi	1200	1.365	2.003	0.687

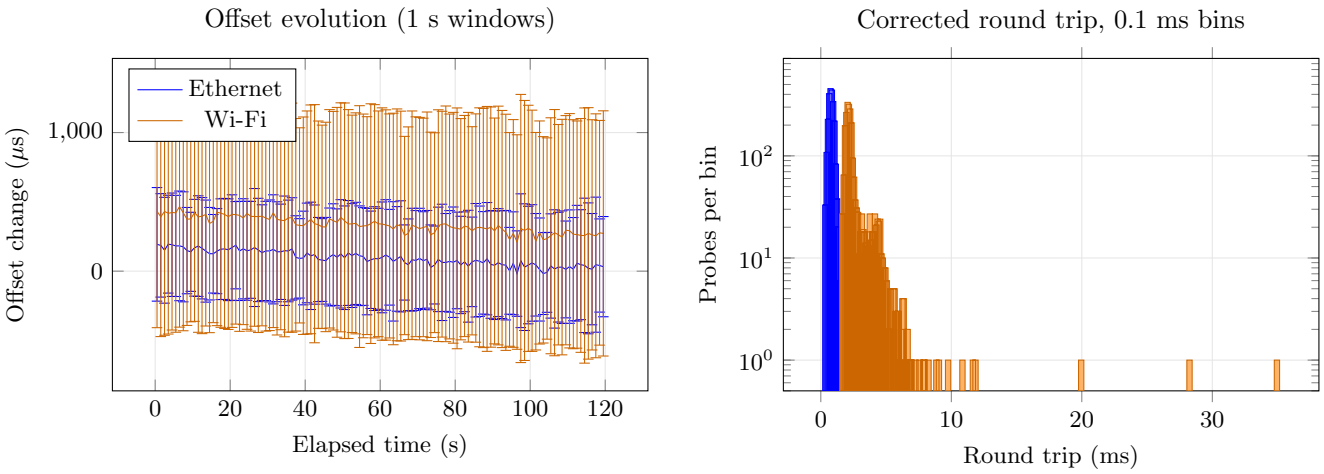


Fig. 13. Stale. Idle experiment, September 28, 2026: offset evolution and corrected round-trip distribution. The left panel subtracts one constant offset and plots one-second interval midpoints with conditional error bars using the 50 ppm window assumption. The right panel includes all observed round trips in 0.1 ms bins with logarithmic counts. Offset spread is not absolute accuracy.

Ethernet gave about ± 0.26–± 0.27 ms offset bounds, versus ± 0.69–± 0.71 ms over the Pi Wi-Fi path. The idle Ethernet affine estimate at elapsed 59.990 s was Desktop minus Pi $= 1118378.889885639$ s ± 0.270 ms under its model. The large positive value is the difference between clock origins, not a network delay. Sub-0.1 ms absolute accuracy has not been demonstrated. The observed Ethernet median RTT was about 1 ms, versus about 2 ms on Wi-Fi; these are round trips of the standalone probes, not motor-control cycles.

Combining a fresh wired bound with the assumed 0.1 ms GPIO allowance suggests a Desktop–Teensy uncertainty floor of roughly ± 0.36–± 0.37 ms before extrapolation, fitting and other mapping allowances. This is a prospective budget, not a measured integrated result. A cable removal does not itself reset the monotonic clocks, but subsequent drift needs a validated holdover model; 1 ppm rate error alone accumulates 0.06 ms per minute. No untethered holdover test was performed. A reboot invalidates the old calibration. These measurements cannot be retroactively applied to the earlier robot runs or interpreted as a reduction in their physical communication delay.

Raw integer-nanosecond timestamps, reports and plots are in the standalone package's `out/20260928_073819_idle/` and `out/20260928_074049_cpu_load/`. The latest analyzed summary is `out/desktop_pi_clock`.

`latest.json`. `run_measurement.py` launches acquisition; `analyze.py` and `affine_bounds.py` produce the intervals and plots. This report's copied figures, source hashes and derived tables are under `docs/figures/pi_desktop_clock/`; reproduce the tables with `docs/tools/report_pi_desktop_clock.py`.

A.7 Available Cycle Time and Smaller-Message Options

A.7.1 What "Free" Time Is Supported by the Measurements?

For the existing robot, the defensible quantity is *spare time at target receipt*: $S = 1000/f - (T_r - T_s)$ in ms. Both timestamps are on Teensy, so Desktop clock-offset uncertainty does not enter this subtraction. Better alignment improves the separation into forward and reverse delay; it neither creates more spare time nor changes the already measured round trip. The measurements stop at complete target receipt, before motor-output application. They therefore do not establish CPU idle time or an exact interval from motor application to the next ADC tick. The spare-time numbers in this subsection are the stale September moving runs.

TABLE 16
Stale. Observed spare time in the original unpadded moving runs. Quartiles are in ms. Late means the target arrived after the next nominal sample deadline; the denominator includes only complete matched cycles.

Rate (Hz)	Period (ms)	Spare Q1	Spare median	Spare Q3	Late / matched cycles
25	40	30.9	31.8	32.0	0/198 (0.0%)
50	20	10.4	11.9	12.3	10/392 (2.6%)
100	10	0.5	1.7	2.1	156/770 (20.3%)

The most representative measured margins are therefore approximately **32 ms at 25 Hz, 12 ms at 50 Hz and 1.7 ms at 100 Hz**. The 100 Hz run had 156/770 matched targets arrive late, about 20%; a positive median is not a guaranteed computational budget. At 50 Hz, 10/392 were late (2.6%); none of the 198 matched 25 Hz cycles were late. These short runs at different track positions do not prove long-term deadline reliability. Extra Desktop computation consumes approximately the same amount of spare time per cycle if other delays stay unchanged; it can also change queuing. New compute workloads must be checked against tails, not just these medians.

A.7.2 ADC and Motor Resolution

The current ADC already produces values in 0–1023, i.e. 10 useful bits per channel. Three losslessly packed channels need 30 bits, fitting in 4 bytes instead of the current six binary UDP bytes. Dropping three LSBs gives $q = a \gg 3$ in 0–127: seven bits/channel, or 21 bits in 3 bytes. Relative to lossless packing, this sacrifices precision to save only *one additional byte per triplet*. Truncation reconstructed as $8q$ underestimates by 0–7 ADC counts; using the bin midpoint removes that systematic downward reconstruction choice but still leaves quantization error. The light thresholds and noise calibration must use consistent units. Recorded steering has not yet been replayed or driven with this quantization, so its effect near tape-detection thresholds is unverified.

Motor values are PWM targets, not measured speeds. The existing full range -255 to $+255$ needs nine bits per wheel, or 18 bits for both, packable into 3 bytes without loss. Six bits of *magnitude* plus a direction bit gives 0–63 magnitudes in each direction, fitting each wheel in one byte. Mapping magnitude to the configured PWM maximum M gives a step of about $M/63$: about 4 PWM counts at $M = 255$, or 2 at $M = 128$. Six bits total, including sign, would be coarser. It is plausible that six magnitude bits suffice for this toy controller, but these data do not establish the minimum usable resolution; low-speed starting and tight-turn behavior need checking. Again, reduced precision saves only one byte beyond losslessly packing both targets. Exact zero and the configured PWM limit must be preserved.

A.7.3 Headers and Status

The present LAN application header is **24 bytes (192 bits)**, not 24 or 26 bits. It contains 4-byte magic, 1-byte version, 1-byte type, 2-byte body length, 8-byte session and 8-byte sender sequence. There is no requirement for a 26-bit sensor header. A 24–26-bit total header cannot retain the existing independent session and sequence fields; reducing it that far changes the stale-command and reset-detection design. The serial ASCII records do not use this LAN header at all.

A possible LAN revision could use a 12-byte application header: version/type 1 byte, flags 1 byte, length 2 bytes, job token 4 bytes and sequence 4 bytes. The job token must be newly negotiated and bound to the active peer, with explicit rollover and old-session rejection rules; it is not authentication. A shorter session field has a higher collision risk than the current 64-bit field. UDP already provides datagram boundaries, but its own transport header and the IP/link headers remain. Application compression cannot remove those headers, Wi-Fi contention or OS scheduling latency.

With that header and job-relative timestamps, a compact lossless SENSOR could occupy 23 bytes: header 12, sample-start time 4, ADC duration 2, packed ADCs 4 and diagnostic flags 1. Its header sequence identifies the sample. A COMMAND could occupy 19 bytes: header 12, source sample reference 4 and packed targets 3. Pi-side receipt

timestamps and full counters would be kept in local logs for later joining. This illustrative 42-byte sensor/command pair replaces 118 bytes, saving 76 application bytes per cycle, or 1.90, 3.80 and 7.60 kB/s at 25, 50 and 100 Hz. These are design budgets, not implemented packets; clock-epoch, sequence and log-joining rules must change together.

STATUS can separate essential feedback from verbose diagnostics. Keep immediate stop/fault reporting and command receipt confirmation; send cumulative counters and repeated configuration at a lower rate, for example 5 Hz. An illustrative 25-byte compact UDP status is a 12-byte header plus acknowledged sample/command reference (4), Teensy receipt time relative to the job (4), applied target pair (3), and state/flags (2). It retains the receipt timestamp needed for timing. Pi-local dispatch times and other diagnostics can remain in the Pi log and be joined offline by ID. A normal receipt report may be piggybacked on subsequent sensor traffic only if its added delay is acceptable; waiting for the next sample can delay that report by up to 10, 20 or 40 ms. Critical stop/fault feedback should not wait. These changes reduce telemetry traffic but must not erase command-loss accounting or make feedback appear instantaneous.

Reducing each per-command STATUS from 75 to 25 bytes saves another 1.25, 2.50 and 5.00 kB/s at those three nominal rates, before adding the slower diagnostic stream. This is a bandwidth saving; the ACK/status return path is outside the sample-to-target-receipt definition of round trip, so it cannot simply be subtracted from that measured latency.

A.7.4 Estimated Communication Savings: Binary UART First

The clearest near-term saving is a binary UART protocol, because every removed byte saves $10/115200\,\mathrm{s} = 0.0868\,\mathrm{ms}$ of serialization. The padded-message experiment already supports this mechanism. As a concrete *unimplemented* size budget, consider a fixed-length serial frame with 2-byte sync marker, 1-byte type/version, 4-byte job token, 2-byte sample sequence and 2-byte CRC: 11 bytes of common framing/check fields. A single command per source sample uses that sample sequence as its reference.

TABLE 17
Illustrative binary UART sizes, including framing and CRC. These proposed formats have not been installed or measured.

Record	Full precision	Reduced precision	Fields beyond the 11-byte common part
Sensor	22 bytes	21 bytes	Start time (4), ADC duration (2), ADCs (4 or 3), flags (1)
Motor target	14 bytes	13 bytes	Two packed targets (3 or 2)

The four-byte microsecond time is relative to an agreed job epoch and wraps after 71.6 min; sequence wraps after 655.36 s at 100 Hz. Unwrapping, reset detection, session changes and maximum accepted age must be specified explicitly. ADC duration preserves the acquisition-end timestamp; sequence gaps and flags can report sample loss, with full counters in diagnostics. CRC validation, resynchronization and bounded parsing are required for a serial byte stream. A CRC detects corruption; it does not authenticate commands. If multiple commands per sample or more diagnostics are needed, allocate additional fields rather than silently overloading IDs.

Relative to the recorded median sensor lengths of 41, 40 and 42 bytes and the 17-byte motor command, the lossless 22/14-byte proposal saves 22–23 bytes at 25/100 Hz and 21 bytes at 50 Hz. Table 18 subtracts only that calculated serial time from the observed median round trip. It assumes all other delays remain unchanged, with no additional buffering or framing overhead; changes in CRC/parsing work, queues and ACK scheduling could alter the result.

TABLE 18
Speculative binary-UART projection, not new measurements. Rate is Hz; times are ms. Bytes saved are per sensor-plus-command pair. The last column additionally drops three ADC LSBs and uses six-bit motor magnitudes, saving two further bytes (0.174 ms per cycle).

Rate	Bytes saved	Wire saving	Projected round trip	Projected spare	Spare, reduced precision
25	22	1.91	6.3	33.7	33.9
50	21	1.82	6.3	13.7	13.9
100	23	2.00	6.3	3.7	3.9

Thus lossless binary UART could plausibly shorten the typical cycle by about **1.8–2.0 ms**, leaving roughly 33.7, 13.7 and 3.7 ms spare at 25, 50 and 100 Hz. The proposed quantization adds only about 0.17 ms to that saving; retaining sensor and motor precision initially is therefore attractive. Neither projection eliminates Wi-Fi/OS tails or demonstrates that 100 Hz will always meet its deadline.

Shrinking only the UDP packets leaves the UART bottleneck unchanged. Even saving 100 network bytes is just 8 μs of raw serialization on a 100 Mb/s link, or 0.133 ms at an illustrative 6 Mb/s radio data rate, before link overhead. The actual Wi-Fi data rate and contention vary, so these are arithmetic examples, not measured savings. Smaller or less frequent STATUS packets may reduce queue pressure, but that gain is unmeasured. The measurements support

prioritizing lossless UART packing, then compact headers/status, and only then evaluating whether reduced numerical precision is worthwhile.

APPENDIX B
TELEMETRY VIDEO

The recording command is `onDesktop/runOperatorVideo`. It starts the same plant run as `runOperator`, including the initialization datagram of Section 5.2, and then burns a timing banner into the webcam movie. It uses the clock summary already stored by Appendix A. It does not collect a new per-job GPIO or UDP clock map. The September movies in Section A.5 used that older map; their one-way split is not what this banner shows.

B.1 Capture

Camera name, frame size and timing limits are read from `onDesktop/video.json`. The installed file names the camera `Web Camera`, requests a 1920×1080 frame, and sets

$$T_{\mathrm{record}} = T_{\mathrm{startup}} + T_{\mathrm{drive}} + T_{\mathrm{tail}}$$

with $T_{\mathrm{startup}} = 30\,\mathrm{s}$, $T_{\mathrm{tail}} = 2\,\mathrm{s}$ and a maximum of 45 s. The command refuses to start the camera or the controller when T_{record} exceeds that maximum. The default drive is 8 s, so the default recording is 40 s. A 10 s drive records 42 s. `--duration` sets the drive window, the same window `runOperator` gives both programs. `--camera` replaces the camera name for one run. `--speed` is passed through. The wheels turn unless `--disable-motors` is present. The robot path remains Wi-Fi 10.0.0.20.

The recorder is `onDesktop/video/record_timed.swift`. It must print `RECORDING` before the controller starts. Each saved frame carries the video time and a Desktop `CLOCK_MONOTONIC` timestamp. The capture clock is converted through host time onto the same monotonic clock the C++ operator uses, so a banner row can be chosen by comparing that timestamp with the operator event log. The Swift tools are compiled into `lineFollowerDistributedBin/` when their sources are newer than the binaries.

After `runOperator` returns, the script copies the operator record and, from the controller log, the Pi `events.csv` and `summary.json`. One directory under the repository `tmp/video/` holds `raw.mp4`, the frame clock, both event logs, `overlay.json`, `telemetry\_preview.png` and `telemetry.mp4`. That directory is the recording. It is not one of the three configuration files kept on the Pi.

B.2 Banner

`onDesktop/video/prepare_overlay.py` builds one row per recorded frame. `overlay_video.swift` draws a 360 px banner across the bottom of the 1920×1080 frame, with a divider at $x = 920$. The rendered movie begins at the first frame after arming and ends two seconds after the first stop. The longer raw file, including the startup allowance, is left beside it.

The left side follows the latest Desktop decision whose log time is at or before the frame. It shows the local date, the three ADC readings, the signed targets and whether the wheels are enabled, Desktop compute time, transmit latency, the Teensy round trip when a motor acknowledgement exists, and the time since arming. Two lines under that list count commands for the whole job: issued by the Desktop, accepted by the Pi, and confirmed by the Teensy, then how many the Pi replaced before the UART write and how many remain unconfirmed. A missing acknowledgement is not counted as a lost command. Replaced commands are excluded from that remainder.

The right side is fixed for the whole clip. Quartiles use nearest rank: for n sorted values and fraction p, the reported value is the entry at index $\lceil np \rceil - 1$. Compute time is printed to three decimal places; the other rows use one. Round trip and the sample period are measured without the Desktop clock map, and the renderer draws those two rows in a separate color.

B.3 Times on the banner

Desktop compute time is the operator's target time minus the time it received that sample, both on the Desktop monotonic clock.

When `boot.json` and `clock_calib.json` share a hash8, transmit latency is rebuilt for each decision from the same mapping the operator uses. Let t_{c} be the Teensy sample-complete time in milliseconds, o the nominal Pi$-$Teensy offset in milliseconds, m the median Wi-Fi midpoint in milliseconds, and t_{rx} the Desktop receipt time in milliseconds. The mapped completion instant and the transmit latency are

$$t_{\mathrm{D}} = t_{\mathrm{c}} + o + m, \qquad \tau = t_{\mathrm{rx}} - t_{\mathrm{D}}. \tag{23}$$

The midpoint m is a clock origin, as in Section 5.2. The note under the table is the uncertainty u of Equation (12), and the three columns are the quartiles of τ.

When the hashes differ, the event log does not keep the per-sample delivery bound. The live transmit number, the first column and the median are then the run median of that bound, and the third column is its 99th percentile. The note says so. Equation (23) is not used for that run.

Round trip uses only the Teensy clock. If t_{a} is the ADC start and t_{k} is the Teensy time in the motor acknowledgement, both in microseconds,

$$L = (t_{\mathrm{k}} - t_{\mathrm{a}})/1000 \qquad [\mathrm{ms}]. \tag{24}$$

Spare time is the sample period minus L. At the 100 Hz default the period row is 10.0 ms in every column. Cycles without a Teensy acknowledgement contribute to the command counts and to compute time, and they leave the round-trip cell blank.

B.4 Motors-disabled check

On October 11, 2026, at 13:35 Pacific time, `runOperatorVideo --duration 8 --disable-motors` recorded the track with the wheels stopped. The drive lasted 7.979 s after arming. The clock file on the Pi belonged to an earlier boot, so the transmit row is the raw delivery bound: median 5.2 ms and 99th percentile 6.8 ms. Desktop compute had quartiles 0.007, 0.009 and 0.011 ms. Equation (24) had quartiles 7.1, 7.5 and 8.7 ms, and the spare quartiles were 1.2, 2.5 and 2.9 ms. The Desktop issued 777 commands, the Pi accepted 777, the Teensy confirmed 768, the Pi replaced 9 before the UART write, and none remained unconfirmed. Figure 14 is one frame from that movie, two seconds after arming.

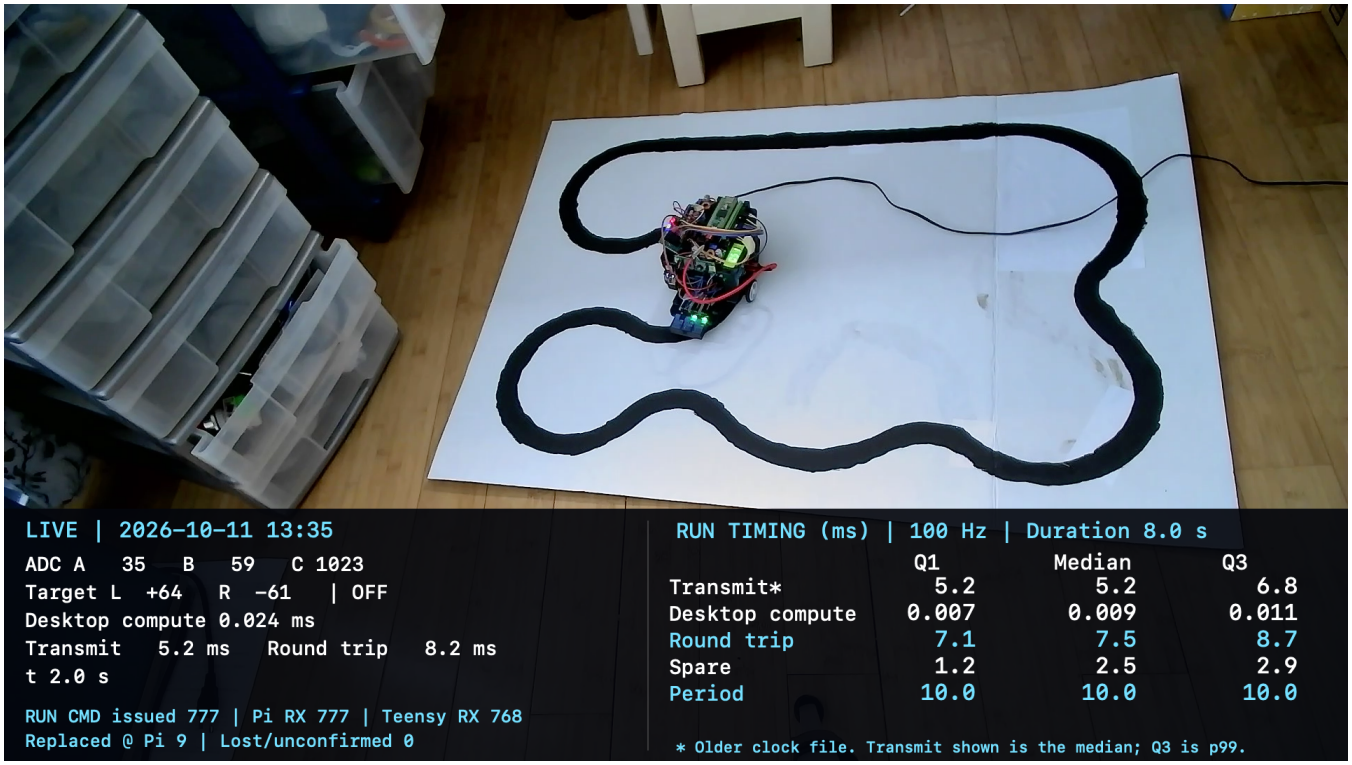


Fig. 14. Telemetry frame at $t = 2.0$ s from the October 11 motors-disabled recording. The clock hashes did not match, so the transmit row is the raw delivery bound described above. Round trip and spare use the Teensy clock.

APPENDIX C
LONG-TERM GOAL: REMOTE NEURAL CONTROL

This appendix preserves the generalized requirements, the original functional Figure 1, and the original high-workload Tables 1 and 2. They describe a research goal, not the installed robot or the first remote-control demonstration. A binary local I/O link, containerized neural simulator and 1 kHz traffic are future choices; no replacement Pi model is specified. The current robot uses a Pi Model B Plus, a 2.4 GHz USB Wi-Fi adapter and 115200-baud UART. Its existing local controller defaults to 20 Hz; the implemented remote demonstration targets 100 Hz and has been tested with motor output disabled.

The longer-term objective is line following through a remote spiking neural generator, including comparison of input-neuron selections and tolerable feedback delay. Live neurons remain an alternative source. A future forward-only wheel law is an experimental restriction; the current local controller permits reverse steering. The robot-motion model is retained separately in Appendix D.

C.1 Future Functional Communication Schematic

Figure 15 preserves the original Figure 1 as a future concept; robot power distribution is in Section 3. The desktop's USB-C Ethernet adapter connects to an ASUS *LAN* port. The current Pi uses a 2.4 GHz USB Wi-Fi adapter; use that known connection as the initial transport baseline. In this future arrangement the desktop remains wired; only the last hop to the moving robot is wireless. The programming laptop reaches both computers through that same router, without requiring a second Pi radio. Router specifications are documented by ASUS.

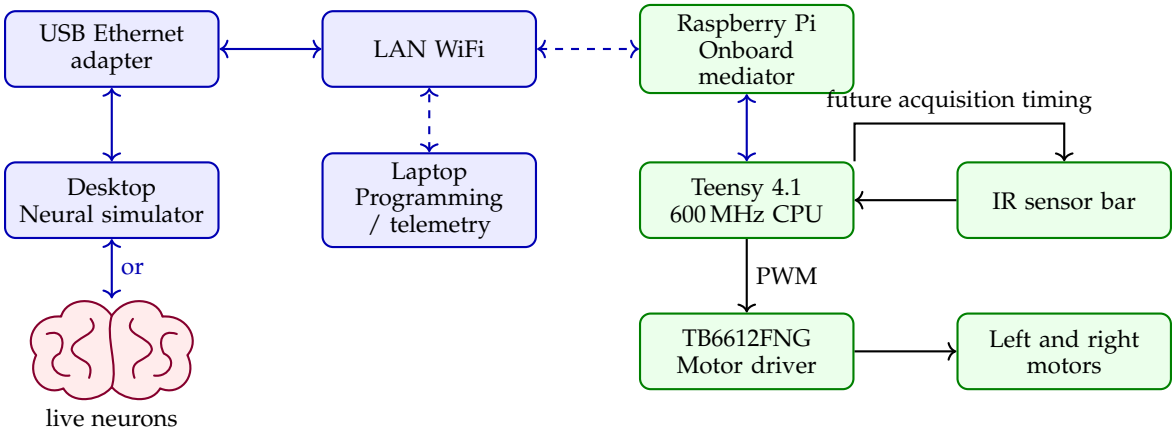


Fig. 15. Long-term functional layout (original Figure 1). The separate programming laptop and neural source are future elements, not the current deployment; the Pi model is left unspecified in this concept. Green boxes are on the robot platform. Solid links are wired; dashed links are wireless. Live neurons are shown as an alternative to the neural simulator, interfaced through the desktop. The programming laptop accesses the desktop and Pi through the ASUS router.

C.2 Future Workload and Connectivity Requirements

Tables 19 and 20 preserve the original Tables 1 and 2 as future traffic sizing for the functional layout above. Rates use decimal kB/s. Payload bandwidth excludes framing, timestamps, sequence numbers, status, acknowledgements, overflow records, and transport overhead. The revised initial design uses 1 kHz feedback, steering commands, and sensor reports; these controller rates are explicit configurable choices, distinct from the required 1 ms neural step. A lower controller rate must be recorded with its command-hold policy and included in the delay experiment.

TABLE 19
Desktop–Pi connectivity requirements and initial sizing

Item / direction	Required traffic or connection	Rate, capacity, or timing criterion
Desktop → Pi	One binary UDP datagram per native spike bin; 2-bit counts for every neuron	1,000 frames/s; 50 B/frame at $N = 200$: 50 kB/s; 125 B/frame at $N = 500$: 125 kB/s
Pi → desktop	Selected-channel float32 requests; $m = 20$ or 50 when selecting 10% of neurons	Initially 1,000 commands/s; 80 or 200 B/command: 80 or 200 kB/s; generally $4mf_{\mathrm{cmd}}$ B/s
Combined payload	Both directions at the initial 1 kHz rates	130 kB/s (1.04 Mbit/s) baseline; 325 kB/s (2.60 Mbit/s) sizing case, plus overhead
Physical path	Desktop USB-C / USB 3 adapter → Gigabit Ethernet → ASUS RT-N56U LAN; router ↔ Pi USB 2.4 GHz radio (current baseline)	One wireless hop, a few metres; one Pi Wi-Fi adapter; router USB ports are not the desktop network connection
Delay and cadence	Preserve 1 ms bin timestamps and transmit promptly; measure application-to-application age	Former USB references: 0.3–0.5 ms desktop-to-Pi equivalent and 0.2–0.3 ms return. These are comparison targets, not Wi-Fi guarantees; acceptance comes from the delay-margin experiment
Integrity and jitter	Sequence/bin IDs, command application acknowledgements, clock-offset/drift estimates or round-trip bounds	Report median, 99th/99.9th percentile and maximum observed delay, missing/late frames, longest interruption, and simulation deadline misses

TABLE 20
Pi–Teensy connectivity requirements and initial sizing

Item / direction	Required traffic or connection	Rate, capacity, or timing criterion
Pi → Teensy	Steering u and baseline speed v_0; initial encoding: two float32 values	Initially 1,000 commands/s; 8 B payload: 8 kB/s. Add sequence, timestamp and validity information
Teensy → Pi	All three calibrated reflectances; proposed encoding: three float32 values	Initially 1,000 reports/s; 12 B payload: 12 kB/s. Add sample timestamp, sequence and status
Combined payload	Simultaneous command and sensor streams	20 kB/s (0.160 Mbit/s), plus framing and status; this proposed 1 kHz interface is separate from the current UART controller in Section 4
Physical link	One onboard USB data cable; binary frames over USB CDC virtual serial	Bidirectional, asynchronous; The present Teensy uses external Vin with its USB power bridge cut; any future USB data link must preserve that power arrangement. Preserve frame boundaries in the application protocol
Transport and execution	Command delivery; timestamped sensor delivery; local command latch	Initial estimates: 0.2–0.3 ms Pi-to-Teensy and 0.3–0.5 ms return; verify experimentally. A chosen 1 kHz latch adds up to one local tick after receipt
Validity and load	Bounded queues, stale-command timeout, sample IDs, configured stop/lost-link policy	No unbounded backlog; log missed reports and actual command age. Keep PWM and sensor acquisition independent of host delays

C.3 System Architecture & Workload Segregation

The closed loop has three compute nodes: the remote desktop, the onboard Pi, and the onboard Teensy. A separate programming laptop manages the desktop and Pi but does not participate in the timing-critical control loop.

- **Remote neural simulation desktop:** A stationary macOS desktop running the controlled spiking SLDS in an Ubuntu 24.04 Podman container, with $N = 200$ observed neurons ($N = 500$ sizing case) and a 1 ms step paced to wall-clock time. Use a native `linux/arm64` image on Apple Silicon. The desktop owns the generator state and applies timestamped feedback from the Pi.
- **Onboard mediator:** The Raspberry Pi receives spike packets, computes steering and selected-channel feedback, and retains optical sensor history. Benchmark the current Model B Plus before setting a sustainable workload; no future replacement model is selected. Its one radio carries both the neural traffic and management traffic.
- **Onboard real-time I/O:** One **Teensy 4.1** (600 MHz ARM Cortex-M7) manages local sensor acquisition, hardware timers and independent wheel PWM. The Pi connects to it over USB CDC.
- **Programming laptop:** A future separate service laptop can reach the simulator and Pi through ASUS. In the present setup, Desktop itself performs programming and management (Section 2). This service has no control-

loop deadline and is intended for higher-throughput bulk transfers. Schedule large transfers outside trials; limit live telemetry during trials.

C.4 Desktop Container and Shared-Network Operation

On macOS, Podman runs Ubuntu containers inside a Linux virtual machine. macOS owns the USB Ethernet adapter; the container uses virtual networking, without USB passthrough. Publish the simulator's UDP feedback port, bind its receiver to all container interfaces, and send Pi feedback to the desktop's Ethernet IP address rather than its private container address. For example, publish `-p 0.0.0.0:5001:5001/udp`; the simulator sends spikes to the Pi's UDP port 5000. Use DHCP reservations or fixed addresses on one local subnet and verify inbound access through the desktop firewall. Podman's `--network=host` refers to the Linux VM, not directly to macOS. See the Podman machine and port-publishing documentation.

Keep normal router mode with DHCP enabled for the standalone network and leave WAN unused; use the main LAN with client isolation disabled. For the current adapter choose a suitable 2.4 GHz channel, disable Pi Wi-Fi power saving, and keep other connected networks on different subnets. The current baseline uses the 2.4 GHz USB radio; management traffic still shares the final Wi-Fi hop. No internet connection is needed for the robot LAN. File transfers use reliable protocols such as SSH/SCP; live neural traffic uses binary UDP with explicit loss and lateness handling. A second Pi radio is unnecessary for this topology.

The desktop's simulator, VM scheduling, network queues, Wi-Fi contention and the Pi's processing all contribute delay and jitter. Sustaining 1,000 bins/s does not guarantee sub-millisecond delivery. Measure the full bidirectional workload while the robot moves and while bounded management telemetry is active. Keep simulator and Pi receive loops nonblocking, mark missing spike bins as missing rather than zero, and record deadline misses instead of hiding stalls with bursts of apparently current bins.

C.5 Neural Model: Scope and Baseline

C.5.1 Objective, Dimensions, and Baseline

The robot experiment evaluates **line-following control through spikes**; recovery of the hidden neural state is not an objective and no latent-state tracker is required in the control loop. The robot baseline uses $N = 200$ observed neurons, latent dimension $d = 5$, $K = 3$ modes, and native step $\Delta = 1\,\mathrm{ms}$. The communication sizing case retains $N = 500$. Mode count and latent dimension are separate parameters: the three modes share a five-dimensional latent state.

The desktop simulator holds a fixed bank $\{A_s, \mu_s, Q_s\}_{s=0}^{K-1}$, the loading $C \in \mathbb{R}^{N\times d}$, calibrated baseline offsets $b_0 \in \mathbb{R}^N$, and an uncontrolled transition matrix Π^0. Here A_s is a *discrete-time latent* matrix, not a synaptic connectivity matrix. Retain the baseline decay scale $\tau_A = 10\,\mathrm{ms}$ and uncontrolled mean mode dwell $\tau_{\mathrm{dwell}} = 300\,\mathrm{ms}$ as initial settings; both are swept in robot experiments. Fixed model coefficients do not imply stationary activity under sensor feedback. The zero-input generator is the stationary reference; the driven robot is a controlled switching system.

C.5.2 Fixed Model Construction and Stationary Reference

All times in equations are in seconds. The mode is $s_k \in \{0, \dots, K-1\}$ and the latent state is $x_k \in \mathbb{R}^d$. A common uncontrolled mean dwell gives

$$\Pi^0_{ii} = 1 - \frac{\Delta}{\tau_{\mathrm{dwell}}}, \qquad \Pi^0_{ij} = \frac{\Delta}{(K-1)\tau_{\mathrm{dwell}}}\ (i \neq j), \qquad \pi^0_i = \frac{1}{K}. \tag{25}$$

Here π^0 is the stationary mode distribution. With $K = 3$ and $\tau_{\mathrm{dwell}} = 0.30$ s, the diagonal probability is $0.996\overline{6}$ and each off-diagonal probability is $0.001\overline{6}$. The latent state remains continuous across a mode change; it is not reset to the new center.

Construct the mode matrices in continuous time, then discretize:

$$F_s = U\left(D_s + \frac{\nu}{\tau_A} J_s\right) U^\top, \qquad A_s = \exp(F_s\Delta), \qquad U^\top U = I_d. \tag{26}$$

The dimensionless parameter $\nu \geq 0$ controls non-normal coupling. D_s is real block diagonal with scalar blocks $-a/\tau_A$, $a > 0$, or damped rotation blocks

$$\frac{1}{\tau_A}\begin{pmatrix} -a & -\omega \\ \omega & -a \end{pmatrix}, \qquad a > 0. \tag{27}$$

J_s is strictly block upper triangular in that partition, normalized to $\|J_s\|_F = 1$. Its coupling changes transient behavior without changing the eigenvalues of the diagonal blocks. U is a fixed shared orthogonal frame. Choose distinct centers $\mu_s \in \mathbb{R}^d$ and distinct mode matrices; save them rather than regenerating them during a run. One reproducible construction draws seeded Gaussian vectors z_s, sets $\bar{z} = K^{-1}\sum_s z_s$, and uses $\mu_s = \sigma_\mu(z_s - \bar{z})$, with center scale

σ_μ recorded. Similarly, a seeded Gaussian matrix followed by QR supplies U. The block coefficients, coupling matrices, and seed are configuration data.

For diffusion matrix $L \in \mathbb{R}^{d\times d}$, with units of latent state per square root second,

$$Q_s = \int_0^{\Delta} e^{F_s t} L L^\top e^{F_s^\top t}\, dt, \qquad \Sigma_s = A_s \Sigma_s A_s^\top + Q_s, \qquad \bar{\Sigma} = \frac{1}{K}\sum_s \Sigma_s. \tag{28}$$

A simple diffusion choice is $L = \sigma_L I_d$; σ_L is saved with the bank. Compute exponentials, integrals, Lyapunov solutions, and a factor $Q_s = H_s H_s^\top$ offline on the desktop; share the accepted fixed model with the Pi. The desktop simulator generates $w_k = H_s \xi_k$, $\xi_k \sim \mathcal{N}(0, I_d)$. The fixed-mode covariance Σ_s is not the conditional covariance of the switching process, which also contains transitions between centers.

Measure non-normality and transient gain over a declared horizon H_{tr} (initially $10\tau_A$):

$$h_s = \frac{\|F_s F_s^\top - F_s^\top F_s\|_F}{\|F_s\|_F^2}, \qquad G_s^{\mathrm{tr}} = \max_{0\le j\le \lfloor H_{\mathrm{tr}}/\Delta\rfloor} \|A_s^j\|_2. \tag{29}$$

Require $\rho(A_s) < 1$ in each mode and measured $G_s^{\mathrm{tr}} > 1$ for a transient-amplifying bank. A $\nu = 0$ bank is the normal-dynamics comparison. The time τ_A sets asymptotic decay: for a scalar decay block with $a = 1$, a perturbation scales as $e^{-j\Delta/\tau_A}$. It does not set the mode dwell or guarantee an equally fast response in the presence of transient amplification.

For the uncontrolled switching linear part, form the $Kd^2 \times Kd^2$ block operator

$$\mathcal{S}^0_{ji} = \Pi^0_{ij}(A_j \otimes A_j), \qquad \rho(\mathcal{S}^0) < 1. \tag{30}$$

This tests mean-square stability of the zero-input bank; $\otimes$ denotes the Kronecker product and ρ the spectral radius. The controlled bank must additionally satisfy the switching-stability requirement below. Record spectral margins, transient gains, and numerical acceptance thresholds with each accepted bank.

C.5.3 Neural Loading, Rate Calibration, and Initialization

Draw a fixed full-column-rank loading $C = \sigma_C \widehat{C} \in \mathbb{R}^{N\times d}$; for example, draw independent Gaussian rows and normalize each row of $\widehat{C}$ to unit Euclidean norm. Reject an ill-conditioned loading and record its singular values. Here σ_C sets log-rate modulation depth. Define componentwise clipping by $\mathrm{clip}(z; l, h) = \min(h, \max(l, z))$; logarithms of rates use their numerical values in Hz.

Generate a discarded zero-input pilot of T_p bins after burn-in, using the latent update in Equation (53) with all inputs zero. Write $b_{0,i} = \widehat{b}_i + \gamma$, where $\widehat{b}_i$ are fixed neuron offsets (zero is the simplest choice). For the pilot states $x_t^{(p)}$, choose the scalar γ by monotone root finding so that

$$\bar{f}_i(\gamma) = \frac{1}{T_p}\sum_{t=1}^{T_p} \exp\Big[\mathrm{clip}(\widehat{b}_i + \gamma + C_{i:}x_t^{(p)}; \log 0.1, \log 100)\Big], \tag{31}$$

$$\frac{1}{N}\sum_{i=1}^{N} \bar{f}_i(\gamma) = 8 \text{ Hz}. \tag{32}$$

This specifies a population mean target, useful for spike bandwidth estimates. Report $\mathrm{median}_i\, \bar{f}_i$ separately. Report both clamp fractions

$$q_\pm = \frac{1}{NT_p}\sum_{t,i} \mathbf{1}\{\eta_{ti}^{\mathrm{raw}} \in \mathcal{C}_\pm\}, \qquad \eta_{ti}^{\mathrm{raw}} = b_{0,i} + C_{i:}x_t^{(p)}, \quad \eta_- = \log 0.1, \quad \eta_+ = \log 100, \tag{33}$$

where $\mathcal{C}_+ = [\eta_+, \infty)$ and $\mathcal{C}_- = (-\infty, \eta_-]$, and $\mathbf{1}$ is the indicator function. If clamp occupancy exceeds the declared calibration limit, reduce loading or latent scales and recalibrate before evaluation. Freeze C, b_0 and all bank parameters after calibration.

Initialize the generator with $s_0 \sim \mathrm{Categorical}(\pi^0)$ and $x_0 \sim \mathcal{N}(\mu_{s_0}, \Sigma_{s_0})$, then discard a separate zero-input burn-in. This initial Gaussian is a convenient seed, not an exact stationary switching-state draw. Start with a burn-in of $\max(20\tau_{\mathrm{dwell}}, 100\tau_A)$ and extend it if rate and latent moment checks have not stabilized. Save pilot length, burn-in length, initial state, and separate random seeds for model construction, calibration, and evaluation. Spikes are conditionally independent across neurons given the latent state and applied input; their shared fluctuations come from the latent dynamics. No synaptic connectivity or biological refractory period is represented.

C.5.4 FFC and Projected-Transient Neuron Selection

Use subspace dimension $r = 2$ for $d = 5$. Define the feed-forward controllable (FFC) proxy here as the dominant covariance frame:

$$V_{\mathrm{FFC},s} = \mathrm{eig}_r(\Sigma_s), \qquad V_{\mathrm{FFC}} = \mathrm{eig}_r(\bar{\Sigma}), \tag{34}$$

where eig_r returns orthonormal eigenvectors for the r largest eigenvalues. For each fixed mode, $\Sigma_s = \sum_{j\geq 0} A_s^j Q_s (A_s^\top)^j$ is the infinite-horizon noise-input Gramian. It need not equal the Gramian of the selected bias actuator B_s.

Define a comparison based on peak-transient directions:

$$j_s^\star = \arg\max_{0\leq j\leq \lfloor H_{\mathrm{tr}}/\Delta \rfloor} \|A_s^j\|_2, \qquad A_s^{j_s^\star} = U_s^{\mathrm{tr}} S_s^{\mathrm{tr}} (V_s^{\mathrm{tr}})^\top, \tag{35}$$

$$Z_s = (I_d - V_{\mathrm{FFC},s} V_{\mathrm{FFC},s}^\top) U_{s,1:r}^{\mathrm{tr}}, \qquad P_s^{\mathrm{tr}} = \mathrm{orth}(Z_s), \tag{36}$$

$$\bar{\Gamma}^{\mathrm{tr}} = \frac{1}{K}\sum_s P_s^{\mathrm{tr}} (P_s^{\mathrm{tr}})^\top, \qquad V_{\mathrm{tr}} = \mathrm{eig}_r(\bar{\Gamma}^{\mathrm{tr}}). \tag{37}$$

Choose the earliest maximizing lag in a tie. If Z_s has rank below r, use $P_s^{\mathrm{tr}} = U_{s,1:r}^{\mathrm{tr}}$ and flag the fallback. This projected-transient heuristic is not a proof of feedback controllability. Averaging projectors avoids the sign ambiguity of averaging basis vectors. The singular values σ_ℓ of $V_{\mathrm{FFC}}^\top V_{\mathrm{tr}}$ give principal angles $\theta_\ell = \arccos(\mathrm{clip}(\sigma_\ell; 0, 1))$; consensus frames need not be orthogonal.

For either frame $V_\bullet$, map to neurons and rank normalized row energies:

$$W_\bullet = C V_\bullet, \qquad I_i^\bullet = \frac{\|W_\bullet(i,:)\|_2^2}{\max_j \|W_\bullet(j,:)\|_2^2}, \qquad \mathcal{I}_\bullet = \mathrm{Top}_m(I_1^\bullet, \ldots, I_N^\bullet). \tag{38}$$

Break score ties by neuron index. If every row energy is zero, reject that selection. Compare the top m FFC neurons with the top m transient neurons and report overlap. For mixed inputs, take $\lfloor m/2 \rfloor$ top FFC neurons and fill the remaining slots with the highest-ranked unused transient neurons. Draw the random comparison uniformly without replacement using a saved seed. Keep m identical across comparisons; exclusive top-score sets can be smaller and must not silently change the actuator budget.

C.5.5 Selected Neurons and Explicit Input Authority

The Pi knows the fixed model and selects an input subset $\mathcal{I}$ of size m at initialization. Its selection matrix $S_{\mathcal{I}} \in \mathbb{R}^{N\times m}$ contains the corresponding columns of the identity. Start with $m = \mathrm{round}(0.10N)$, hence 20 channels at $N = 200$, and keep m configurable. FFC row-energy scores derived from CV_{FFC} provide the first selection rule. The steering decoder may use **all** N **neurons**; input selection and output selection are independent.

The zero-input generator changes spike rates through $b_0 + Cx$; emission bias alone does not actuate x or the mode. The robot therefore adds two explicit control maps:

- $B_s \in \mathbb{R}^{d\times m}$: selected-channel input to the latent dynamics;
- $R_i \in \mathbb{R}^{K\times m}$: selected-channel input to transition logits, conditional on source mode i.

These maps are part of the robot specification; they cannot be inferred as physical stimulation effects merely from knowing A_s or the FFC frame. A reproducible initial synthetic choice is

$$B_s = \beta_s C^\top S_{\mathcal{I}} / \zeta, \qquad \zeta = \max\{\|C^\top S_{\mathcal{I}}\|_2, \epsilon\}, \tag{39}$$

where $\epsilon > 0$ is a saved numerical regularizer in the chosen latent units and β_s sets latent displacement per native step per unit applied command. This back-projection is a declared modeling choice, not evidence of neuron-to-latent causality in the stationary generator. Precompute B_s on the Pi; do not recompute it at every generator tick.

For a candidate input set, inspect the finite-horizon, fixed-mode input Gramian

$$W_s(H) = \sum_{j=0}^{H-1} A_s^j B_s B_s^\top (A_s^\top)^j, \tag{40}$$

and the resulting signed steering response. Full latent controllability is not required for the scalar steering task, but the chosen input must have adequate authority in the decoded direction. FFC ranking alone is not a closed-loop performance guarantee. The projected peak-transient comparison defined below is a heuristic; its neuron scores do not establish input controllability.

C.5.6 Bias Transport Delay and Neural Response Delay

The Pi forms bounded selected-channel requests $a_k^{\mathrm{cmd}} \in \mathbb{R}^m$. Each message carries a sequence number and command timestamp. Model link/queue delay D_k in native bins and a separate neural input time constant τ_b:

$$a_k^{\mathrm{del}} = a_{k-D_k}^{\mathrm{cmd}}, \tag{41}$$

$$\tilde{a}_k = \alpha_b \tilde{a}_{k-1} + (1-\alpha_b) a_k^{\mathrm{del}}, \qquad \alpha_b = e^{-\Delta/\tau_b}, \tag{42}$$

$$\delta b_k = g_b S_{\mathcal{I}} \tilde{a}_k. \tag{43}$$

Set $\alpha_b = 0$ when $\tau_b = 0$ and use zero input before the first command. The delay expression represents an ordered stream; a timestamped event queue implements variable latency, packet loss, and the configured stale-command policy without replaying obsolete commands. The desktop simulator latches inputs at a specified native-bin boundary and reports the sequence number and bin index when each request first enters its input filter. This acknowledgement distinguishes transport delay from slow neural response. Cross-device command ages require an estimated clock offset/drift or host round-trip timing bounds; raw timestamps from independent clocks are not directly subtracted.

The default $g_b = 1$ preserves direct log-rate biasing of selected neurons. The same filtered command also enters the latent and mode equations below. Thus an input can affect rates directly, affect them through latent evolution, and alter future mode transitions. These routes must be tested separately: $g_b = 0$ isolates dynamical effects; $B_s = R_i = 0$ gives an emission-bias-only control. This prevents direct rate modulation from being mistaken for a benefit of non-normal dynamics.

C.5.7 Controllable Markov Transitions

Let $i = s_k$. Input modifies the next-mode transition row through bounded logits,

$$h_{ij,k} = \log \Pi^0_{ij} + \mathrm{clip}\big((R_i \tilde{a}_k)_j; -\ell_{\max}, \ell_{\max}\big), \tag{44}$$

$$\bar{\Pi}_{ij,k} = \frac{\exp(h_{ij,k} - \max_j h_{ij,k})}{\sum_q \exp(h_{iq,k} - \max_j h_{ij,k})}. \tag{45}$$

Use strictly positive Π^0_{ij} for this baseline. Bound exit probability to avoid switching on nearly every 1-ms tick or effectively locking a mode:

$$p^{\mathrm{exit}}_{i,k} = \mathrm{clip}\left(1 - \bar{\Pi}_{ii,k}; \Delta/\tau_{\max}, \Delta/\tau_{\min}\right), \tag{46}$$

$$\Pi_{ii,k} = 1 - p^{\mathrm{exit}}_{i,k}, \qquad \Pi_{ij,k} = p^{\mathrm{exit}}_{i,k} \frac{\bar{\Pi}_{ij,k}}{1 - \bar{\Pi}_{ii,k}} \quad (j \neq i), \tag{47}$$

$$s_{k+1} \sim \mathrm{Categorical}(\Pi_{i:,k}). \tag{48}$$

Initial bounds are $\tau_{\min} = 50\,\mathrm{ms}$ and $\tau_{\max} = 3\,\mathrm{s}$; these bound the exit hazard, not the length of each realized dwell. At zero input the construction reproduces Π^0, whose 300-ms mean dwell lies inside these limits. Transitions remain stochastic: a command biases their probability, not their exact occurrence time.

One initial mode-targeting map is

$$(R_i)_{j,:} = \kappa(\mu_j - \mu_i)^\top M B_i, \tag{49}$$

where $M = (\bar{\Sigma} + \epsilon I)^{-1}$ is a fixed positive-definite latent scaling matrix, $\bar{\Sigma}$ is the mean fixed-mode Lyapunov covariance, and κ is a calibrated gain. This favors destinations aligned with the requested latent displacement. Verify that rows actually differ on the allowed inputs: adding an identical logit to every destination has no effect. The mode policy sees only the selected-channel command, not a hidden line-error shortcut. The actual mode is available locally to the desktop simulator; the Pi need not estimate or receive it for steering. Initial dimensionless limits are $a_{\max} = 1$ and $\ell_{\max} = 2$. Calibrate and store β_s, κ, the encoder G, and the decoder weights before evaluation; reject a bank/input map whose bounded commands cannot measurably alter destination probabilities and produce steering of both signs. Sweep τ_b from a zero-lag reference through 5, 10, 20, and 40 ms. For a mode-only ablation, retain the already calibrated R_i when disabling B_s in the latent update; do not recompute R_i to zero.

C.5.8 Latent Evolution and Spike Generation

After the controlled mode draw, use

$$x_{k+1} = \mu_{s_{k+1}} + A_{s_{k+1}}(x_k - \mu_{s_{k+1}}) + B_{s_{k+1}} \tilde{a}_k + w_{k+1}, \tag{50}$$

$$w_{k+1} \sim \mathcal{N}(0, Q_{s_{k+1}}), \tag{51}$$

$$\eta_{k+1} = \mathrm{clip}(b_0 + C x_{k+1} + \delta b_k; \log 0.1, \log 100), \tag{52}$$

$$\lambda_{k+1} = \exp(\eta_{k+1}), \qquad n_{k+1,i} \sim \mathrm{Poisson}(\Delta \lambda_{k+1,i}). \tag{53}$$

This ordering defines the bin affected by an accepted request. All gains, input limits, initialization, random seeds, and timing parameters are saved with each trial. With zero input, this reduces to the stationary generator. Because the transition policy is now input dependent, the zero-input mean-square stability test in Equation (30) is insufficient. A conservative sufficient certificate is a common $P \succ 0$ with $A_s^\top P A_s - P \prec 0$ for every mode; bounded inputs and mode centers then act as bounded forcing with finite process-noise covariance. Failure to find a common certificate is not itself proof of instability, but requires a more specific controlled-switching analysis and stress testing.

The baseline population mean-rate target is 8 Hz/neuron, calibrated by Equation (32). Also record the median across neuron mean rates; it need not equal the population mean. Keep the 100-Hz upper clamp initially. A neuron at 8 Hz reaches 100 Hz after an unclipped log-rate increase of $\log(100/8) \simeq 2.53$, which can arise from mode centers, non-normal transients, or input. Whether this occurs must be measured, not inferred from the baseline rate alone. Report unclipped log-rate quantiles, per-neuron clamp occupancy, and command-conditioned saturation. Do not raise the clamp merely to obtain more steering authority. A Poisson rate cap bounds the conditional intensity, not the number of spikes in an individual bin, and does not impose a biological refractory period. A refractory generator would be a separate, recalibrated variant.

C.5.9 Pi Readout, Sensor History, and Forward Steering

The Pi retains timestamped histories of the three calibrated reflectances, estimated line position, confidence, spike packets, and previous commands. The initial selected-channel encoder is a bounded causal map of line-error history, for example

$$a_k^{\mathrm{cmd}} = \mathrm{clip}\left(G\phi_k; -a_{\max}, a_{\max}\right), \qquad \phi_k = [e_k, \dot{e}_k^{\mathrm{filtered}}, \ldots]^\top, \tag{54}$$

with confidence gating and a declared lost-line policy. Teensy samples sensors and reports them to the Pi; it does not need to reconstruct the neural state. This placement gives the Pi ownership of sensor history and of the bias requests sent to the desktop simulator.

The motor decoder uses all neurons if desired. For causal filtered rates,

$$r_k = \alpha_r r_{k-1} + (1-\alpha_r) n_k/\Delta, \qquad \alpha_r = e^{-\Delta/\tau_r}, \qquad u_k = \tanh\big(w^\top (r_k - r_0) + c\big). \tag{55}$$

No $\hat{x}_k$ or inferred mode is needed. Calibrate w, r_0, c so a centered line gives near-zero mean steering across modes, and measure the remaining variance and switch-triggered excursions. Use generator truth only for offline diagnosis and evaluation; a decoder supplied with true mode labels is an explicitly labelled oracle ablation, not the operational controller.

Use $u \in [-1, 1]$ for steering, reserving x for the latent state. For $u > 0$ (right turn), slow the right wheel; for $u < 0$ (left turn), slow the left wheel:

$$v_L^* = v_0\big(1 - \max(0, -u)\big), \qquad v_R^* = v_0\big(1 - \max(0, u)\big). \tag{56}$$

At $u = 0$ both targets equal v_0; at $|u| = 1$ the inner wheel stops. Neither reverses. If both must remain strictly moving, impose $|u| \le u_{\max} < 1$. For wheel separation L_w, the ideal mean speed is $v_0(1 - |u|/2)$ and clockwise yaw rate is $v_0 u / L_w$. At full steering the minimum ideal axle-centre turn radius is $L_w/2$; forward-only steering cannot turn about the axle centre.

Teensy has two independent wheel command channels. With no encoders, the physical implementation is calibrated PWM and cannot verify actual speed; an ideal speed actuator is only the first simulation condition. Add unequal gains, lag, deadband, battery/load effects, and slip to subsequent trials. Later encoders can close independent local wheel-speed loops, and absolute motion tracking can supply evaluation ground truth without changing the neural steering interface.

C.6 Delay-Margin Experiment and Comparisons

The main experiment estimates the **largest tolerable feedback delay** for a declared speed, track family, neural model, and success criterion. There is no single universal delay limit. Sweep added command delay initially over 0, 2, 5, 10, 20, 40, 80, and 160 ms, refine around failures, and repeat across seeds; separately sweep τ_b, rate-filter time τ_r, and timing jitter. Zero added delay still retains the sampled update ordering and any modeled actuator response. Link latency, input filtering, latent response, random mode-switch waiting time, stochastic first-spike waiting time, readout filtering, and wheel lag are distinct contributions. Do not interpret the first observed post-command spike as a deterministic latency measurement: compare repeated paired trials with matched seeds and measure changes in expected rates and steering as well as first-spike statistics.

For each setting, report RMS and high-percentile line error, line-loss probability, completed distance/laps, steering variation, clamp occupancy, and actual command age. Define success thresholds before the sweep and report success probability across seeds rather than assuming a monotone binary delay boundary. Start with ideal forward wheel dynamics, then repeat with motor mismatch and jitter. Include the following matched comparisons:

- FFC-selected, random, mixed, and projected-transient-selected input neurons as defined above, with the same input count, bias limits, comparable delivered input energy and firing-rate budget, and normalized latent/mode gains;
- all-neuron versus sparse spike readouts, each calibrated with the same training budget;
- emission-only, latent-only, mode-only, and combined input routes;
- stochastic spikes versus conditional expected rates, and a conventional sensor controller as a performance reference.

For meaningful input-subset comparisons, keep the underlying neuron-to-latent and neuron-to-logit mapping rule fixed and select its columns; do not give each subset an independently optimized arbitrary actuator. No conventional controller contributes hidden corrective steering during evaluation of the neural loop. A one-mode run can diagnose dynamics and transport, but the primary robot experiment retains the configurable three-mode model.

C.7 Real-Time Control Protocol & Closed-Loop Data Pipeline

The asynchronous loop is desktop spike generation → Pi readout → Teensy wheel commands and sensor acquisition → Pi sensor-history encoder → desktop neural feedback. The desktop–Pi link uses UDP over wired Ethernet plus one Wi-Fi hop; the Pi–Teensy link uses USB CDC. Tables 19 and 20 give the traffic requirements. Neither link is a blocking request–response chain.

C.7.1 Data Payloads & Packing

Use packed binary frames with an explicitly defined byte order, frame type, length, sequence and timestamp/bin fields. USB CDC is a byte stream, so handle partial reads and multiple frames per read. UDP preserves datagram boundaries, but the receiver must detect missing, duplicate, reordered and late packets. UDP does not remove Wi-Fi's own link-layer retries. Keep application queues bounded and log all drops; declare the missing-bin decoder policy and stale-command policy with each trial.

- **Desktop → Pi (spikes):** One native bin per UDP datagram at 1 kHz. The default 2-bit format encodes counts 0–3 and uses 50 bytes for 200 neurons or 125 bytes for 500. Framing and a clipping indicator/counter are additional; counts above 3 are logged and optionally sent as overflow records. A 1-bit occupancy ablation uses 25 or 63 bytes, respectively. Send each bin promptly; batching ten bins adds up to 9 ms of waiting to the oldest bin.
- **Pi → Teensy (actuation):** Initially two float32 values, steering u and speed v_0, plus sequence, timestamp and validity information. Teensy applies Equation (56) independently to the wheels, with a local stale-command timeout and configured stop policy.
- **Teensy → Pi (sensors):** Proposed: three float32 calibrated reflectances, plus acquisition timestamp, sequence and status. The Pi retains sensor history and computes line-error features.
- **Pi → desktop (neural feedback):** Timestamped selected-channel requests a^{cmd} or compact encoder coefficients. A float32 vector is 80 bytes for $m = 20$ and 200 bytes for $m = 50$. Configure fixed maps and neuron indices once; compact coefficients are valid only if the desktop expands the same configured map. Each accepted request is acknowledged with its sequence and effective generator bin. Between accepted commands, hold the latest valid request before the neural input filter, subject to the configured expiry policy.

For a constant per-neuron rate f, put $\mu = f\Delta$. The probability of multiplicity is $p_{\geq 2} = 1 - e^{-\mu}(1 + \mu)$; converting counts to occupancy loses the fraction $1 - (1 - e^{-\mu})/\mu$ of expected spikes. At 8 Hz and 1 ms these are approximately 0.00318% of bins and 0.399% of spikes; at 100 Hz they are 0.468% and 4.84%, respectively. Thus 1-bit packing is a reasonable measured approximation near baseline, but 2-bit counts cheaply preserve most multiplicity at higher transient rates. Two bits encode 0–3, three encode 0–7, and four encode 0–15: an inclusive clip value of 4 needs three bits, and 8 needs four. Preserve full counts in the reference simulator and compare each wire-format approximation with it. At 100 Hz, the probability of a count exceeding 3 is about 3.85×10^{-6} per bin, small but not zero. Record overflow rather than silently treating the clipped distribution as exact Poisson data.

C.7.2 Closed-Loop Timing Budget and Validation

The former all-USB estimate of approximately 12.0–12.7 ms is not a prediction for the revised wireless/container system. Measure the terms below under the actual bidirectional workload. The original 5 ms Pi compute entries cannot describe serial processing of every incoming 1 ms frame: that would accumulate a backlog. With the initial 1 kHz controller, benchmark the combined Pi readout/encoder workload for sustained service within the available 1 ms cycle, or explicitly choose and report a slower controller schedule while still consuming every neural bin.

TABLE 21
Timing terms for the remote-desktop loop; values marked as references are not guarantees

Term	Measurement boundary / requirement	Initial reference or acceptance basis
Spike delivery	Desktop bin ready to Pi application receive; includes container/VM, Ethernet and Wi-Fi	Measure distribution; former USB reference 0.3–0.5 ms
Pi readout	Receive/decode/filter spikes and generate wheel request	Benchmark actual service time and scheduling delay; sustain 1,000 bins/s
Wheel command delivery	Pi application send to Teensy complete-frame receipt	USB reference 0.2–0.3 ms; measure
Local I/O	Command receipt to local latch; timestamp actual optical acquisition separately	Initial 1 kHz I/O schedule; up to one tick of latch wait, plus execution
Sensor delivery	Timestamped sensor frame ready on Teensy to Pi application receive	USB reference 0.3–0.5 ms; measure
Pi feedback compute	Sensor history update and selected-channel encoding	Benchmark jointly with readout; initial feedback period 1 ms
Feedback delivery	Pi send to desktop simulator receive, including VM ingress	Measure distribution; former USB reference 0.2–0.3 ms
Generator acceptance	Desktop receipt to request entering input filter at a native-bin boundary	Up to one 1 ms bin under on-time scheduling; record VM/simulator deadline misses

Assemble latency from measured causal intervals, including scheduling waits; its inverse is not the controller frequency. Align the VM, Pi and Teensy clock domains, or report round-trip bounds. Input filtering, latent/mode response, readout filtering and motor lag remain separate contributions to the delay-margin experiment in Section C.5.

Validate both neuron counts with simultaneous bidirectional traffic, robot motion and bounded management telemetry. Report the packet-age/loss metrics in Table 19, queue depths and actual command application ages. Judge the measured delay/jitter profile against the declared line-following success criteria. Neither 1 kHz cadence nor adequate average bandwidth establishes a hard deadline; Wi-Fi and macOS/VM scheduling provide no hard sub-millisecond guarantee.

APPENDIX D
ROBOT-MOTION SIMULATION AND DELAY EXPERIMENTS

This retained simulation model is a future software/test environment, not an implemented robot simulator. The mechanical plant maps a steering command u to wheel motion, robot pose, and the next three sensor readings. Its minimum simulation state is the axle-center position (X, Y), heading θ, and actual wheel rim speeds v_L, v_R. The caster supports the chassis and follows its motion; it is not a steering actuator. Wheel separation L_w, effective rolling radii R_L, R_R, sensor forward offset ℓ_s, sensor spacing, line width, and nominal speed v_0 must be recorded as measured quantities or explicit simulation assumptions.

D.1 Geometry baseline from the September 15 CAD assembly

The historical simulation geometry is taken from `cad_parts/assembly_sep15/main_overview.FCStd`, with the robot assembled, rather than from the exploded copies. Table 22 supplies the simulation baseline. These are dimensions of the saved CAD, not measurements of tire deflection, optical response, or the completed robot. The wheel width/hub and sensor package envelopes remain illustrative.

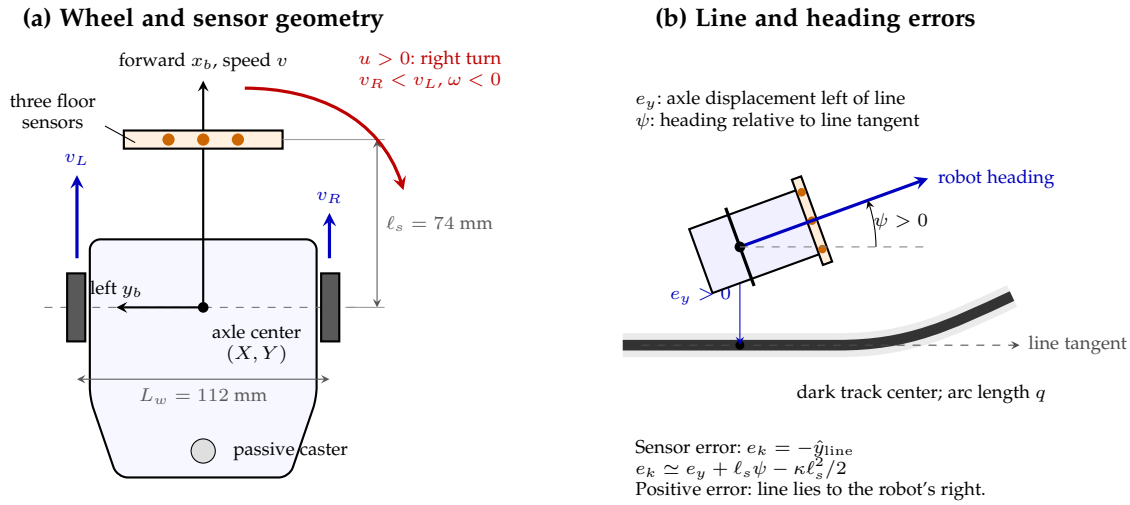
In the CAD frame, $+Y$ points toward the sensor bar, $+X$ toward the robot's right, and $+Z$ upward. The axle midpoint is $(0, 2, -5)$ mm. Thus the planar body coordinates used in the equations are $x_b = Y_{\mathrm{CAD}} - 2$ and $y_b = -X_{\mathrm{CAD}}$, in millimetres. The nominal floor is $Z_{\mathrm{CAD}} = -20$ mm, tangent to both wheel references and the caster ball.

TABLE 22
CAD-derived driving-model dimensions. Convert millimetres to metres in the simulation.

Quantity	Nominal value	Basis and interpretation
Wheel-center spacing L_w	112 mm	Wheel midplanes at CAD $X = \pm 56$ mm; this is the nominal contact spacing, not the 100 mm deck width.
Wheel radii R_L, R_R	15 mm each	30 mm diameter references; calibrate effective rolling radii under load.
Wheel envelope width	8 mm each	Illustrative width; total outside wheel span is 120 mm.
Sensor forward offset ℓ_s	74 mm	Reference face centers at CAD $Y = 76$ mm, axle at $Y = 2$ mm.
Sensor lateral coordinates	$+15.3,\ 0,\ -15.3$ mm	Left/center/right in the body frame; CAD channels A/B/C. Adjacent pitch 15.3 mm, outer-center span 30.6 mm.
Axle-to-caster distance	60 mm aft	Caster at body $(x_b, y_b) = (-60, 0)$ mm; axle at $(0, 0)$.
Lower-deck underside clearance	20 mm	Main plate underside at CAD $Z = 0$; hanging mounts and panels have smaller clearance.
Reference sensor-face clearance	18 mm	Downward faces at CAD $Z = -2$ mm; sensor-holder bottom is 9 mm above the nominal floor. Optical mounting height remains unverified.

Use $L_w = 0.112$ m, $R_L = R_R = 0.015$ m, $\ell_s = 0.074$ m and $y_j \in \{0.0153, 0, -0.0153\}$ m initially. Verify that wiring channels A/B/C occupy the correspondingly engraved CAD positions before applying this sign mapping to hardware. Line width, optical footprint, speed, motor gains/lag, and loaded tire behavior still require measurements or declared trial assumptions. In particular, the 18 mm reference face height is not a validated sensing distance; check the assembled optical height and response before selecting the reflectance model.

Figure 16 defines the mechanical geometry and sign conventions used below. Historical September 15 CAD views appear in Appendix G; the current mechanical package is described in Section 1.



Top view; wheel arrows illustrate positive steering.

Ground frame: θ and ω positive counterclockwise.

Fig. 16. Simulation geometry using the historical September 15 baseline (schematic, not to scale). The wheel-speed difference turns the robot; the forward sensor bar observes a combination of lateral displacement, heading error, and line curvature. Panel (b) defines the geometric errors; actual detection is limited by the three-sensor span.

D.2 Differential-drive kinematics and turning authority

Use a ground frame with counterclockwise-positive heading θ and a body frame with forward and left-positive axes. If Ω_L, Ω_R are wheel angular speeds, ideal rolling without slip gives

$$v_L = R_L \Omega_L, \qquad v_R = R_R \Omega_R, \tag{57}$$

$$v = \frac{v_L + v_R}{2}, \qquad \omega = \dot{\theta} = \frac{v_R - v_L}{L_w}, \tag{58}$$

$$\dot{X} = v \cos\theta, \qquad \dot{Y} = v \sin\theta. \tag{59}$$

Thus positive u still means a right turn, but has negative ω under this heading convention. With ideal wheel tracking of Equation (56),

$$v = v_0(1 - |u|/2), \qquad \omega = -\frac{v_0 u}{L_w}, \qquad \kappa_{\text{robot}} = \frac{\omega}{v} = -\frac{u}{L_w(1 - |u|/2)}. \tag{60}$$

Steering changes both yaw rate and forward speed. For a required clockwise curvature $\kappa_R = -\kappa_{\text{robot}}$, the ideal steady command is

$$u_{\text{req}} = \frac{L_w \kappa_R}{1 + L_w |\kappa_R|/2}, \qquad |\kappa_R| \le \frac{u_{\max}}{L_w(1 - u_{\max}/2)}. \tag{61}$$

At $u_{\max} = 1$, the current CAD baseline gives a minimum axle-center radius of $L_w/2 = 56$ mm and $|\kappa_{\text{robot}}| \le 17.86\,\text{m}^{-1}$. A curve tighter than this is geometrically infeasible under the chosen wheel law regardless of neural latency. Reducing v_0 lowers the required response bandwidth, but does not change this ideal curvature limit. These formulas describe axle-center motion; the forward sensor bar traces a different path during a turn. For example, at the 56 mm minimum axle radius the middle sensor traces a circle of radius $\sqrt{56^2 + 74^2} \simeq 92.8$ mm about the same instantaneous turn center. A feasible axle trajectory therefore does not by itself ensure that the line remains under the sensors.

As an explicit numerical example, choose $v_0 = 0.10\,\text{m/s}$ (a simulation setting, not a measured maximum). Straight motion requires $\Omega_L = \Omega_R = 6.67\,\text{rad/s}$, about 63.7 rpm. At $u = 1$, the ideal inner wheel stops, $v = 0.05\,\text{m/s}$ and $\omega = -0.893\,\text{rad/s}$.

D.3 Line-relative state and sensor geometry

Let q be arc length along the dark-track center, $\kappa(q)$ its counterclockwise-positive curvature, e_y the axle-center displacement to the left of the line, and ψ the robot heading minus the line tangent heading. In a region with a unique nearest line point and $1 - \kappa e_y > 0$,

$$\dot{q} = \frac{v \cos\psi}{1 - \kappa(q) e_y}, \qquad \dot{e}_y = v \sin\psi, \qquad \dot{\psi} = \omega - \kappa(q)\dot{q}. \tag{62}$$

On a straight line near alignment, $\dot{e}_y \simeq v\psi$ and $\dot{\psi} \simeq -v_0 u / L_w$. A lateral error alone therefore does not describe the control state: heading error determines how rapidly the line moves across the sensors.

Place sensor $j \in \{1,2,3\}$ at body coordinates (ℓ_s, y_j), with y_j positive left and normally symmetric about zero. Its world location is

$$p_j = \begin{pmatrix} X \\ Y \end{pmatrix} + \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} \ell_s \\ y_j \end{pmatrix}. \tag{63}$$

The simulator computes each reading from that location, line width, and sensor footprint; it must not supply true e_y or ψ to the Pi. For example, the dark-track response for a dark strip of width w_{line} with an effective lateral footprint width w_{spot} can use

$$z_j = \text{clip}\left(\frac{w_{\text{line}}/2 + w_{\text{spot}}/2 - |d_j|}{w_{\text{spot}}}; 0, 1 \right), \tag{64}$$

where d_j is signed distance from p_j to the nearest line center and $w_{\text{spot}} > 0$. This is a simple softened-edge model; replace it with a measured reflectance profile when available, and add illumination offset, noise, clipping, and sample timing separately.

After calibration over the actual light floor and dark track, let $r_j \in [0,1]$ denote calibrated reflectance (light floor: 1; dark track: 0), and define the dark-track weight $z_j = 1 - r_j$. The simulated z_j above follows this same convention; convert it to $r_j = 1 - z_j$ for the sensor interface. Appendix F.2 describes the current electrical wiring and raw readout; floor/track calibration remains to be measured. Map the physical module positions A, B and C to the chosen y_j coordinates before computing the centroid. A basic line centroid and the sensor error used in Equation (54) are

$$\hat{y}_{\text{line}} = \frac{\sum_j y_j z_j}{\sum_j z_j}, \qquad e_k = -\hat{y}_{\text{line}}, \qquad \sum_j z_j > z_{\min}. \tag{65}$$

Positive e_k means the line is to the right of the sensor-bar center and initially calls for positive u. Reject the estimate below the saved detection threshold $z_{\min}$; also flag broad or multiple peaks at crossings. Do not convert an undetected line into a zero error. For small heading error and slowly varying curvature,

$$e_k \simeq e_y + \ell_s \psi - \frac{1}{2}\kappa \ell_s^2. \tag{66}$$

At the CAD offset of 74 mm, a small heading error of 5° contributes approximately 6.46 mm to the sensor error. The leading curvature term for a 250 mm-radius line contributes about 10.95 mm in magnitude. Both are substantial relative to the 15.3 mm outer-center half-span. The 15.3 mm sensor pitch also leaves discrete gaps: a line can fall between sensors if the line and sensing footprints are too narrow. Simulate all three readings and their detection threshold; the 30.6 mm center span is not continuous guaranteed coverage.

The forward bar provides heading sensitivity and geometric preview, while curvature shifts its reading even when the axle is exactly on the line. A single centroid cannot separate e_y, ψ, and curvature. Timestamped sensor history helps the Pi respond to their evolution. It does not provide absolute position or independently measured wheel speed.

D.4 Actuator dynamics, traction, and chassis effects

First simulate ideal wheel speeds. Then replace each speed channel by a bounded empirical actuator model,

$$\tau_{m,j}\dot{v}_j + v_j = \text{clip}\big(g_j \max(0, v_j^*(t - T_j) - d_j^{\text{motor}}); 0, v_{j,\max}\big), \qquad j \in \{L, R\}. \tag{67}$$

Here $\tau_{m,j}$ is motor/load lag, T_j is command delay, g_j is effective gain, d_j^{motor} is a speed-equivalent deadband, and $v_{j,\max}$ is the speed limit. This is a model of PWM-to-motion behavior, not a claim that the encoder-free hardware closes a speed loop. Sweep unequal gains and time constants, supply-voltage dependence, and additional slip or external disturbances. At $u = 0$, unequal actual wheel speeds still give nonzero yaw.

The ideal kinematic model contains no mass or inertia because it prescribes wheel speeds. Their effects enter the real actuator response and traction. If force-level simulation is needed, a planar extension is

$$m_{\text{robot}}\dot{v} = F_L + F_R - F_{\text{res}}, \qquad I_z\dot{\omega} = \frac{L_w}{2}(F_R - F_L) - M_{\text{res}}, \qquad |F_j| \le \mu_{\text{fr}} N_j. \tag{68}$$

Here F_j denotes longitudinal tire force and N_j normal load. The coefficient μ_{fr} describes friction, and I_z is yaw inertia. The signed terms F_{res} and M_{res} represent rolling and caster resistance. Wheel torque and rotational dynamics must also be supplied for that extension; do not impose exact wheel-speed tracking simultaneously. Placing components low and close to the drive axle limits the contributions of their height and distance to load transfer and yaw inertia, while the caster must retain enough load for stable support. Tire compliance, caster friction, an uneven floor, and chassis pitch can change both yaw response and sensor height. Check these effects through measured response or bounded disturbances before attributing tracking failures to the neural model. The CAD-based estimate of about 138 g is printed plastic only; it is not m_{robot} and cannot determine I_z. Purchased components and their installed masses/locations must be added (Section G.3).

For a level floor and a centered lateral CoG, the 60 mm axle-to-caster lever arm gives the static approximation

$$\frac{N_C}{m_{\mathrm{robot}}g} = -\frac{x_{\mathrm{CG}}}{0.060}, \qquad N_L + N_R = m_{\mathrm{robot}}g - N_C, \qquad -0.060 < x_{\mathrm{CG}} < 0, \tag{69}$$

where x_{CG} is the forward-positive CoG coordinate in metres and N_C is caster load. Directly over the axle ($x_{\mathrm{CG}} = 0$), this ideal static model leaves the caster unloaded. Position the loaded CoG inside the three-contact support triangle and verify the resulting loads; the overview does not contain a measured full-robot CoG.

D.5 Mechanical interpretation of feedback delay

During an uncorrected interval T, a locally constant heading error and a newly encountered curve give leading-order error magnitudes of order

$$|\Delta e_y| \sim v|\psi_0|T + \tfrac{1}{2}|\kappa|v^2T^2, \qquad \Delta q \simeq vT. \tag{70}$$

At the declared straight speed of 0.10 m/s, the CAD offset corresponds to $\ell_s/v = 0.74\,\mathrm{s}$ of geometric preview; 100 ms of delay corresponds to 10 mm of forward travel. These are local estimates, not stability bounds. The geometric preview time is approximately ℓ_s/v when $v > 0$, but the three sensors do not reveal an arbitrary future curve, so preview is not a guaranteed latency allowance. The line must remain within the finite sensor span while the delayed neural response and wheel dynamics take effect. Sweep speed, curve radius, sensor offset/span, wheel mismatch, and delay jointly. Score axle-center error and sensor-line loss separately, using simulated pose only for evaluation. This distinguishes insufficient turning authority, loss of sensor visibility, actuator limitations, and delayed neural feedback.

APPENDIX E
THREE-SENSOR REFLECTANCE CALIBRATION

This appendix describes the calibration performed by `photo3_calib.py`. Place the robot symmetrically over a straight black tape on a light floor, with sensor B centered on the tape and the robot aligned along it. The robot turns slowly in place so that each sensor, A, B and C, repeatedly crosses both surfaces. The installed modules produce *high* ADC readings over black tape and *low* readings over the light floor. Each channel is calibrated separately to account for its pedestal and sensitivity.

E.1 Motion and Sampling

The two wheels receive equal PWM magnitudes with opposite physical directions. A left turn drives the left wheel backward and the right wheel forward; a right turn reverses those directions. Starting from the centered orientation, the sequence is:

1) Wait while stopped, then turn approximately 60° left to reach one end of the scan.
2) Make N one-way sweeps of approximately 120°, alternating right and left. The default $N = 6$ gives three sweeps in each direction.
3) Turn approximately 60° toward the original centered heading and stop. For even N, this final turn is right; for odd N, it is left.

There is a stopped pause after every motion segment. Sensor readings are recorded during motion; only the N full sweeps contribute to calibration statistics. Initial positioning and the final return are excluded. The target acquisition rate is 40 A/B/C triplets per second; the three channels are read sequentially.

The angular motion is controlled by timing. Let s be the PWM scale and T_{180} the observed time for a 180° turn at $s_0 = 0.5$. Unless a sweep duration is supplied explicitly, the nominal full-sweep time is

$$T_{120} = T_{180}\frac{120}{180}\frac{s_0}{s}. \tag{71}$$

With $T_{180} = 1.07\,\mathrm{s}$ and $s = 0.15$, $T_{120} \simeq 2.378\,\mathrm{s}$; the initial and final half-sweeps last about 1.189 s each. If T_p is the pause and T_c the initial countdown, the nominal sequence duration is

$$T_{\mathrm{sequence}} = T_c + (N+1)T_{120} + (N+2)T_p, \tag{72}$$

approximately 22.0 s at the defaults, before communication and report-writing overhead. Equation (71) assumes turn rate proportional to PWM. The sweep angle is approximate and depends on motor starting friction, floor contact and battery voltage. A measured `--sweep-time` overrides the estimate.

E.2 Separating White and Black Plateaus

For channel $i \in \{A, B, C\}$, pool its raw 10-bit ADC readings from the full sweeps and sort them as $v_{i,(0)}, \ldots, v_{i,(M-1)}$. Quantiles use linear interpolation: for $r = (M-1)p$ and $j = \lfloor r \rfloor$,

$$Q_i(p) = v_{i,(j)} + (r-j)\big(v_{i,(\min(j+1,M-1))} - v_{i,(j)}\big). \tag{73}$$

Define robust amplitude anchors and their span,

$$q_{i,L} = Q_i(0.02), \qquad q_{i,H} = Q_i(0.98), \qquad R_i = q_{i,H} - q_{i,L}. \tag{74}$$

The white and black sample sets are selected by

$$\mathcal{W}_i = \{v_i :\ v_i \le q_{i,L} + 0.20R_i\}, \tag{75}$$

$$\mathcal{B}_i = \{v_i :\ v_i \ge q_{i,H} - 0.20R_i\}. \tag{76}$$

These bands cover the bottom and top 20% of the *anchor amplitude span*. Readings between the bands are classified as transitions. The percentile anchors reduce sensitivity to isolated extrema when setting the bands, but values beyond the anchors are still included if they satisfy a band condition.

E.3 Reported Statistics and Acceptance

For either selected set $\mathcal{S} \in \{\mathcal{W}_i, \mathcal{B}_i\}$, report its sample count, mean and sample standard deviation:

$$n = |\mathcal{S}|, \qquad \mu = \frac{1}{n}\sum_{v\in\mathcal{S}} v, \qquad \sigma = \sqrt{\frac{1}{n-1}\sum_{v\in\mathcal{S}}(v-\mu)^2}. \tag{77}$$

Write these endpoints as $(\mu_{i,W}, \sigma_{i,W}, n_{i,W})$ and $(\mu_{i,B}, \sigma_{i,B}, n_{i,B})$. Here n is the number of selected readings. The reported σ describes variation among those readings, including sweep-to-sweep changes. Since samples are selected by amplitude and collected while moving, these deviations are practical estimates of endpoint variability.

A channel is accepted only when all of the following hold:

- The anchor span R_i is at least `--min-contrast`, default 80 ADC counts.
- Each white and black set contains at least eight readings.
- Each surface contributes at least two readings in at least $\max(2, \lceil N/2 \rceil)$ distinct sweeps. At $N = 6$, both surfaces must recur in at least three sweeps.
- No more than half of all full-sweep readings lie between the two endpoint bands.

The overall calibration is valid only after a completed run with all three channels accepted and the final motor stop acknowledged. An interruption stops the robot at its current heading. The saved file then holds the summary of whatever full sweeps finished, without the raw readings.

The resulting endpoint means and deviations feed the normalization and uncertainty model in Section 4. Each run replaces `/home/pi/plant_lineFollower/config/photo3_calib.json`. That file stores the printed summary: for each sensor, the white and black mean, sample standard deviation and count, plus validity, the motor polarity, the boot `hash8`, the Pi local time and seconds since boot. It does not store individual ADC readings.

E.4 Parameters

TABLE 23
Calibration parameters. Speed denotes PWM fraction.

Parameter	Default	Meaning
`--speed`	0.15	Equal turn PWM magnitude; $0 < s \leq 1$, with nonzero rounded PWM.
`--sweeps`	6	Number of one-way full sweeps; at least 2.
`--turn180`	1.07 s	Reference half-turn duration at PWM scale 0.5.
`--sweep-time`	Estimated	Explicit full-sweep duration at the chosen speed; replaces Eq. (71).
`--rate`	40 Hz	Target sampling and drive-refresh rate; 10–100 Hz.
`--pause`	0.3 s	Stopped pause after each segment; nonnegative.
`--countdown`	3 s	Stopped delay before starting motion; nonnegative.
`--min-contrast`	80 counts	Minimum anchor span; greater than 0 and at most 1023.
`--polarity`	`rr`	Left/right wiring signs: `ff`, `fr`, `rf`, or `rr`.
`--port`	`/dev/serial0`	Pi–Teensy UART device.
`--check`	False	Verify readout while keeping the robot stopped.

APPENDIX F
BENCH-TEST HISTORY

These records preserve earlier motor and sensor tests. Their individual test programs, rates and USB-era behavior are distinct from the current dual-motor UART line follower in Section 4. Pin-level diagrams have been consolidated in Section 3.

F.1 Motor-B Bench Prototype (September 13)

This appendix records the September 13, 2026 motor-B prototype. Figure 4 and its power notes were updated on September 23 for the shared battery and UART wiring. Current `drive_motorB.py` uses UART with Teensy firmware revision 9; the software/test account below retains the original USB setup with revision 3. Its client is preserved as `drive_motorB_usb.py`. Teensy generates PWM and direction signals; TB6612FNG ("TB66") supplies motor current. Motor B is the right wheel; motor A remains undriven in this test.

F.1.1 Electrical Connections and Power Paths

Current wiring is shown in Figure 4 in the main electrical section.

Battery and bucks. The ammeter is in series between the battery plug negative terminal and common ground. The two series cells supply 7.4 V nominal (8.4 V fully charged for 4.2 V cells); use compatible 2S protection/charging. S1 feeds both branches; opening S2 removes motor power while leaving the 5 V logic supply on. The motor buck feeds VM at 6.0 V. The logic buck feeds Pi and Teensy Vin in parallel at 5.0 V, replacing the independent Pi supply. Converter models, current capacities and motor stall current remain unrecorded. The motor buck may lose regulation as the pack discharges; dropout depends on converter and load (TI buck example). Discharge/load performance has not been characterized here.

Logic and returns. Cutting Teensy's USB power bridge isolates VUSB from Vin. Its regulator still feeds TB6612 VCC at 3.3 V, not 5 V. The diagram identifies UART TX/RX pins; current firmware uses 115200 baud. Pi, Teensy, driver and both converter grounds join the ammeter's load-side common return. Route motor-current return directly to the motor buck, not through Teensy's signal ground. Retain local bypassing and motor suppression (Section 3). The 6 V rail feeds VM only. See the PJRC Teensy reference for 3.3 V I/O and the Toshiba datasheet for VM/VCC.

F.1.2 Pin Assignment and Drive States

TABLE 24
Implemented motor-driver connections. Numbers are Teensy GPIO labels, not header-position counts.

TB6612 signal	Connection	Purpose / present operation
PWMA	Teensy 3	Channel A PWM input; held HIGH while B runs, with both A direction inputs LOW
AIN1, AIN2	Teensy 4, 5	Both LOW: channel A remains high impedance when PWMA and STBY are HIGH
STBY	Teensy 6	HIGH enables the driver; LOW puts both channels in standby
BIN1, BIN2	Teensy 7, 8	Forward command: BIN1 HIGH, BIN2 LOW
PWMB	Teensy 9	20 kHz hardware PWM, 8-bit duty command 0–255
VCC	Teensy 3.3 V	Logic supply; separate from VM
VM	Buck OUT+	Regulated 6 V motor supply
GND	Common ground	Teensy GND, buck IN-/OUT-, and pack negative
AO1 / AO2	Left motor terminals	Channel A; not driven by this test program
BO1 / BO2	Right motor terminals	Channel B; forward direction depends on lead polarity

The program maps percentage s to duty $d = \mathrm{round}(255s/100)$. At 50%, $d = 128$. During PWM HIGH, channel B applies forward drive; during PWM LOW it uses the driver's short-brake state. `STOPB` instead pulls STBY LOW, giving high-impedance outputs so the motor coasts. PWM duty is not measured wheel speed; no encoder feedback is implemented. Pin 9's hardware PWM is configured with `analogWriteFrequency`, as described in the PJRC PWM documentation. The bridge states follow the Toshiba control table cited above.

Sensor integration. The September 13 motor-only firmware described here has since been extended. Modules A, B and C now connect to A11, A12 and A13 (GPIO 25, 26 and 27). Installed firmware revision 9 uses this mapping; the sensor bench subsection F.2 records the 5 V sensor supply and 3 Hz UART readout. Pins 3–9 retain the motor assignments above.

F.1.3 Software Operation and Command Lifetime

The Pi discovers the Teensy USB serial device (currently `/dev/ttyACM0`), opens it exclusively, checks `VERSION`, then sends `STOPB`. It sends newline-terminated ASCII `MOTORB d` commands approximately every 100 ms while driving and requires an `OK` reply to each. This roughly 10 Hz bench protocol is separate from the future 1 kHz binary controller interface in Table 20; 20 kHz PWM runs locally between USB commands.

Firmware starts in standby. USB disconnection or more than 750 ms without a valid nonzero motor command returns it to standby. `STOPB`, zero duty and invalid commands also stop drive. `STATUSB` reports commanded duty and standby state; it does not measure terminal voltage, current or rotation. The Python script attempts `STOPB` on completion, Ctrl+C, termination or communication errors. Only one serial-control program should run at a time. The earlier onboard-LED and pin-41 LED commands remain supported.

The original test used a historical login. For current access use `ssh pi2`. The following preserved command examples require an explicit bench-test setup:

```
python3 ~/robot0/drive_motorB_usb.py --check
python3 ~/robot0/drive_motorB_usb.py --speed 50 --seconds 3
python3 ~/robot0/drive_motorB_usb.py --speed 50
```

The first command checks communication and leaves the driver stopped. The second runs for three seconds; the third continues until Ctrl+C. Lift the wheel for bench tests. If the physical direction is reversed, turn off motor power before exchanging the BO1/BO2 motor leads.

F.1.4 Observed Test and Electrical Checks

Both three-second and continuous motor-B runs succeeded after soldering a previously unsoldered row of breakout-board header pins. Before that repair, USB commands were acknowledged but the motor did not turn. This confirms why a software acknowledgement alone is not an electrical test.

For a voltage check, run at 100% PWM while measuring at the driver board, relative to its GND: VCC should be about 3.3 V, VM about 6 V, STBY/BIN1/PWMB about 3.3 V and BIN2 near 0 V. BO1 should be near VM and BO2 near ground, allowing for driver voltage drop. Measure while the command is active; afterward standby is expected. With all power removed, check common-ground continuity and soldered header connections. Motor-A operation, stall current, regulator load response and the firmware timeout have not been physically characterized in this test.

F.2 Three-Sensor Bench Readout and Wiring History

This appendix records the three-module prototype, with wiring updated on September 23, 2026. The modules are named **A, B and C**; their analog outputs (AO) now connect to Teensy **A11, A12 and A13** (GPIO **25, 26 and 27**), respectively. The September 14 bench readout used the earlier GPIO 21–23 mapping. These names identify wiring channels. In the September 15 CAD layout, the engraved A/B/C positions are left/center/right when looking forward toward the sensor head, with body lateral coordinates $+15.3, 0, -15.3$ mm (Table 22). Verify that the wired modules occupy those positions; the earlier bench test established acquisition, not installed channel placement.

F.2.1 Electrical Schematic

Current wiring is shown in Figure 5 in the main electrical section.

Voltage and ground. All three module VCC pins now receive 5 V directly from the logic buck, alongside Pi and Teensy Vin. Teensy's 3.3 V output continues to power TB6612 VCC, not the sensors. The PJRC Teensy 4.1 reference specifies a fixed 0–3.3 V analog-input range and no 5 V tolerance. A module's 5 V supply does not establish that its AO is safe for Teensy: verify the AO range before direct connection, and add suitable attenuation if it can exceed 3.3 V. No attenuation circuit has been specified or shown here. Join sensor, Pi, Teensy, driver and buck grounds; route motor current directly to the motor buck rather than through sensor ground leads.

F.2.2 Current Pin Labels

TABLE 25
Current module connections. GPIO numbers are Teensy's printed pin labels, not physical header-position counts.

Module	Module pin	Connection	Use
A	AO	25 / A11	First raw value, printed as `A`
B	AO	26 / A12	Second raw value, printed as `B`
C	AO	27 / A13	Third raw value, printed as `C`
A, B, C	VCC	5 V buck OUT+	Shared supply with Pi and Teensy Vin, connected in parallel
A, B, C	GND	GND	Common signal and power reference
A, B, C	DO	Not connected	Digital comparator output not used

The aliases are **25 = A11, 26 = A12, 27 = A13**, as shown on the PJRC pinout card. AO denotes each module's analog output; it does not mean Teensy ADC A0. The motor allocation remains PWMA=3, AIN1=4, AIN2=5, STBY=6, BIN1=7, BIN2=8 and PWMB=9; it does not overlap these ADC pins. The older single-sensor test on GPIO 14 / A0 remains supported in firmware but is not part of the three-module wiring above.

F.2.3 Readout from the Pi

Current firmware and transport. Revision 9 was installed from the Pi on September 23. It reads modules A/B/C on A11/A12/A13 (GPIO 25/26/27), accepts UART commands at 115200 baud and uses pin 31 for the test LED. Nine UART sensor readings completed successfully with both motors in standby; this verifies acquisition, not the modules' full AO voltage range or reflectance calibration.

The `ADC3` command reads the three configured pins sequentially with 10-bit resolution, then returns one line:

```
ADC3 <A_raw> <B_raw> <C_raw>
```

Each value is an integer from 0 to 1023. The channels are acquired sequentially, not simultaneously. No conversion to volts or calibrated reflectance is performed by the three-sensor client.

After SSH to the Pi, run:

```
cd ~/robot0
./photo3_teensy25-27.py
# Optional finite check: nine sets, approximately three seconds
./photo3_teensy25-27.py --samples 9
```

The program prints one timestamped A/B/C set at **3 Hz**, approximately every 0.333 s, until Ctrl+C. A nine-set bench check verified all three values and the expected sampling intervals; this verifies acquisition, not floor/track calibration. The first set is read immediately, so nine timestamps span about 2.67 s.

To also save readings in the designated output directory:

```
cd ~/robot0
set -o pipefail
./photo3_teensy25-27.py 2>&1 | tee -a out/teensy41/photo3_readings.log
```

The Python script remains in `robot0`; shared firmware is under `robot0/teensy41/firmware/halo_teensy_firmware/`, and captured readings and checks belong under `robot0/out/teensy41/`. Run only one Teensy serial client at a time. The sensor reader sends `STOPB` at startup; in the current firmware this stops both motors. ADC requests do not refresh the 750 ms motor-command watchdog. This reader is a stationary bench test, not simultaneous closed-loop motor and sensor control.

Interpreting the readings. The breakout modules provide their existing analog signals; the current three-wire-per-module connection provides no independent emitter switching. No emitter-off subtraction is implemented. Compare the raw values on the actual dark track and light floor at a fixed height to establish each module's polarity, usable range and channel differences. Do not assume that a higher raw count universally means a darker or more reflective surface.

APPENDIX G
HISTORICAL CAD BASELINE AND MASS ESTIMATES

This appendix preserves the September 15 mechanical design and its estimates. Its dimensions, views and mass budget are historical; they do not describe the September 15 September 24 package or its later roof.

The September 15 design is a two-level differential-drive chassis with a rounded, rear-tapered deck outline. The working package is `cad_parts/assembly_sep15/`; its `main_overview.FCStd` combines the September 15 lower deck, photo-sensor holder, two T joints, both pillar frames, and upper electronics deck. It contains 16 plastic parts and 24 illustrative hardware/electronics objects, each included once in the assembled view. The exploded group repeats these objects only for inspection. Nominal control dimensions are given in Table 22.

G.1 Chassis 3D Printing & PETG

PETG is the September 15 printing material. The overview estimates use prior QIDI Q2 slice records with a 0.4 mm nozzle, 0.20 mm layers, four walls and 35% infill, scaled by September 15 component-family CAD volume. These assumptions supersede the earlier three-wall, 15–20% infill concept in this document. They do not constitute a newly sliced full-robot job. Each component still requires its own suitable orientation and supports; the revised pillar lettering/trim has not yet been sliced in a newly approved orientation. The assembled overview is for inspection, not printing.

G.2 Physical Stack & Component Placements

- **Lower deck:** A nominal $100 \times 105 \times 4$ mm plate carries the drive-motor mounts below it, rear caster mount, Raspberry Pi mounting/outline features, and an 18650 placement outline. The rear switch plate tilts outward by 20° and the power-plug bore is 8.4 mm. The modeled Pi and battery markings locate intended components; they are not complete purchased-part solids or a verified wiring/clearance arrangement.
- **Pillars and head joint:** Each pair is one braced part with two nominal 16×10 mm posts, two horizontal links and two crossed 3×3 mm links. Front posts are 42 mm tall above the 3 mm T joints; aft posts are 45 mm tall directly above the lower deck. All four tops meet at CAD $Z = 49$ mm. Their ends have 2 mm blind screw pilots, 8.2 mm deep; these are no longer insert pockets. The outer faces are trimmed to the deck outline and the pairs have raised F/A identification.
- **Upper deck:** The matching nominal 100×105 mm floor is 3 mm thick at CAD $Z = 49 \ldots 52$ mm. It has four 3 mm pillar mounting holes, engraved pillar outlines, an integrated Teensy holder with a 30×50 mm floor opening, and two low end walls. The Teensy reference PCB underside is 7 mm above the floor top. The USB cable support, separate cable-clamp cap and board-retaining clamp are included. Power A/B locations, the ammeter display opening and screw holes, and two hollow 30° motor-driver supports occupy the surrounding plate; electronics envelopes are illustrative.
- **Sensor head:** A 70 mm-wide front holder is connected by the two T joints. It carries three optical-module references with 15.3 mm pitch and centers 74 mm ahead of the drive axle. The holder's overall installed depth is 51 mm including its arms. Verify physical sensing height and channel placement before driving.

The lower-deck top is 24 mm above the nominal floor; the upper floor underside/top are 69/72 mm above it. The clear deck-to-deck gap, from the lower-deck top to the upper-floor underside, is 45 mm. The complete modeled envelope is approximately $120 \times 161 \times 101.2$ mm (width $\times$ length $\times$ height), including wheel references, the forward sensor head, rear switch projection and cable-clamp cap. This envelope does not account for unmodeled battery packs, wiring loops or purchased-part tolerances.

G.3 Mass Budgeting and Gravity Metrics

The overview's September 15 estimate is **138 g of finished plastic**, with a rough planning range of 111–166 g. Total PETG usage including the scaled support/brim allowance is about **143 g**; the estimated sum of separate print-job times is **14.2 h**, with a rough 9–19 h planning range. These are scaled historical slicer estimates, not measured print durations or fresh slicing of the September 15 assembly. Hardware, electronics, cables, batteries and fasteners are excluded from the plastic mass, and assembly labor is excluded from time.

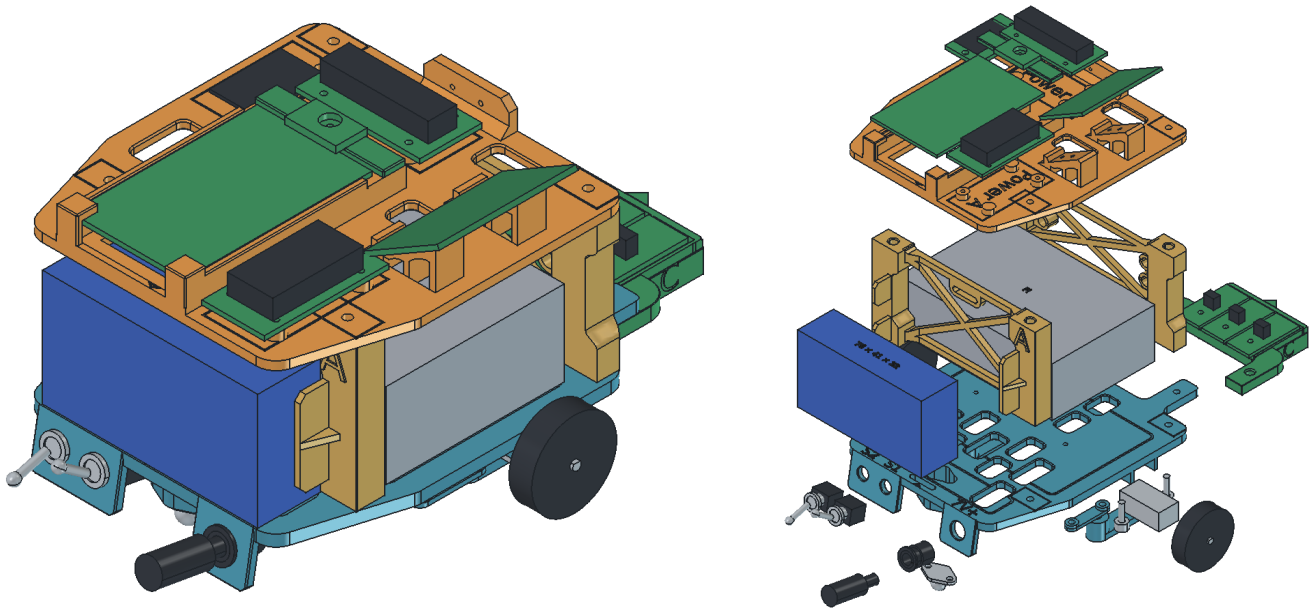
TABLE 26
Approximate PETG budget for the September 15 CAD parts. Rounded entries may not sum exactly.

Component family	Plastic parts	Finished plastic (g)	Printing (h)
Lower deck, motor clamps and cable clips	8	49.4	5.4
Photo-sensor holder	1	11.7	1.4
T head joints	2	3.9	0.5
Front and aft pillar frames	2	27.2	2.4
Upper deck and two clamps	3	46.2	4.5
Total	16	138.4	14.2

The earlier 450–550 g whole-robot mass target remains a planning target, not a CAD-derived or weighed total. At the central plastic estimate, it would leave approximately 312–412 g for all nonprinted components. Their masses and installed positions have not been fully recorded, so the overview cannot establish total mass, CoG or yaw inertia. The lower placement of the Pi/batteries and upper placement of the Teensy/driver supersede the older conceptual stack drawing. Check the loaded support triangle and caster load using Equation (69), rather than assuming that a CoG exactly over the axle is a verified balanced configuration.

G.4 September 15 CAD Visualization

Figure 17 uses the assembled and exploded views exported from `main_overview.FCStd` in `2026_plant_3Dprint_designs/lineFollower`. Lower-deck parts are dark teal, pillars gold, upper deck orange, sensor mount and electronics green, wheels and clamps dark, and battery/Pi mockups blue and silver.



(a) Assembled robot

(b) Exploded arrangement

Fig. 17. CAD assembly from `main_overview.FCStd` in `2026_plant_3Dprint_designs/lineFollower`. Both views show the same components; exploded spacing is illustrative and the two views use different scales. These images are inspection views, not print layouts.

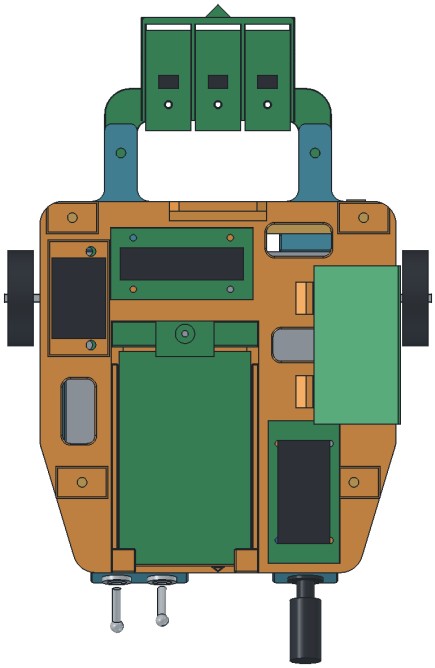


Fig. 18. Top view from `main_overview.FCStd` in `2026_plant_3Dprint_designs/lineFollower`, with the sensor head toward the top. The upper deck shows the Teensy holder and retaining clamp, Power A/B mounting areas, ammeter opening on the left, and inclined driver supports on the right. The lower Pi and battery mounting features are beneath this level.

The native overview opens with `Assembled` visible. Hide that group and show `Exploded` to inspect separated parts; double-click `Page1` or `Page2` for the embedded vector drawing sheets. The separate report is `cad_parts/assembly_sep15/overview/robot_overview.pdf`. Snapshot dimensions, source CAD hash and mass assumptions for this document are recorded in `docs/figures/cad_sep15_geometry.json`; figures are copied locally so the TeX document does not depend on a running FreeCAD instance.